Technical Report

OFDM Transceiver Prototype

An Adaptive Audio-Band OFDM Transceiver for Amateur HF Radio —
Article Reproduction, Sensitivity Extension to −17.9 dB,
Adaptive Link Layer, and a Fixed-Point RTL Reference Model

Author

Ivan Gubochkin igubochkin@gmail.com

September 3, 2026

Abstract

This report documents a complete software prototype of an audio-band OFDM digital communication system for amateur HF radio, developed in Python (NumPy/SciPy) as a faithful reproduction — and then a substantial extension — of the PHY layer published by bashkirtsevich [2]. The reproduction is bit-exact against every worked example in the article (44/44 checks: CRCs, Base38/QTH encodings, convolutional-code outputs, scrambler and interleaver sequences) and reaches $-7.2\,\mathrm{dB}$ sensitivity versus the article's $\approx -7.5\,\mathrm{dB}$ on the same channel model; the residual gap was diagnosed as synchronization loss, not the decode chain. Three synchronization defects found during the diagnosis — a Zadoff–Chu time–frequency ambiguity worth $\sim$1 dB, an SNR-dependent tone-detection metric, and a preamble transmitted 7 dB below data-symbol power — are analysed and corrected.

The waveform is then extended with SNR-adaptive link modes that scale the coherent symbol-tiling factor ($4\times$ / $16\times$ / $64\times$), lowering the sensitivity floor to $-17.9\,\mathbf{dB}$ in the 6 kHz band[1] (78 % of Shannon capacity at $-20\,\mathrm{dB}$); with a measured monotone LLR recalibration worth 1.5–2 dB at the sensitivity edge; with an IRA LDPC code, Gray-mapped 16-QAM, and CRC-gated chase-combining HARQ; and with a complete link layer: a 13-rung rate ladder spanning 7.8–1059 bit/s over $-17.9\ldots+4.7\,\mathrm{dB}$, receiver-driven rate adaptation through a 20-bit piggybacked link-control word, QoS with fragment-boundary preemption, simplex channel access, and closed-loop AFC frequency netting that pulls a $+284\,\mathrm{Hz}$ initial offset inside a 12 Hz deadband in five exchanges. An SSB RF-chain simulator with per-station reference oscillators makes carrier offsets emerge from oscillator physics rather than parameters, and an integer-only fixed-point twin of the receiver — Q15 arithmetic, block-floating-point FFT, NCO/CORDIC primitives, division-minimised channel estimation — serves as a golden reference for a future RTL implementation, matching the floating-point receiver within 0.5 dB and surpassing it at $-9/-10\,\mathrm{dB}$. All results are backed by self-verifying regression suites (44 article checks, 10 end-to-end cases, 24 fixed-point checks, whole-system simplex and AFC scenario tests) and by a blind-decoded WAV demonstration of a complete two-station session over the fading RF channel.

[1] Throughout this report, SNR follows the article's convention: total signal power versus total noise power in the full 6 kHz Nyquist band of the 12 kHz-sampled audio. The signal itself occupies only $\approx$2.1 kHz (300–2400 Hz), so the same operating point reads $\approx$4.5 dB higher against in-band noise only, and $\approx$3.8 dB higher in the ham-standard 2.5 kHz convention (-17.9 dB here ≈ -14.1 dB ham).

Contents

List of Tables

List of Abbreviations

Abbreviation	Definition
ACK	Acknowledgement
AFC	Automatic Frequency Control
AGC	Automatic Gain Control
ARQ	Automatic Repeat reQuest
AWGN	Additive White Gaussian Noise
BEC	Binary Erasure Channel
BER	Bit Error Rate
BFP	Block Floating Point
BPSK	Binary Phase-Shift Keying
BSC	Binary Symmetric Channel
CFO	Carrier Frequency Offset
CP	Cyclic Prefix
CRC	Cyclic Redundancy Check
DFT/FFT	(Fast) Discrete Fourier Transform
FEC	Forward Error Correction
FIR	Finite Impulse Response (filter)
HARQ	Hybrid ARQ (soft-combining retransmission)
RTP	Real-time Transport Protocol (IETF RFC 3550)
HF	High Frequency (3–30 MHz)
IRA	Irregular Repeat–Accumulate (LDPC construction)
LC	Link Control (word)
LDPC	Low-Density Parity-Check (code)
LFSR	Linear-Feedback Shift Register
LLR	Log-Likelihood Ratio
LO	Local Oscillator
LPF	Low-Pass Filter
MMSE	Minimum Mean-Square Error (equalizer)
NCO	Numerically Controlled Oscillator
NVIS	Near-Vertical-Incidence Skywave
OFDM	Orthogonal Frequency-Division Multiplexing
PAPR	Peak-to-Average Power Ratio
PER	Packet Error Rate
PHY	Physical Layer
ppm	Parts Per Million
QAM	Quadrature Amplitude Modulation
QoS	Quality of Service
QPSK	Quadrature Phase-Shift Keying
QSB	Signal fading (amateur-radio Q code)
QSO	Two-way radio contact (amateur-radio Q code)
RTL	Register Transfer Level
SNR	Signal-to-Noise Ratio
SQNR	Signal-to-Quantisation-Noise Ratio
SSB	Single-Sideband (modulation); USB = upper sideband
STF	Short Training Field (IEEE 802.11 preamble style)
TCXO	Temperature-Compensated Crystal Oscillator
ZC	Zadoff–Chu (sequence)
ZF	Zero-Forcing (channel estimate)

1 Introduction

1.1 Background

The starting point of this work is the Habr article *"Development of a digital amateur-radio protocol based on OFDM: the PHY layer"* by bashkirtsevich [2]: an audio-band OFDM modem designed to be fed into the microphone input of an ordinary SSB transceiver. The article specifies the complete physical layer — a 128-bin OFDM numerology in the 300–2400 Hz audio band, Zadoff–Chu pilots [6], a rate-1/3 $K = 7$ convolutional code [19] with puncturing, packet framing with CRC protection, a two-stage synchronization preamble, and a four-fold symbol-tiling scheme for coherent SNR gain — together with worked numeric examples, an air-channel simulator, simulated BER/PER curves, and a 14 km over-the-air test log.

The article is a rich but informal specification: several implementation details are omitted, one convention (the LLR sign) is stated inconsistently with the article's own code, and the published sensitivity figures depend on synchronization details the text does not fully pin down. Reproducing it independently is therefore a non-trivial exercise in reverse engineering as much as in DSP implementation.

1.2 System Overview

Figure 1 shows the system as it stands at the end of the work reported here. What began as a PHY reproduction has grown into a four-layer communication system, implemented as the pure NumPy/SciPy package `ofdm_phy` with no build step:

- **PHY** — the OFDM modem (`ofdm.py`), the frame transceiver (`transceiver.py`), and the bit pipeline (FEC, interleaver, scrambler, packets, CRC, PAPR reduction);
- **link layer** — a rate ladder and receiver-driven adaptation controller (`link.py`) and a full station with QoS queues, ARQ/HARQ, simplex channel access, and AFC netting (`station.py`);
- **RF/channel** — the article's audio-domain channel simulator (`channel.py`) and an SSB RF chain with per-station reference oscillators (`rf.py`);
- **fixed-point twin** — an integer-only reimplementation of the receiver (`ofdm_phy/fixed/`) structured as an RTL datapath, cross-validated against the float model.

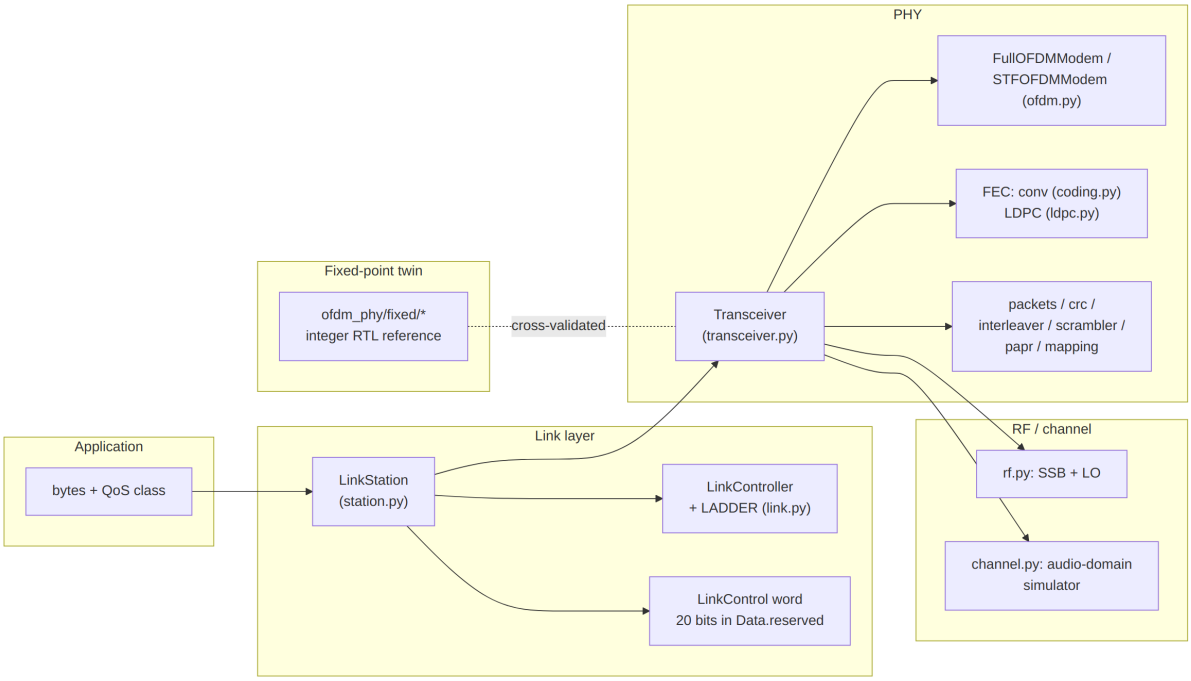


Figure 1: Layered architecture of the prototype. The simulation-side layers (`channel.py`, `rf.py`) are exactly what a real deployment replaces with a soundcard and an SSB transceiver.

Three design principles shaped every layer. First, *faithful defaults*: the plain `Transceiver` reproduces the article's behaviour exactly (convolutional FEC, raw LLRs), and every improvement — LLR recalibration, LDPC coding, HARQ, AFC — is opt-in and enabled at the link layer, so the article-reproduction experiments remain valid permanently. Second, *self-describing frames*: each link mode owns a distinct Zadoff–Chu preamble root, so the receiver identifies the mode from the preamble itself and a mode switch can never deadlock the link. Third, *everything measured feeds back*: per-frame SNR drives rate adaptation, per-frame CFO drives AFC netting, failed decodes export their LLRs for HARQ combining, and every ACK or timeout nudges learned per-rung sensitivity offsets.

1.3 Scope of This Report

This report documents the complete prototype in `ofdm_transceiver_proto`: the article reproduction and its verification, the synchronization analysis that closed the sensitivity gap, and every subsequent extension. All quantitative claims are backed by scripts in `experiments/` (Section 11); the measured artefacts (curves, JSON sweeps, WAV recordings) live in `results/`. Developer-oriented documentation with the same structure exists as Markdown with Mermaid diagrams in `docs/`: the layer documents (architecture, PHY, link, RF, fixed-point) alongside four interface documents written for whoever implements *against* the system rather than inside it — the channel drivers, the two consoles, the USB transport, and the modem's command/event model (Section 12.16).

1.4 Contributions

- **Bit-exact article reproduction** — 44/44 worked examples verified; BER/PER curves (article Figures 23–24) reproduced on the article's channel model; measured sensitivity $-7.2\,\mathrm{dB}$ versus the article's $\approx -7.5\,\mathrm{dB}$, with the residual attributed to synchronization by a genie-synchronized control measurement.
- **Synchronization analysis and repair** — three defects identified and fixed: the Zadoff–Chu time–frequency ambiguity ($\sim$1 dB), an SNR-dependent tone-detection metric, and a preamble transmitted 7 dB below data power. An alternative 802.11-style preamble (proposed in the article's comment thread) was implemented and shown statistically equivalent.
- **SNR-adaptive modes** — 16× and 64× tiling modes with per-symbol residual-CFO hypothesis search, group-coherent ZC detection, and per-mode preamble roots, extending the sensitivity floor to $-17.9\,\mathrm{dB}$ (78 % of Shannon capacity at $-20\,\mathrm{dB}$).
- **LLR recalibration** — a measured monotone reliability map correcting a shape (not temperature) miscalibration of the demodulator's LLRs, worth 1.5–2 dB at the BPSK/QPSK sensitivity edge.
- **Coding and modulation extensions** — an IRA LDPC code (N=768, K=256) with normalized min-sum decoding, Gray-mapped 16-QAM extending the ladder to $+4.7\,\mathrm{dB}$ and 1059 bit/s, and CRC-gated chase-combining HARQ.
- **Adaptive link layer** — a 13-rung measured rate ladder, receiver-driven adaptation over a 20-bit piggybacked link-control word, QoS with fragment-boundary preemption, stop-and-wait ARQ, and simplex channel access with carrier sense; validated by a whole-system two-station test with a mid-transfer fade.
- **AFC frequency netting** — closed-loop carrier netting through the LC word ($+284\,\mathrm{Hz}$ initial offset converged inside a 12 Hz deadband in five exchanges), with trim-budget and anchor guard rails against common-mode frequency drift.
- **RF-chain simulator** — SSB modulator/product detector at a 48 kHz IF rate with one shared reference oscillator per station, making CFO emerge physically from ppm errors and thermal drift (validated to 0.1 Hz).
- **Fixed-point RTL reference model** — an integer-only receiver covering the full frame family (conv+LDPC, BPSK/QPSK/16-QAM, all modes, HARQ, calibrated LLRs), within 0.5 dB of the float receiver at $-7\,\mathrm{dB}$ and ahead of it at $-9/-10\,\mathrm{dB}$.

1.5 Signal Profile at a Glance

Table 1 summarizes the on-air appearance of the waveform in the style of a signal-identification database entry [1], for quick comparison with established sound-card HF modes such as VARA HF.

Table 1: Signal identification profile (sigidwiki-style summary).

Field	Value
Frequency range	any SSB voice channel (the waveform lives in the 300–2400 Hz audio passband); amateur HF in the source article's context
Mode	USB (audio passband, AGC off)
Modulation	OFDM, 23 subcarriers (16 data + 7 Zadoff–Chu pilots), BPSK / QPSK / 16-QAM
FEC	convolutional K=7 rate 1/3 punctured to 1/2, 2/3, 3/4 (soft Viterbi); optional IRA LDPC
Bandwidth	≈2.1 kHz occupied (bins 3–25 at 93.75 Hz spacing)
Symbol rate	22.1 / 5.8 / 1.46 symbols/s (NORMAL / ROBUST / EXTREME tiling)
ACF	10.7 ms (intra-symbol tile); 45.3 / 173 / 685 ms (symbol, mode-dependent)
Net data rate	7.8–1059 bit/s, 13-rung adaptive ladder (Table 6)
Sensitivity	down to −17.9 dB SNR (noise in the 6 kHz Nyquist band; ≈ −14 dB re 2.5 kHz)
Identification	three-part preamble: two Newman-phased tone combs, then a tiled Zadoff–Chu symbol whose root (17/19/21) labels the link mode; the EXTREME bootstrap is heard as a ≈21 s steady warble
Duplex / access	simplex; carrier sense, stop-and-wait ARQ (selective-repeat burst extension in the C implementation, Section 12)
Status	research prototype (this work): bench and virtual-channel operation; not a deployed on-air standard
Short description	audio-band adaptive OFDM data modem for SSB transceivers, reproducing and extending the PHY of [2]

1.6 Report Structure

Section 2 describes the PHY layer as implemented. Section 3 presents the synchronization analysis that closed the sensitivity gap. Section 4 describes the SNR-adaptive link modes. Section 5 covers LLR quality measurement and recalibration. Section 6 describes the LDPC, 16-QAM, and HARQ extensions. Section 7 presents the link layer, and Section 8 the AFC netting loop. Section 9 documents the RF-chain and channel simulators. Section 10 describes the fixed-point reference model. Section 11 consolidates the validation and benchmark results. Section 12 documents the portable C implementation, its microcontroller feasibility, and the demonstration infrastructure. Section 13 discusses limitations and future work, and Section 14 concludes.

2 PHY Layer

2.1 Numerology

The waveform is designed to pass through an ordinary SSB transceiver's audio path (AGC off). Table 2 lists the parameters, all taken from the article [2].

Table 2: OFDM numerology.

Parameter	**Value**
Sample rate	12 kHz
FFT size	128 bins → 93.75 Hz subcarrier spacing
Occupied band	300–2400 Hz → bins 3–25, 23 subcarriers
Pilots	7, Zadoff–Chu root 3, bins 3, 6, 10, 14, 17, 21, 25
Data carriers	16
Cyclic prefix	32 samples (25 %)
Symbol tiling	4× / 16× / 64× by link mode (+6 dB per 4×, coherent)
CFO tolerance	±375 Hz design, ±300 Hz specified

Two consequences of that grid are worth stating explicitly, because everything downstream is measured against them.

One symbol carries 16 modulation symbols. With 16 of the 23 subcarriers carrying data, a single OFDM symbol holds 16μ coded bits before tiling — 16 at BPSK, 32 at QPSK, 64 at 16-QAM — and the tiling factor then divides the rate by 4, 16 or 64. The whole ladder of Section 4, from 7.8 to 1059 bit/s, is that one number multiplied by a modulation order, a code rate and a tiling factor; nothing else in the waveform moves.

Seven of twenty-three carriers are pilots. Roughly 30 % of the grid is spent on channel estimation, every symbol, at every rung. That is a deliberate choice for HF: pilots every 3–4 bins track a fading channel across the 2.1 kHz occupied band, and the Wiener estimator built on them is what makes coherent accumulation over 64 tiled repeats possible at all. It is also the single largest rate saving still available — reclaiming even half of it would beat every coding improvement measured in this report — which is why pilot reduction and 2-D channel estimation head the future-work list (Section 13). The pilot pattern is fixed by the article and was not varied here; changing it would invalidate the bit-exact reproduction that Section 11 rests on.

Symbol *tiling* is the article's central trick for negative-SNR operation: each OFDM symbol body is repeated back-to-back `sym_tile` times, and the receiver accumulates the repeats coherently before the FFT. Each 4× factor buys ≈6 dB of effective SNR at a 4× rate cost.

2.2 Frame Structure

Every frame (Figure 2) carries a two-stage preamble — Newman-phased tone combs [15] for detection and coarse CFO, then Zadoff–Chu symbols for sample-exact timing — followed by a header and the data block. The header (25 bits: `ver(2)` | `typ(3)` | `mod(2)` | `spd(2)` | `len(8)` | `CRC-8`) is always sent in the most robust configuration (BPSK, rate-1/3, six tiled symbols = 93 coded bits); its `mod`/`spd`/`len` fields drive the data-block demodulation, and `ver=2` marks an LDPC-coded data block (Section 6). Preamble bins carry deliberate gain (×2 for ZC symbols, $\times\sqrt{5.75}$ for tone combs) so the whole frame has uniform per-sample power — Section 3 shows why detection sensitivity depends on this.

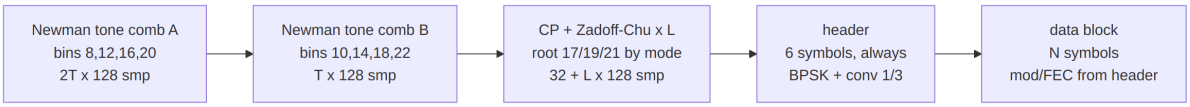


Figure 2: Frame structure. T is the mode's tone-tiling factor and L the mode's Zadoff–Chu symbol count.

2.3 Streamed Bursts

The preamble and header are a fixed cost paid *per frame*: 0.637 s in NORMAL, 2.49 s in ROBUST, 9.92 s in EXTREME (Table 3). For a full 27-byte frame that is ≈24 % of the air time in every mode; for a short frame at the top rung it is 74 %. Extended frames (Section 6) attack this by making one frame carry more. Streaming attacks it from the other side: one preamble and one header for N blocks,

$$[\text{preamble}][\text{header}][\text{blk}_0][\text{ZC}][\text{blk}_1]\ldots$$

with the preamble's own Zadoff–Chu block re-emitted every fourth block to refresh timing and residual CFO. All blocks share the packet type, size, modulation and code rate — that is precisely what allows a single header to describe them — and the block *count* is not carried in the waveform at all: the link layer signals it, or the receiver decodes until the samples run out.

Table 3: Fixed per-frame cost by mode, measured.

Mode	Tone field	ZC	Header	Total
NORMAL	0.320 s	0.045 s	0.272 s	0.637 s
ROBUST	1.280 s	0.173 s	1.040 s	2.49 s
EXTREME	5.120 s	0.685 s	4.112 s	9.92 s

Note that the header is comparable to the tone field, so dropping only the preamble buys 1.14×; dropping preamble *and* header buys 1.30×. Measured against the same packets sent as independent frames over the article channel (`experiments/stream_mode.py`, 300 bursts = 6 000 blocks per SNR point), 20 × 27-byte NORMAL packets take 16.23 s streamed against 28.16 s per-frame — 1.73× the throughput — for a sensitivity cost of 0.08 dB (Figure 3).

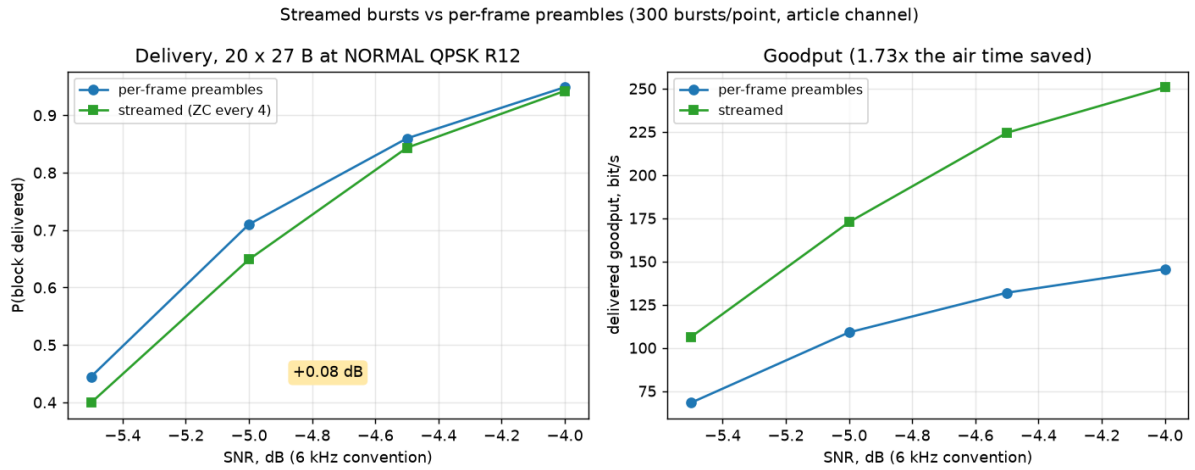


Figure 3: Streamed bursts against per-frame preambles (`experiments/stream_mode.py`). Delivery is scored per block, so the comparison is packets delivered rather than frames sent.

Three properties took measurement rather than reasoning to establish. First, **the Zadoff–Chu resync is not what holds the stream together**: with resync disabled entirely a 24 s stream decodes at the same packet-error rate, because the per-symbol frequency search and the pilot channel estimate already do all the tracking. The resync earns its place on sample-clock offset — a 32-sample cyclic prefix slips in ≈133 s at 20 ppm — and as a re-entry point for a receiver that missed the opening preamble. Second, **failures do not cascade**: block offsets are deterministic, so a block that fails its CRC costs exactly that block, and its raw LLRs are retained for chase combining on the retransmission. Third, **the cost grows with burst length**: a 5-block EXTREME burst measured ≈0.35 dB for only 1.21×, which is why streaming is recommended at NORMAL rungs and not at the modes where decibels are most expensive.

One invariant is easy to violate: the clip-and-filter threshold is a *whole-waveform* RMS, so a burst is not the concatenation of separately built frames. A one-block burst with no resync is bit-identical to an ordinary frame, and both the float and fixed-point test suites assert exactly that.

2.4 Transmit Chain

Figure 4 shows the transmit chain (`Transceiver.build_frame`). Packet bits (with CRC) are FEC-encoded, zero-padded to a whole number of OFDM symbols *before* interleaving, interleaved over the 16 data carriers, scrambled by a 15-bit LFSR, mapped (BPSK/QPSK/16-QAM Gray), and assembled into OFDM symbols with the ZC pilots and a Hermitian mirror so the time-domain signal is real. After tiling and CP insertion the preamble is prepended and a clip-and-filter stage (clip at RMS+6 dB, low-pass at 3 kHz) bounds the PAPR.

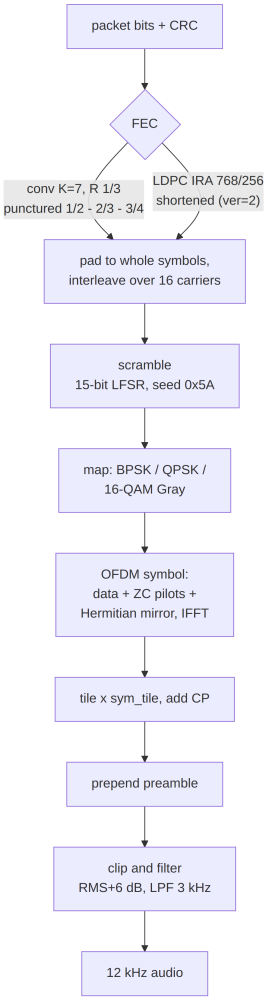


Figure 4: Transmit chain.

Two ordering invariants are easy to break and worth stating: TX applies interleave *then* scramble, so RX must descramble *then* deinterleave; and because the FEC block is padded to whole symbols before interleaving, RX must crop the deinterleaved stream back to the codec's block length before decoding.

2.5 Receive Chain

Figure 5 shows the receive chain (`Transceiver.demod_frame`). After the analytic-signal transform, synchronization runs in three stages (detailed in Section 3); the demodulator then processes each tiled symbol: CP removal and coherent tile accumulation (with per-symbol phase tracking), FFT, pilot-based channel estimation (zero-forcing estimate at the pilots, Wiener smoothing with an exponential frequency-correlation model of length 8 bins, MMSE equalization), and LLR formation weighted by the per-carrier symbol SNR, clipped to ± 20. One convention is load-bearing across four modules: LLR sign is **positive = logical 1**, consistent with the article's BPSK mapping table (the article's prose states the opposite of its own code).

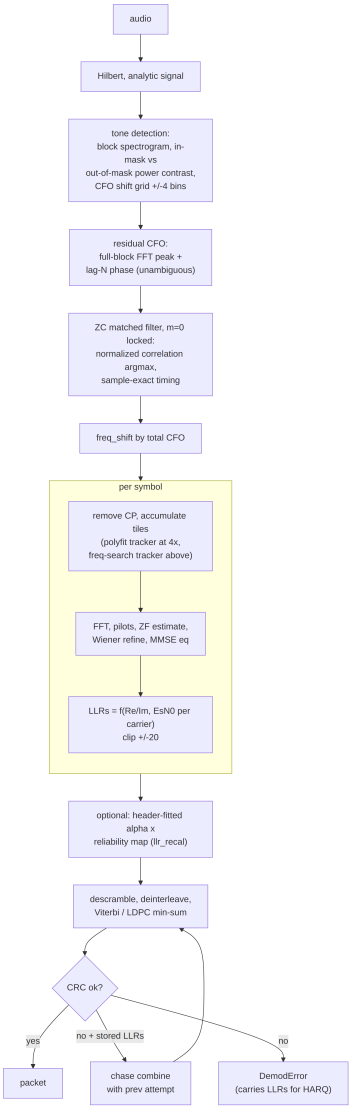


Figure 5: Receive chain, including the opt-in recalibration stage (Section 5) and the HARQ path (Section 6).

A CRC failure does not simply discard the frame: the `DemodError` exception carries the decoded header and the data-block LLRs, which is what makes the chase-combining HARQ of Section 6 possible without any extra signalling.

2.6 Soft Demodulation: from Symbols to LLRs

This subsection states exactly what the demodulator computes (`OFDMModem.demodulate_symbol_cp_so`). After tile accumulation and FFT, each data carrier k of a symbol obeys

$$Y_k = H_k S_k + N_k, \qquad N_k \sim \mathcal{CN}(0, \sigma^2), \tag{1}$$

where H_k is the channel coefficient and S_k the transmitted constellation point. With $\hat{H}_k$ the Wiener-smoothed pilot estimate, the MMSE equalizer output is

$$\hat{S}_k \;=\; \frac{Y_k\,\hat{H}_k^{*}}{|\hat{H}_k|^2 + \hat{\sigma}^2}. \tag{2}$$

The noise power $\hat{\sigma}^2$ is tracked decision-directed: the mean-square error of $\hat{S}_k$ against the nearest constellation point (per axis for BPSK/QPSK; against the sliced 16-QAM grid for $\mu = 4$), referred back through the channel by the mean $|\hat{H}_k|^2$, and smoothed with an exponential filter ($\alpha = 0.1$) across symbols. The per-carrier symbol SNR

$$\rho_k \;=\; \frac{|\hat{H}_k|^2}{\hat{\sigma}^2} \tag{3}$$

weights every LLR, so bits on faded carriers enter the decoder as weak evidence rather than as confident errors.

The log-likelihood ratio of a bit b is defined here as

$$L(b) \;=\; \ln \frac{P(b=1 \mid \hat{S})}{P(b=0 \mid \hat{S})}, \tag{4}$$

i.e. **positive LLR means logical 1** — the load-bearing sign convention noted above. Under the max-log approximation $\ln \sum_i e^{x_i} \approx \max_i x_i$, the general form

$$L(b_i) \;\approx\; \rho_k \Big(\min_{s \in \mathcal{S}_0^{(i)}} |\hat{S}_k - s|^2 - \min_{s \in \mathcal{S}_1^{(i)}} |\hat{S}_k - s|^2 \Big) \tag{5}$$

collapses to closed forms per modulation (as implemented; overall scale is immaterial to the scale-invariant Viterbi and min-sum decoders, but the *shape* across carriers and bit positions is what Section 5 calibrates):

$$\text{BPSK:} \quad L = 4\,\rho_k \,\mathrm{Re}\,\hat{S}_k, \tag{6}$$

$$\text{QPSK:} \quad L_I = 2\,\rho_k \,\mathrm{Re}\,\hat{S}_k, \qquad L_Q = 2\,\rho_k \,\mathrm{Im}\,\hat{S}_k, \tag{7}$$

$$\text{16-QAM (Gray, unit } a = g/\sqrt{10}\text{):} \quad L_{b_0} = 2\,\rho_k \,\mathrm{Re}\,\hat{S}_k/g, \qquad L_{b_1} = 2\,\rho_k \big(2a - |\,\mathrm{Re}\,\hat{S}_k|\big)/g, \tag{8}$$

with the imaginary-axis pair L_{b_2}, L_{b_3} analogous, and g the per-symbol gain estimate. The 16-QAM forms are the piecewise-linear max-log metrics of the Gray mapping: the outer bit (b_0) is the axis sign, the inner bit (b_1) discriminates inner from outer amplitude levels, changing sign at $|\mathrm{Re}\,\hat{S}| = 2a$. All LLRs are clipped to ± 20 before decoding.

3 Synchronization and the Sensitivity Gap

The decode chain reproduced the article bit-exactly almost immediately; the measured *sensitivity* did not. This section documents the diagnosis — the most instructive part of the reproduction — and the resulting synchronizer.

3.1 Diagnosis Method

The tool was a *genie-synchronized* control receiver: the same demodulator fed the true timing offset and CFO from the channel simulator. With genie sync the chain measured -7.6 dB (BPSK rate-1/3, article channel, PER $\leq 10\,\%$)[2]; the real receiver initially sat several dB worse. Every improvement below was validated by closing part of that gap, and the final residual is $\sim$0.3 dB (Section 11).

[2] All sensitivities in this report use the article's SNR convention: noise counted across the full 6 kHz Nyquist band, of which the signal occupies $\approx$2.1 kHz. Add $\approx$4.5 dB to compare against in-band noise only, $\approx$3.8 dB for the ham 2.5 kHz convention.

3.2 Three Defects and Their Fixes

Preamble power. Transmitting the tone combs and ZC symbols at nominal bin amplitude leaves the preamble region ∼7 dB below the frame-average power (4 tones versus 23 loaded carriers). Detection then fails at SNRs where the payload would decode. The fix is the gain scaling of Section 2: $\times\sqrt{5.75}$ on tone bins and $\times 2$ on ZC bins equalizes per-sample power across the frame.

Tone-detection metric. The article's tone detector thresholds the in-mask energy *fraction* of a block spectrogram. That fraction is SNR-dependent — it fails below ≈0 dB regardless of threshold — and divides by zero on noise-free signals. The implemented metric is a *contrast ratio*: mean in-mask bin power over mean out-of-mask bin power, evaluated over a grid of ±4-bin CFO shifts of the mask, with the denominator regularized by 1 % of the median block power. The regularizer matters: on clean signals the out-of-mask power is ≈0 and the unregularized argmax degenerates into a lottery among CFO hypotheses, producing exact one- and two-bin CFO errors on clean WAV recordings.

Zadoff–Chu time–frequency ambiguity. The ZC matched filter selects the best *normalized* correlation over a search window after the tone preamble (the raw-peak argmax locks onto the periodic tiled data symbols). The subtler defect was scanning the ZC correlation over ±1-bin frequency hypotheses: a Zadoff–Chu sequence's frequency-shifted replica correlates almost as well at a *shifted time* (the classic ZC time–frequency ambiguity), so at low SNR the wrong hypothesis wins, injecting a one-subcarrier (93.75 Hz) CFO error plus a ∼30-sample timing error. Locking the composite detector to the zero-CFO hypothesis — legitimate because the tone stage already leaves a residual well below half a bin — recovered ≈1 dB of sensitivity by itself.

Residual CFO. The tone-stage residual CFO is estimated by a full-length FFT peak over the first tone block (unambiguous over ±1.5 bins), refined by a lag-N phase estimate. A lag-N estimate alone is ambiguous modulo one subcarrier spacing; during development a lag-16 variant produced 276 Hz errors because the lag was not a multiple of the tone-comb period — the general rule (enforced in the code) is that a delay-and-correlate lag must be a multiple of the comb's time-domain period.

3.3 The STF Preamble Variant

A commenter on the article proposed replacing the Newman tone comb with 802.11-style short-training tones every 8 bins. This was implemented as `STFOFDMModem`: tone combs on bins $[8, 16, 24]$ and $[4, 12, 20]$ make the tone block periodic with 16 samples, so the residual CFO comes from a plain Schmidl–Cox-style delay-and-correlate [17] (lag 16 refined by lag 128), unambiguous over ±375 Hz. An A/B sweep over the full ±300 Hz CFO range and all eight modulation/rate configurations shows the two preambles statistically equivalent: identical −7.2 dB sensitivity, all eight per-configuration sensitivities within 0.14 dB, every PER point within 2σ binomial noise (Figure 6). The article-faithful `FullOFDMModem` remains the default.

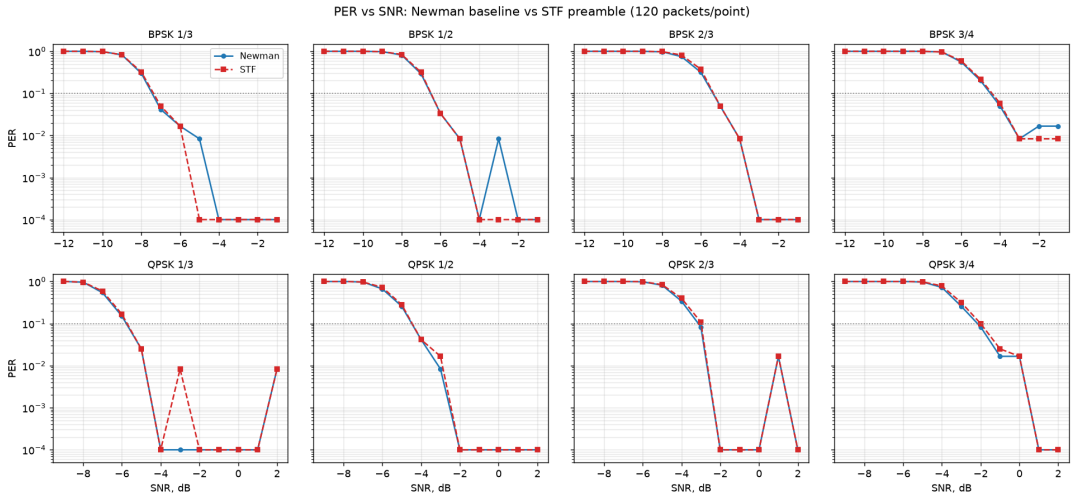


Figure 6: PER versus SNR for the Newman-tone and STF preamble variants over the article channel (same channel realizations per arm).

4 SNR-Adaptive Link Modes

The article's waveform operates around −7 dB. The first major extension scales the coherent-accumulation tile factor (and the preambles with it) into three link modes (Table 4), trading bitrate for sensitivity.

Table 4: Link modes (sensitivities at PER $\leq 10\,\%$, recalibrated receiver, article channel, 6 kHz-band SNR).

Mode	Tiles	ZC root	User rate (BPSK 1/3)	Sensitivity
NORMAL	4×	17	118 bit/s	−7.6 dB
ROBUST	16×	19	31 bit/s	−11.7 dB
EXTREME	64×	21	7.8 bit/s	−17.9 dB

Shannon capacity [18] of the 2100 Hz occupied band at −20 dB (6 kHz reference) is ≈30 bit/s; EXTREME's channel rate of 22 bit/s is 78 % of capacity, so the theoretical wall for this bandwidth is −20.7 dB and the measured −17.9 dB floor sits ≈3 dB from it.

4.1 What Makes the Low-SNR Modes Work

Each of the following was found necessary empirically; removing any one of them collapses the EXTREME mode.

Per-symbol residual-CFO hypothesis search. A 64× tiled symbol integrates coherently over 0.69 s and cannot tolerate even a few Hz of residual CFO across the accumulation window; per-tile phase tracking (the NORMAL-mode polynomial fit) is pure noise at −19 dB. The tiled demodulator instead derotates the whole symbol over a grid of frequency hypotheses, accumulates tiles under each, and keeps the hypothesis with maximum in-band energy — spending the symbol's *entire* energy (≈19 dB E/N_0 even at −20 dB SNR) on the frequency decision. A slew-limited tracker (full grid on the first

symbol, ±2 grid steps thereafter) suppresses per-symbol argmax noise; the search range must cover the worst-case preamble CFO error (±25 Hz for EXTREME).

Finer detection FFT. ROBUST and EXTREME tone detection uses 512-point spectrogram blocks: 4× more tone energy per bin, and a 23.4 Hz coarse-CFO grid whose residual fits a plain lag-N estimate.

Group-coherent ZC correlation. The ZC stage correlates kernels of 2–4 concatenated ZC symbols (magnitudes summed across groups) with a small fractional-CFO hypothesis grid (±15 Hz) — single-symbol correlation is not selective enough at −18 dB, and Section 3's ambiguity argument forbids a wide frequency scan.

Per-mode preamble roots. The receiver cannot read the tile factor from the header, because it needs the tile factor to demodulate the header. The mode is therefore signalled by the preamble itself: each mode owns a distinct ZC root (17/19/21), a wrong-root matched filter simply does not lock, and `demod_frame_auto` tries the modes in turn (Figure 7). Detection thresholds were tuned against noise-only false alarms (0/20 on pure noise, all modes).

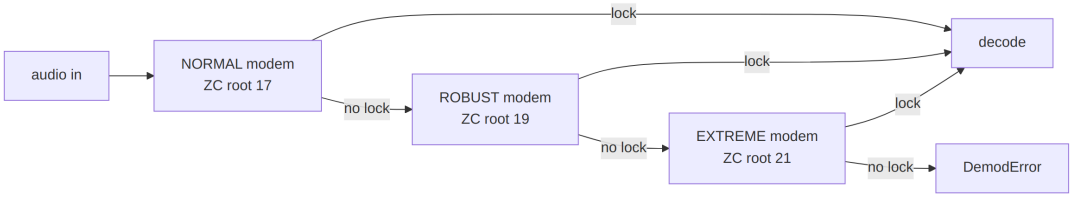


Figure 7: Blind mode detection: the preamble root *is* the mode signal. A consequence used throughout the link layer: a mode switch can never deadlock the link.

4.2 Constellations Across the Operating Range

Figure 8 makes the adaptation policy visible in one picture: for eight SNRs spanning the whole operating range (−17.5 to +20 dB), it shows the *equalized received constellation of the rung the rate controller actually selects* at that SNR (highest rung whose measured sensitivity clears a 1 dB margin). Each panel is produced by the real per-symbol receive chain — coherent tiling accumulation, per-symbol frequency search where the mode uses one, and MMSE equalization (`experiments/figures.py`). At −17.5 dB the EXTREME BPSK decision variable barely emerges from the noise — which is precisely why rung 0 pairs 64× tiling with the rate-1/3 code and soft decisions; the QPSK quadrants firm up through the mid-ladder, and the 16-QAM grid the top rungs rely on only becomes usable above ≈+3 dB, matching the ladder sensitivities of Table 6.

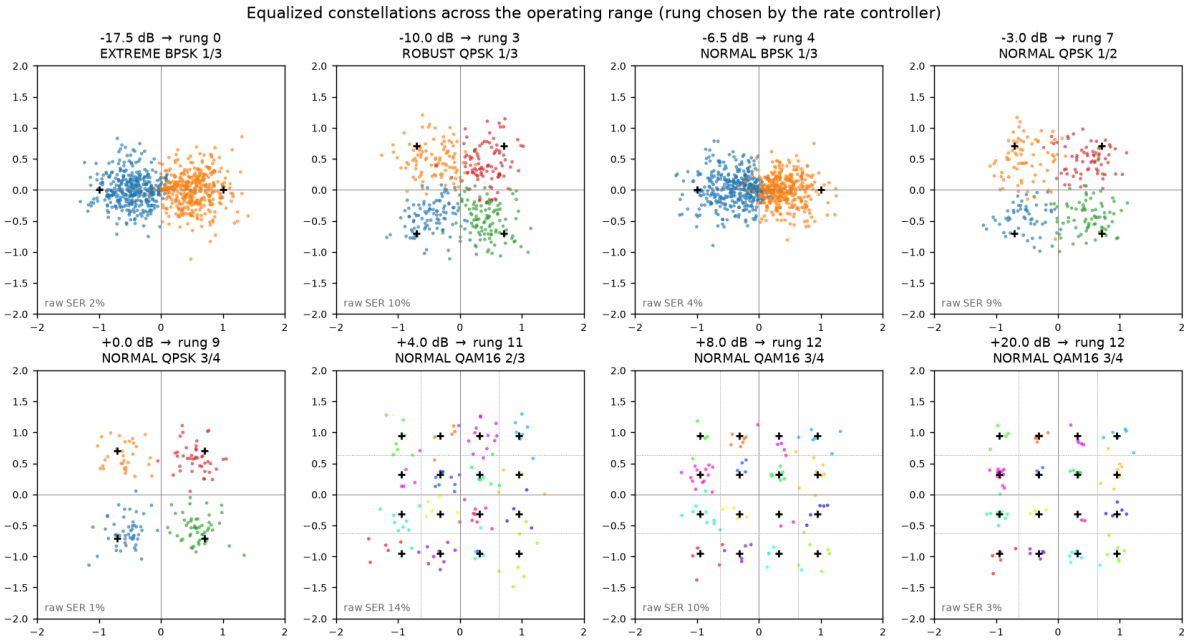


Figure 8: Equalized constellations at the controller-selected rung, from the EXTREME sensitivity floor to a clean channel (article channel, AWGN + CFO). Black crosses mark the ideal constellation points; each received point is colored by the *transmitted* symbol, so raw pre-FEC errors appear as points of the wrong color inside a neighbouring decision region (the per-panel "raw SER" annotation counts them), and the dotted subgrid on the 16-QAM panels marks the decision boundaries between the 16 cells.

4.3 Measured Waterfalls

Figure 9 shows the three modes' PER waterfalls on the article channel. The function `select_mode` implements the resulting adaptation policy (mode + modulation + code rate for an expected SNR), later subsumed by the link-layer ladder of Section 7.

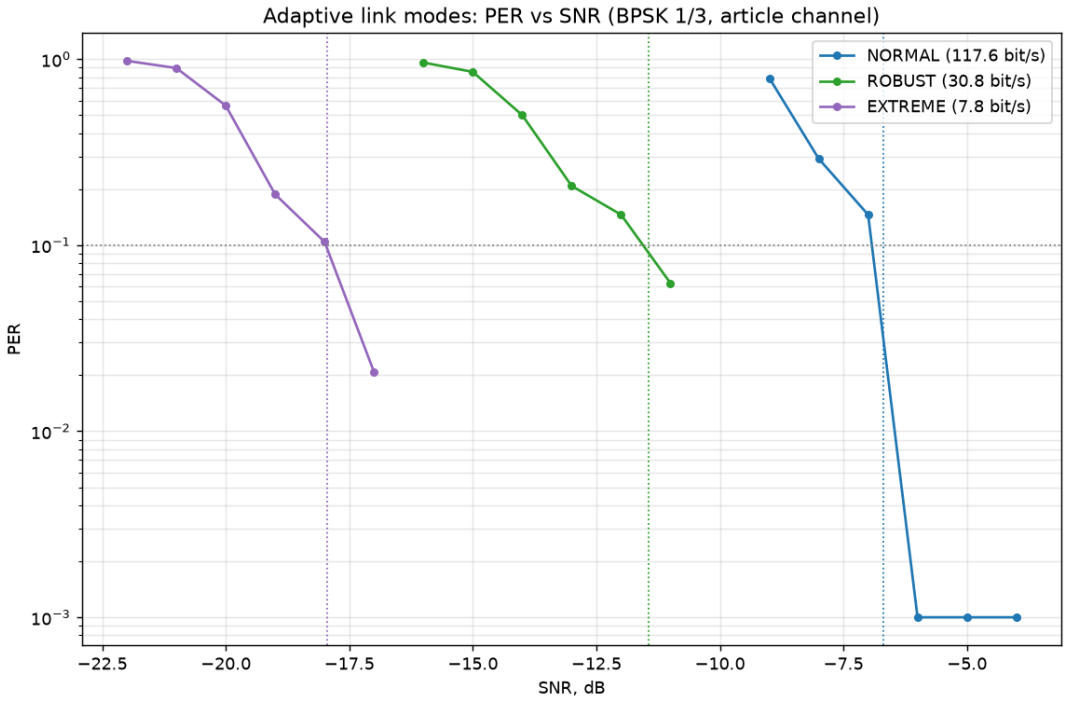


Figure 9: PER versus SNR for the three link modes (`experiments/adaptive_modes.py`).

5 LLR Quality and Recalibration

A code-review pass over the finished system asked a simple question: are the demodulator's LLRs actually calibrated — does $|L|$ mean what it claims? Measuring them against ground truth (`experiments/llr_calibration.py`) found the single largest untapped gain in the system.

5.1 The Miscalibration Is Shape, Not Temperature

Two terms are used throughout this section and deserve definition. *Temperature scaling* is the standard one-parameter recalibration family

$$L' = \alpha \cdot L, \qquad \alpha > 0, \tag{9}$$

one global gain applied uniformly to every LLR. The name comes from machine-learning calibration [8], where a network's logits are divided by a scalar "temperature" T before the softmax ($\alpha \equiv 1/T$) — itself borrowed from the Boltzmann distribution, in which temperature controls how sharply probability concentrates on the most likely outcome: $\alpha < 1$ softens overconfident beliefs, $\alpha > 1$ sharpens underconfident ones. It is the natural first candidate here because it preserves signs and ordering, costs one multiply, and can be fitted blind on a per-frame basis: a correctly calibrated Gaussian LLR ensemble satisfies the consistency condition $\mathrm{Var}[\ell] = 2\,\mathbb{E}[\ell]$ (for $\ell = L \cdot \tilde{x}$, the LLR signed by the true bit), so the moment estimator $\alpha = 2\,\mathbb{E}[\ell]/\,\mathrm{Var}[\ell]$ recovers the gain that restores consistency. By contrast, *shape* refers to any monotone but non-linear correction $L' = f(|L|)\cdot \mathrm{sgn}\, L$ — one that can treat weak and strong LLRs differently. The finding of this section is that the front end's miscalibration is of the second kind, so the first kind cannot repair it.

The raw LLRs ($4 \cdot \mathrm{Re} \cdot E_s/N_0$ per carrier, clipped to ± 20) are *shape*-miscalibrated: bits with $|L| < 2$ err 11 % empirically versus the 39 % their value implies — the decision-directed noise estimation and per-carrier weighting over-spread confidence downward — while the clipped top of the scale overstates. A single temperature cannot fix this: the moment fit is dragged by the clipped mass, and the fitted $\alpha < 1$ while the weak LLRs are empirically *under*confident, a contradiction only a shape correction resolves. Figure 10 shows the measured shape. In the left panel, the measured error rate starts *below* the ideal curve $P(\mathrm{err}) = 1/(1 + e^{|L|})$ (weak LLRs are better than they claim) and crosses it around $|L| \approx 5$, after which errors run orders of magnitude *above* the claim — and beyond $|L| \approx 9$ the measured reliability actually *decreases* as the claimed confidence rises, all the way into the clipped mass at $|L| \approx 19$. The right panel replots the same data as empirical log-odds versus nominal $|L|$: a calibrated front end would sit on the identity line, but the measured curve rises above it at small $|L|$ (wanting $\alpha > 1$) and falls far below at large $|L|$ (dragging any global fit to $\alpha < 1$ — here $\alpha = 0.67$). No straight line through the origin can be on the correct side of both regions simultaneously; the red curve itself, applied as a lookup, is the monotone reliability map of the next subsection. Note where the measured curve peaks: ≈ 7.5 empirical log-odds at $|L| \approx 9$ — the deployed map's cap of 7.4 is that peak, re-measured independently. This is also the figure that explains the LDPC result of Section 6: exact sum–product consumes these numbers as *absolute* probabilities and pays for every lie, while min-sum and Viterbi only compare them.

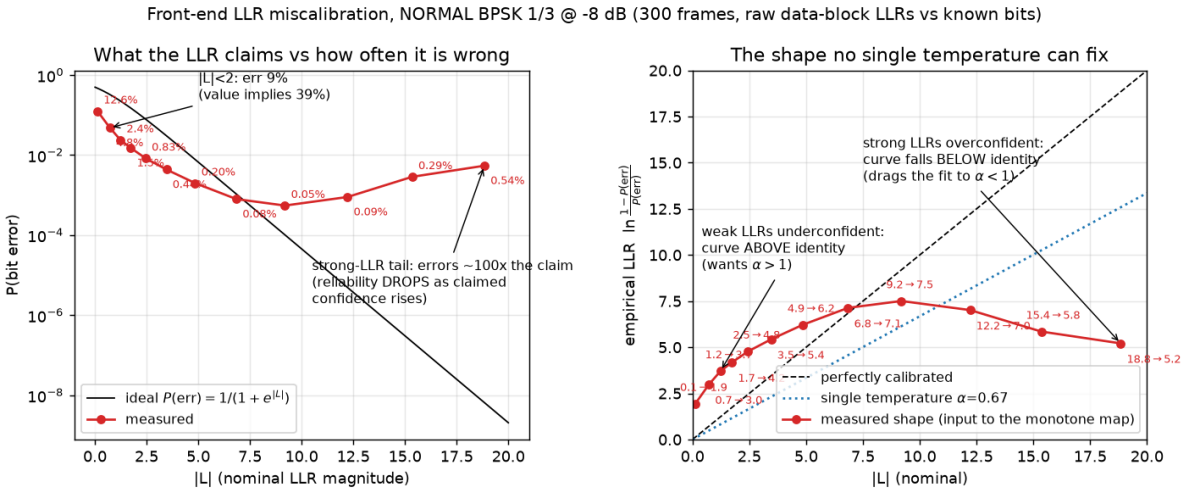


Figure 10: The measured LLR miscalibration shape (NORMAL BPSK 1/3 at −8 dB, 300 frames, raw data-block LLRs against known transmitted bits; reproduce with `experiments/llr_shape.py`, bin table and parameters in `results/llr_shape.json`). Left: claimed versus actual reliability, each bin annotated with its measured error rate. Right: the same data as empirical log-odds, each point annotated with its mapping (nominal → empirical) — above the identity line at small $|L|$, below it at large $|L|$, so no single temperature α can correct both ends.

5.1.1 Measurement Procedure

The data behind Figure 10 (and behind the deployed map of the next subsection) is produced by a self-labeling protocol — the transmitter's bit pipeline is deterministic, so every raw LLR can be paired with the coded bit it was supposed to carry:

1. **Stimulus.** 300 NORMAL-mode frames, BPSK rate 1/3, 27-byte random payloads; each passes the channel simulator with a random start offset (300–1200 samples), a random CFO drawn uniformly from ±60 Hz, and AWGN at the target SNR (−8 dB here, 6 kHz convention). Collection runs as parallel 30-frame chunks with per-chunk seeds, so the run is reproducible and its result is recorded in `results/llr_shape.json`.

2. **Reception.** Each frame goes through the *production* receive chain — preamble detection, CFO correction, tile accumulation, channel estimation, MMSE, LLR formation with the ±20 clip (Section 2.6) — and the raw data-block LLRs are captured *before* any decoding or recalibration. Frames that fail preamble detection are skipped; no selection is made on CRC, so decoding failures do not bias the sample.

3. **Ground truth.** The known payload is run through the TX bit pipeline (FEC encode → pad → interleave → scramble), giving the transmitted coded bit for every LLR position. With $\tilde{x} = \pm 1$ ($+1 \Leftrightarrow$ bit 1, the sign convention), each sample is reduced to the signed product $\ell = L \cdot \tilde{x}$: $\ell < 0$ means the LLR votes for the wrong bit, and $|\ell| = |L|$ is its claimed confidence.

4. **Binning.** Pooled samples are binned by $|\ell|$ (edges $0, 0.5, 1, 1.5, 2, 3, 4, 6, 8, 11, 14, 17, 20$); bins with fewer than 80 samples are dropped. Per bin, the empirical error rate is $\hat{p} = \text{mean}(\ell < 0)$, clamped to $[\,0.5/N,\ 1 - 0.5/N\,]$ (a half-count continuity correction that keeps the log-odds finite when a bin has no errors), and the bin's abscissa is its mean $|\ell|$.

5. **Derived quantities.** The empirical log-odds $\hat{L} = \ln\big((1-\hat{p})/\hat{p}\big)$ give the right panel; the reference curve is $P(\text{err}) = 1/(1 + e^{|L|})$. The single-temperature baseline is the moment fit

$$\alpha \;=\; \frac{2\,\mathbb{E}[\ell]}{\text{Var}[\ell]}, \tag{10}$$

   the consistency condition of a well-calibrated Gaussian LLR ($\text{Var} = 2\,\mathbb{E}$ under the standard approximation $L \sim \mathcal{N}(\pm\sigma^2/2, \sigma^2)$) — the same estimator the runtime header-based temperature fit uses, which is exactly why the figure's $\alpha = 0.67$ is representative of what deployment would apply.

The harness is `experiments/llr_calibration.py` (collection and console reliability tables) and `experiments/llr_shape.py` (the figure and its JSON record); re-running either regenerates all numbers from scratch.

5.2 The Monotone Reliability Map

The correction is a measured monotone map: each $|L|$ bin is replaced by the log-odds of its empirical error rate (10 interior bin edges, saturating at the data-justified maximum of ≈7.4). Capping LLRs at what the data supports also defuses confidently-wrong bits from BSC flips and fades. Measured at the sensitivity edge (NORMAL BPSK 1/3, 40 trials/point):

Table 5: Decodes out of 40 with and without recalibration.

SNR	Raw LLRs	Recalibrated
−8 dB	33/40	38/40
−9 dB	6/40	**29/40**

This is **≈1.5–2 dB of sensitivity** for the existing convolutional chain (Viterbi decoding is scale-invariant but not shape-invariant), and it is what finally lets LDPC sum-product decoding work on this front end (9/40 → 24/40 at −9 dB). The accumulation-limited ROBUST/EXTREME rungs are unaffected *at their operating points* (re-measured at 300 frames/point in Section 6: the map's gain at EXTREME decays to zero by −18 dB, though ≈0.5 dB survives at the deep edge below the rung), and 16-QAM *regresses* under the BPSK-trained map at the ladder rates (re-measured at 300 frames/point in Section 6: −0.5 dB at rate 2/3, though at rate 1/3 the map actually helps — the regression is rate-dependent), so the map is gated to modulations with $\mu \leq 2$ bits/symbol.

5.3 Why the Map Helps Viterbi (and Temperature Cannot)

It is worth being precise about *which* part of the calibration each decoder consumes, because the two decoder families fail in opposite ways:

- **Temperature is provably useless for Viterbi.** The path metric $\mathrm{PM} = \sum_e \beta_e$ is linear in the LLRs (Section 6), so scaling every LLR by any $\alpha > 0$ scales every competing path metric by the same α — the arg-max, and therefore every decoded bit, is unchanged. Figure 11 (left) shows this empirically: the temperature-only curve overlays the raw curve *exactly*, frame for frame, at every SNR. Exact sum–product LDPC is the opposite case: its tanh nonlinearity consumes absolute LLR values, so the scale error alone costs it dB.

- **Shape is what Viterbi actually consumes.** The decoder compares *sums* of LLRs along competing paths, so the decisive quantity is the relative weight of one strong bit against several weak ones. The measured shape (Figure 10) breaks exactly that ratio: a raw $|L| = 9$ bit claims to outweigh four $|L| = 2$ bits, but its empirical log-odds are ≈ 6 while each weak bit is really worth ≈ 4. Two failure modes follow. A single confidently-wrong strong bit (fade, impulse) can outvote several correct weak bits along a path — the map's cap at the data-justified ≈ 7.4 defuses this. And underweighted weak bits waste real information — the map restores their influence in the path sums.

- **HARQ combining needs calibrated addends.** Chase combining adds the LLR streams of two receptions; if the two frames sit on different miscalibrated scales (different SNR, different noise-estimate transients), the sum weights the frames wrongly. Mapping both to honest log-odds first makes addition the theoretically correct combining rule.

- **Puncturing is calibration-neutral.** Depuncture zeros mean "no opinion," and any odd monotone map keeps 0 at 0, so the inserted positions are unaffected by construction.

Figure 11 makes the asymmetry measurable in one sweep (300 frames per point): the same raw LLRs from each received frame are decoded three ways. Temperature-only reproduces the raw decisions bit-exactly at every point — 2 700 frames without a single divergence; the generating script asserts this equality, so scale invariance is enforced, not eyeballed. The monotone map shifts the decode waterfall by ≈1.5 dB: at −9 dB, 63/300 raw versus 216/300 mapped; at −9.5 dB, 15/300 versus 165/300 — consistent with Table 5.

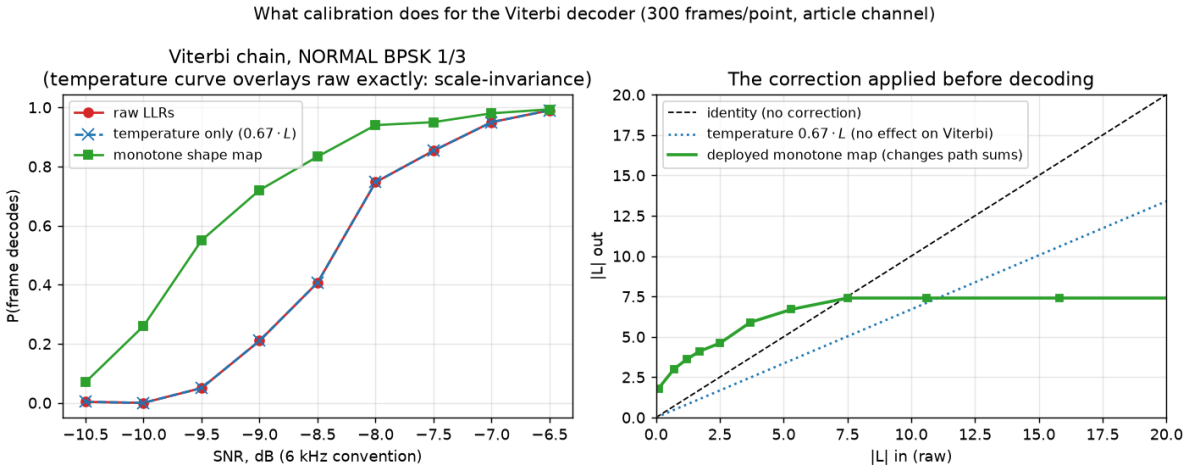


Figure 11: Calibration and the Viterbi decoder (reproduce with `experiments/viterbi_recal.py`; parameters and per-point counts in `results/viterbi_recal.json`). Left: decode probability versus SNR for the same received frames decoded from raw LLRs, temperature-scaled LLRs ($0.67 \cdot L$ — overlays raw exactly), and shape-mapped LLRs. Right: the deployed monotone map against the identity and the temperature line: it boosts weak LLRs above identity and caps the top at the data-justified ≈ 7.4.

5.4 Deployment

Alongside the static map, a per-frame temperature $\alpha = 2\,\mathbb{E}[Lx]/\mathrm{Var}[Lx]$ is fitted from the 96 known bits of each decoded header (a Gaussian-consistency fit), which keeps LLR scales consistent across frames — a prerequisite for the HARQ combining of Section 6. Following the faithful-defaults principle, recalibration is **off** in the plain `Transceiver` and enabled by the link-layer station (`llr_recal="auto"`). The entire rate ladder was re-measured with recalibration enabled; Section 7 carries the resulting sensitivities (mid-ladder BPSK/QPSK rungs gained 0.7–1.4 dB).

6 Coding and Modulation Extensions

6.1 Baseline FEC

The article's FEC — reproduced bit-exactly — is the $K = 7$ convolutional code with generators 133/171/165 (octal) at rate 1/3, punctured to 1/2, 2/3, and 3/4, decoded by a vectorized soft-decision Viterbi [19]. It remains the default and the header is always convolutionally coded, so every extension below is backward compatible.

6.1.1 Encoder and Puncturing

For input bits $u[e]$ (zero-tailed with $K-1=6$ flush bits), the three coded streams are

$$c_p[e] \;=\; \bigoplus_{j=0}^{6} g_p^{(j)}\, u[e-j], \qquad p \in \{0,1,2\}, \tag{11}$$

with taps g_p the binary expansions of $133_8, 171_8, 165_8$. Puncturing transmits only the positions where the rate's mask is 1 (mask column $= e \bmod P$, period P):

$$M_{1/2} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad M_{2/3} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad M_{3/4} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \tag{12}$$

keeping 4/6, 3/6 and 4/9 of the mother-code bits respectively. Rate 1/3 is the all-ones mask.

6.1.2 Soft-Decision Viterbi

The receiver *depunctures* by inserting $\lambda = 0$ ("no opinion") LLRs at the punctured positions — consistent with $L = 0 \Leftrightarrow P(1) = P(0)$ — and runs a max-log Viterbi over the 64-state trellis. With the sign convention of Section 2.6, the expected symbol for a transition from state s' with input u is $x_p(s',u) \in \{-1,+1\}$ ($+1 \Leftrightarrow$ coded bit 1), the branch metric is the soft correlation

$$\beta_e(s',u) \;=\; \sum_{p=0}^{2} \lambda_p[e]\; x_p(s',u), \tag{13}$$

and the path metric recursion is

$$\mathrm{PM}_{e+1}(s) \;=\; \max_{(s',u)\,\to\, s} \big[\, \mathrm{PM}_e(s') + \beta_e(s',u) \,\big], \tag{14}$$

with the winning predecessor stored for traceback (one bit per state per step in the C port — the predecessor of s is determined by which of its two candidates won). Because both β and the LLRs are linear in the demodulator's scale, the decoder is scale-invariant: only the *relative* weighting of carriers and bit positions matters, which is why the front-end shape calibration of Section 5 pays off while a global gain does not.

6.2 LDPC

An LDPC alternative (`ofdm_phy/ldpc.py`) was built as an irregular repeat–accumulate code [13]: $N = 768$, $K = 256$ (rate 1/3), information-node degree 4, girth ≥ 6, linear-time accumulator encoding, and shortening to the payload size. Frames carry it via header `ver=2`, so the receiver needs no negotiation. The construction history is itself a result: a random (4,6) matrix *lost* to the convolutional code (short cycles), and exact sum-product decoding lost several dB end-to-end because it is sensitive to the front end's miscalibrated LLR scale. The shipped decoder is normalized min-sum [4] ($\alpha = 0.8$ float, $\alpha = 0.75$ as $x - (x \gg 2)$ in the integer kernel) — deliberately, because min-sum is scale-invariant. Measured: +0.7 dB over the convolutional code on AWGN at BLER 10 %, compressing to ≈parity end-to-end at −8 dB — the front-end LLR quality, not the code, is the binding constraint (Section 13).

6.2.1 Structure, Encoding, and Decoding

The parity-check matrix has the IRA form $\mathbf{H} = [\,\mathbf{H}_u \mid \mathbf{H}_{\text{acc}}\,]$, where $\mathbf{H}_u$ (512×256, column weight 4, girth ≥ 6 by construction) connects information bits to checks and $\mathbf{H}_{\text{acc}}$ is the dual-diagonal accumulator. Check i therefore reads

$$\bigoplus_{v\in A(i)} u_v \;\oplus\; p_{i-1} \;\oplus\; p_i \;=\; 0, \tag{15}$$

which gives linear-time encoding as a running XOR:

$$p_i \;=\; p_{i-1} \;\oplus\; \bigoplus_{v\in A(i)} u_v, \qquad p_{-1} \equiv 0. \tag{16}$$

Shortening to a k-bit packet ($k \leq 255$) fixes the unused $256-k$ information positions to zero; the corresponding LLRs enter the decoder as $+\infty$-grade priors.

Decoding is iterative normalized min-sum message passing. With $m_{v\to c}$ the variable-to-check and $m_{c\to v}$ the check-to-variable messages, initialized from the channel LLRs L_v:

$$m_{c\to v} \;=\; \alpha \cdot \Big(\prod_{v'\in N(c)\setminus v} \operatorname{sgn} m_{v'\to c}\Big) \min_{v'\in N(c)\setminus v} \big|m_{v'\to c}\big|, \tag{17}$$

$$m_{v\to c} \;=\; L_v \;+\; \sum_{c'\in N(v)\setminus c} m_{c'\to v}, \tag{18}$$

with hard decisions taken on the total $L_v + \sum_c m_{c\to v}$ and iteration stopping early on a zero syndrome $\mathbf{H}\hat{\mathbf{x}}^{\mathsf{T}} = \mathbf{0}$ (60 iterations maximum; good frames converge in a few). The normalization α compensates min-sum's overestimate of check reliability versus exact sum–product; crucially, Eq. (17) is homogeneous in the LLR scale, which is why min-sum survives the front end's miscalibrated scale where exact sum–product (whose tanh nonlinearity consumes *absolute* LLRs) measurably lost several dB.

Figure 12 is the LDPC counterpart of the Viterbi calibration sweep (Figure 11): the same raw LLRs from each received frame are decoded four ways — both kernels, each on raw and on shape-mapped inputs (the exact-SPA kernel is reconstructed for this measurement; the shipped code retains only min-sum), 300 frames per point. The asymmetry the equations predict is exactly what the sweep measures. Exact sum–product on raw LLRs collapses ≈1.5–2 dB *below* min-sum (at −9 dB: 0/300 versus 112/300; at −8.5 dB: 19/300 versus 192/300) because every overconfident bit in Figure 10 is taken literally by the tanh rule. The shape map fully rescues it (0/300 → 234/300 at −9 dB), landing on par with mapped min-sum — once the LLRs stop lying, the exact rule has nothing left to be hurt by. Min-sum, meanwhile, is *helped* but not gated by the map (≈1 dB: 112/300 → 231/300 at −9 dB): its check update ignores absolute scale, but the map's cap still defuses confidently-wrong bits in the variable-node sums, the same mechanism as in the Viterbi path metrics.

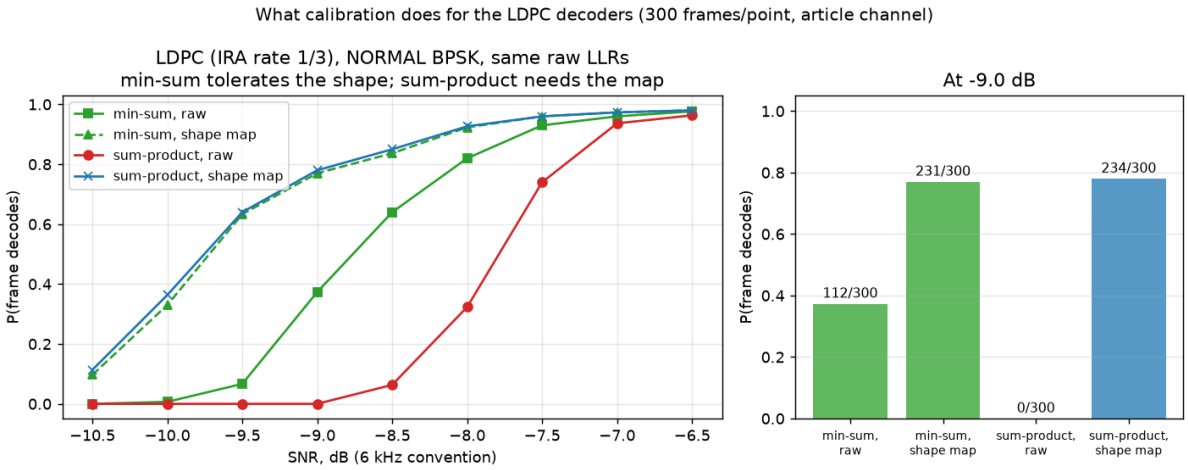


Figure 12: Calibration and the LDPC decoders (reproduce with `experiments/ldpc_recal.py`; parameters and per-point counts in `results/ldpc_recal.json`). The same received frames decoded by normalized min-sum and by exact sum–product, each from raw and from shape-mapped LLRs. Left: decode probability versus SNR. Right: the four variants at −9 dB. Sum–product is unusable on the raw front end and fully recovered by the map; min-sum tolerates the miscalibration and still gains ≈1 dB from it.

6.2.2 The Code Families Head-to-Head, Calibrated

With the front end fixed, the remaining question is which *code* is better — the comparison that the raw front end had rendered moot ("≈parity end-to-end"). Figure 13 puts the fully calibrated chains side by side: convolutional + Viterbi versus LDPC under both kernels, all on shape-mapped LLRs, identical channel-draw sequences per variant, 300 frames per point (binomial uncertainty $\leq \pm 3\,\%$ near the waterfall midpoint). Two conclusions:

- **The LDPC code gain finally materializes end-to-end:** ≈0.3–0.4 dB at the waterfall (at −9.5 dB: 193/300 versus 151/300, i.e. 64 % versus 50 %; at −10 dB: 116/300 versus 83/300). This is consistent with the +0.7 dB AWGN code-level measurement — the front-end miscalibration was masking it, not the code failing to deliver it.
- **Calibrated min-sum ≈ calibrated sum-product:** above −9 dB the two LDPC curves are identical to within one frame in 300. A small exact-SPA edge survives only at the extreme low end (at −10.5 dB: 54/300 versus 40/300; ≈0.1 dB), far below the rung's operating point — so the integer min-sum kernel deployed in the fixed-point and C receivers gives up nothing measurable in the operating region.

The remaining distance to a standards-grade LDPC (≈2 dB theoretical) is now attributable to the code construction itself (short block, $K = 256$, hand-rolled IRA), not to the decoder or the front end — which is precisely the open thread recorded in Section 13.

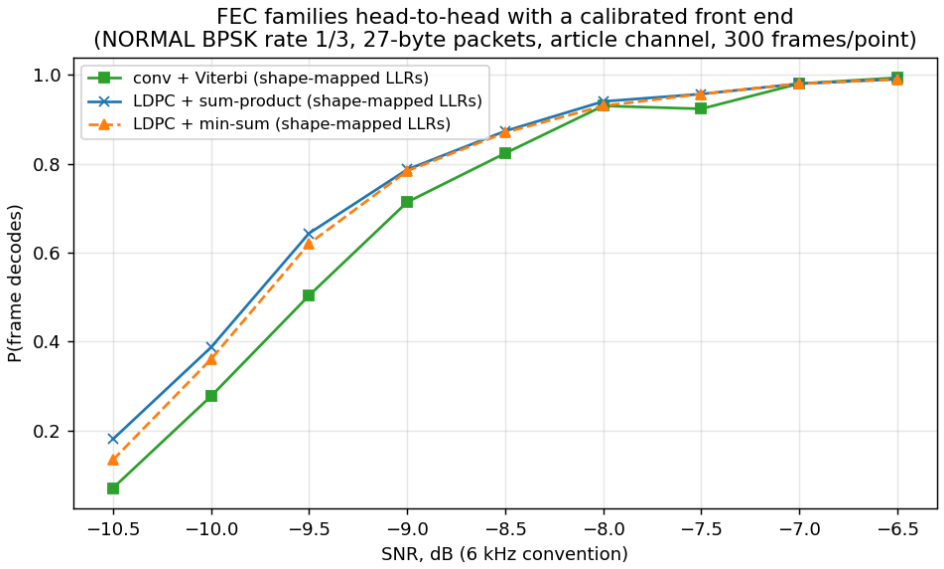


Figure 13: The two FEC families with a calibrated front end (reproduce with `experiments/fec_comparison.py`; every parameter and per-point count is recorded in `results/fec_comparison.json`): convolutional + Viterbi versus LDPC with exact sum–product and with normalized min-sum, all decoding shape-mapped LLRs. LDPC holds ≈0.5 dB over the convolutional chain, and the two LDPC kernels are indistinguishable once calibrated.

6.2.3 The Same Study at the EXTREME Edge

Figure 14 repeats the entire calibration study at the opposite end of the ladder — EXTREME mode (64× tiling, BPSK rate 1/3, the −17.9 dB rung-0 regime), SNR −20.5 to −16.5 dB, 300 frames per point, with the *NORMAL-trained* shape map applied unchanged. Four findings:

- **Scale invariance holds, as it must:** raw and temperature-only Viterbi decodes are bit-identical across all 2 700 conv frames (asserted by the harness).
- **The map transfers, but only below the operating point.** It buys ≈0.5 dB at the deep edge (at −20.5 dB: 138/300 versus 81/300; at −20 dB: 209 versus 157) and the gain decays to *zero* by −18 dB (279 versus 279) — right where the rung actually operates. This refines Section 5's "accumulation-limited rungs are unaffected": accurate at the operating point, where acquisition and accumulation dominate, but not a statement about the region below it.
- **Sum–product's raw-LLR penalty is far milder here** (at −19.5 dB: 152/300 raw versus min-sum's 215, compared with 0/300 versus 112/300 in the NORMAL study): the 64× coherent accumulation reshapes the LLR distribution and shrinks the overconfident tail the tanh rule chokes on. The map still closes the gap completely, and calibrated min-sum and calibrated sum–product are again identical to the frame.
- **The code families reach parity at EXTREME.** The calibrated head-to-head shows no consistent winner (largest deviation 21 frames in 300, alternating in sign) — NORMAL's ≈0.3–0.4 dB LDPC advantage does not replicate. At the deep edge

the loss budget is dominated by acquisition (detection failures count against every variant equally), so the code choice matters correspondingly less.

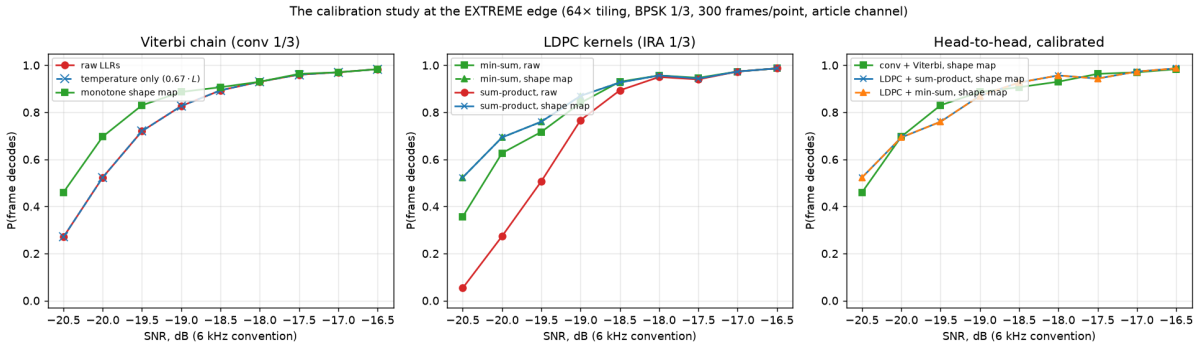


Figure 14: The calibration study repeated at the EXTREME edge (reproduce with `experiments/extreme_recal.py`; parameters and per-point counts in `results/extreme_recal.json`). Left: the Viterbi three-way. Middle: the LDPC kernels. Right: the calibrated head-to-head — parity, unlike Figure 13's NORMAL result.

6.2.4 . . . and for 16-QAM: the Regression Is Rate-Dependent

Figure 15 completes the tour with 16-QAM (NORMAL mode; rate 1/3 in the first two panels for code parity with the LDPC code, whose only rate that is). The result overturns the simple version of the "the map regresses on 16-QAM" statement and replaces it with a sharper one:

- **At rate 1/3 the BPSK-trained map *helps* 16-QAM** by ≈0.4 dB (71/300 → 109/300 at −3.5 dB; 148 → 177 at −3 dB) — the cap's protection against confidently-wrong bits still pays when the code has redundancy to spare.
- **At the ladder rate (2/3) it *regresses*** by ≈0.5 dB, consistently across the waterfall (148/300 → 108/300 at +1.5 dB; 190 → 155 at +2 dB; still 278 → 262 at +4 dB). Under heavy puncturing each surviving bit carries more weight, and a correction trained on BPSK's single LLR class mis-weights 16-QAM's two classes (axis-sign versus inner/outer, Eq. (8)) faster than the thinner code can absorb. The deployment gate ($\mu \leq 2$) protects exactly the rungs that exist — all ladder 16-QAM rungs are rate 1/2 and above — so it remains correct, but for a rate-dependent reason, not a modulation-dependent one.
- **Calibrated min-sum *beats* calibrated sum–product here** (215/300 versus 198/300 at −3 dB; 140 versus 115 at −3.5 dB) — the first configuration in these studies where the two calibrated kernels separate. The mechanism is the same one that sank raw-LLR SPA at BPSK: the BPSK-trained map leaves a *residual* miscalibration on 16-QAM's bit classes, sum–product consumes it as truth, min-sum ignores it. Min-sum is not merely "as good as" the exact rule — it is robust to imperfect calibration in a way the exact rule cannot be.
- The code families otherwise repeat the NORMAL-BPSK pattern at rate 1/3: LDPC min-sum holds a small edge over conv+Viterbi on raw LLRs (185/300 versus 148/300 at −3 dB, ≈0.2 dB), and raw-LLR sum–product shows only a mild penalty (168/300)

— nothing like the BPSK collapse, since at these operating points the 16-QAM LLR distribution rarely reaches the overconfident clipped regime.

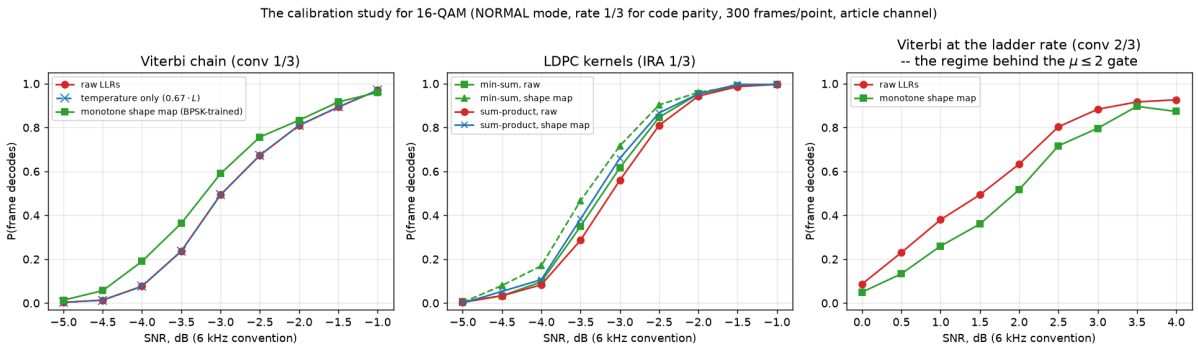


Figure 15: The calibration study for 16-QAM (reproduce with `experiments/qam16_recal.py`; parameters and per-point counts in `results/qam16_recal.json`). Left: Viterbi three-way at rate 1/3 — the map *helps*. Middle: LDPC kernels — calibrated min-sum beats calibrated sum–product for the first time. Right: Viterbi at the ladder rate 2/3 — the map *regresses*, the measured basis of the $\mu \leq 2$ deployment gate.

6.3 16-QAM

Gray-mapped 16-QAM with max-log LLRs (separate inner/outer bit metrics) fills the article's own "QAM could be applied" gap above 0 dB, adding ladder rungs 10–12: 706 bit/s at +0.7 dB, 941 bit/s at +2.6 dB, and 1059 bit/s at +4.7 dB (PER $\leq 10\,\%$, re-measured sensitivities). Figure 16 shows received constellations across that operating range: at +1 dB the sixteen clusters are washed together — which is exactly why the bottom rung pairs the modulation with the rate-1/2 code and soft LLRs rather than hard decisions — while at +5 dB the MMSE equalizer resolves the 4×4 grid cleanly enough for rate 3/4.

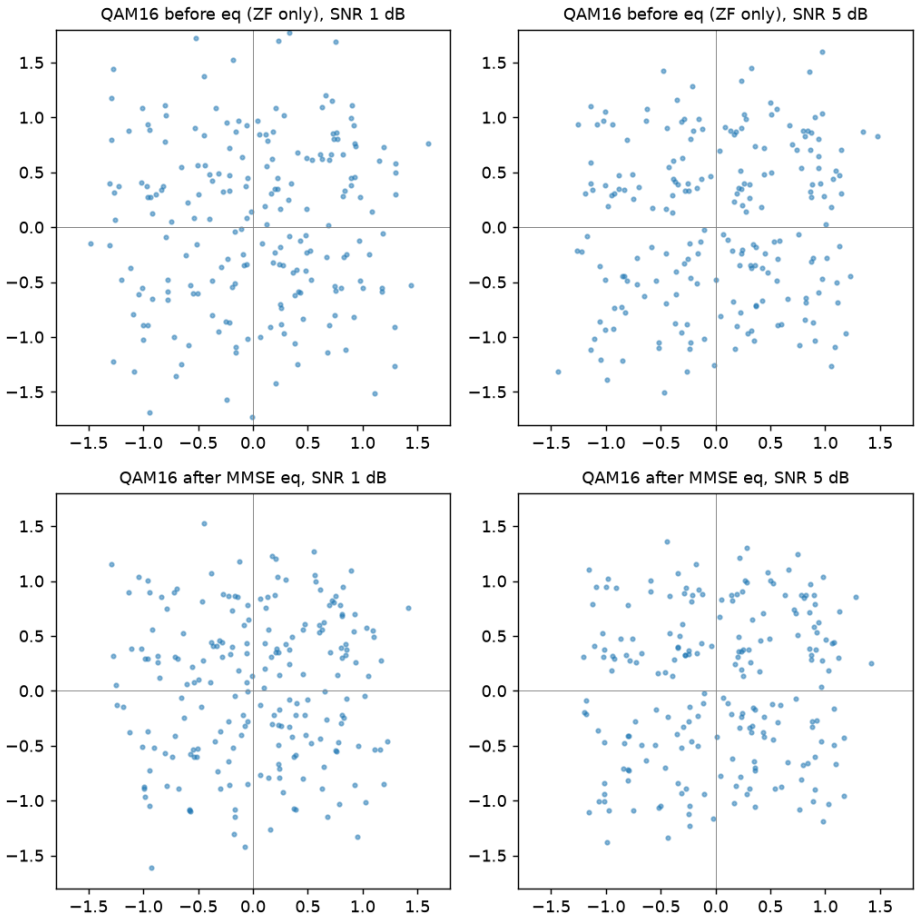


Figure 16: 16-QAM received constellations over the article channel (`experiments/figures.py`): before equalization (ZF pilot estimate only, top) and after Wiener-refined MMSE equalization (bottom), at the bottom (+1 dB) and top (+5 dB) of the 16-QAM operating range.

6.4 Chase-Combining HARQ

Because a CRC failure exports the data-block LLRs (Section 2), retransmissions can be soft-combined in the Chase manner [3]: the station stores the failed attempt's LLRs, and on the next decode tries the fresh frame alone, then fresh + stored (LLR sum), accepting whichever passes CRC (Figure 17). The combining is *blind* — the ARQ sequence number lives inside the undecodable payload, so the receiver cannot know the retransmission is the same frame — and *safe*, because the CRC arbitrates. Measured at −8.5 dB: second-attempt success 15/20 versus 10/20 without combining, worth ≈3 dB on a same-rung retransmission in the noise-limited regime. Fade losses benefit less: they are mostly synchronization-level failures that leave no LLRs to combine.

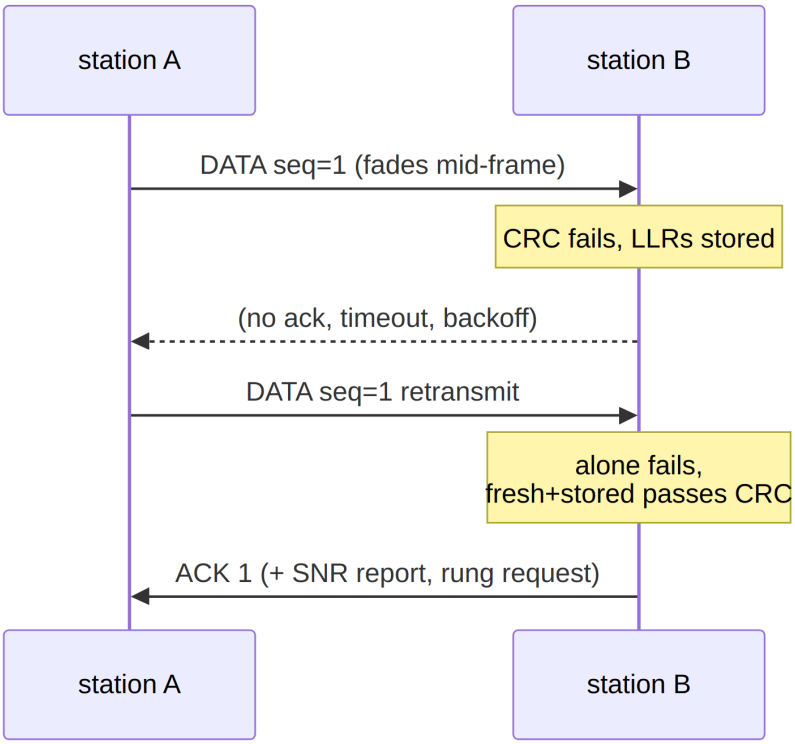


Figure 17: Chase-combining HARQ. No extra signalling: the stored LLRs come from the failed decode itself.

7 Link Layer

Two half-duplex stations form *two directed links*, each governed by its receiver — only the receiver knows its own noise floor. There is no shared "link mode" to negotiate; each side requests what it can hear.

7.1 The Rate Ladder

Thirteen operating points (mode × modulation × code rate, dominated rungs removed) span **7.8–1059 bit/s** over $-17.9 \ldots +4.7$ **dB** (Table 6). Every sensitivity is measured, not derived (`experiments/ladder_sweep.py`, 48 packets/point, recalibrated receiver; full PER curves in `results/ladder_recal.json`). Requests and grants are ladder indices carried in the link-control word. One quirk is documented: rung 1 (ROBUST BPSK 1/3) is dominated by rung 2 (faster, equally sensitive) and is kept only for ladder-index stability — it is never selected.

Table 6: The measured rate ladder (PER $\leq 10\,\%$ sensitivities, recalibrated receiver, 6 kHz-band SNR).

#	mode/mod/FEC	bit/s	dB	#	mode/mod/FEC	bit/s	dB
0	EXTREME BPSK 1/3	7.8	−17.9	7	NORMAL QPSK 1/2	353	−5.3
1	ROBUST BPSK 1/3	31	−11.7	8	NORMAL QPSK 2/3	471	−3.8
2	ROBUST BPSK 1/2	46	−11.8	9	NORMAL QPSK 3/4	529	−2.2
3	ROBUST QPSK 1/3	62	−11.3	10	NORMAL QAM16 1/2	706	+0.7
4	NORMAL BPSK 1/3	118	−7.6	11	NORMAL QAM16 2/3	941	+2.6
5	NORMAL BPSK 1/2	176	−7.3	12	NORMAL QAM16 3/4	1059	+4.7
6	NORMAL QPSK 1/3	235	−7.0				

7.2 The Link-Control Word

Every frame — data, ACKs, everything — piggybacks a 20-bit word in `Data.reserved`: `seq(2) | ack(2) | req_rung(4) | snr_report(4) | freq_corr(5) | flags(3)`. The SNR report uses 2 dB steps; the frequency-correction field carries ±120 Hz in 8 Hz steps (Section 8); flags are last-fragment, no-data (ACK-only), and priority-stream.

7.3 Rate Adaptation: Three Feedback Paths

Figure 18 shows the controller. On the receive side, per-frame SNR estimates pass through a fade-aware filter (the second-lowest of recent measurements, age-windowed to 60 s) before driving the rung request: upshift requires the target rung's sensitivity plus 2.5 dB of held margin, downshift triggers below 1 dB (the hysteresis band prevents ping-pong), and inbound *silence* decays the request and purges the measurement history — without the purge, stale pre-fade measurements resurrect a high rung the moment one frame gets through.

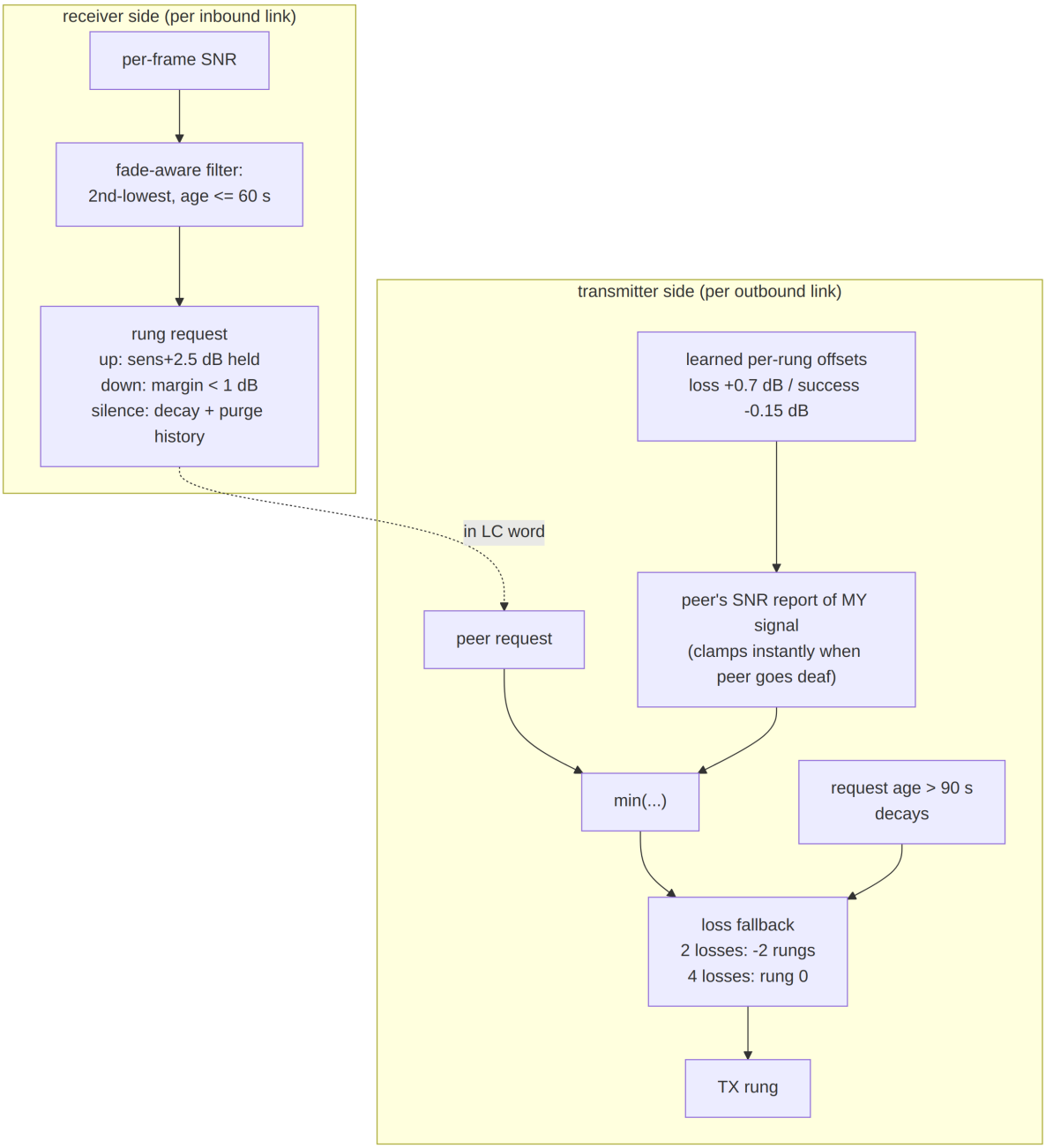


Figure 18: Receiver-driven rate adaptation. The three feedback paths have three different latencies: the peer's explicit request (one round trip), the peer's SNR report of *your* signal (instant clamp when the peer goes deaf), and loss-driven fallback (no feedback at all).

Figure 19 walks the same controller through one session. The three paths differ in what they need in order to act: the peer's explicit request needs a round trip, the peer's SNR report of *our* signal clamps the cap as soon as any frame arrives, and the loss fallback needs no feedback at all — which is the only path still available once the peer has gone silent.

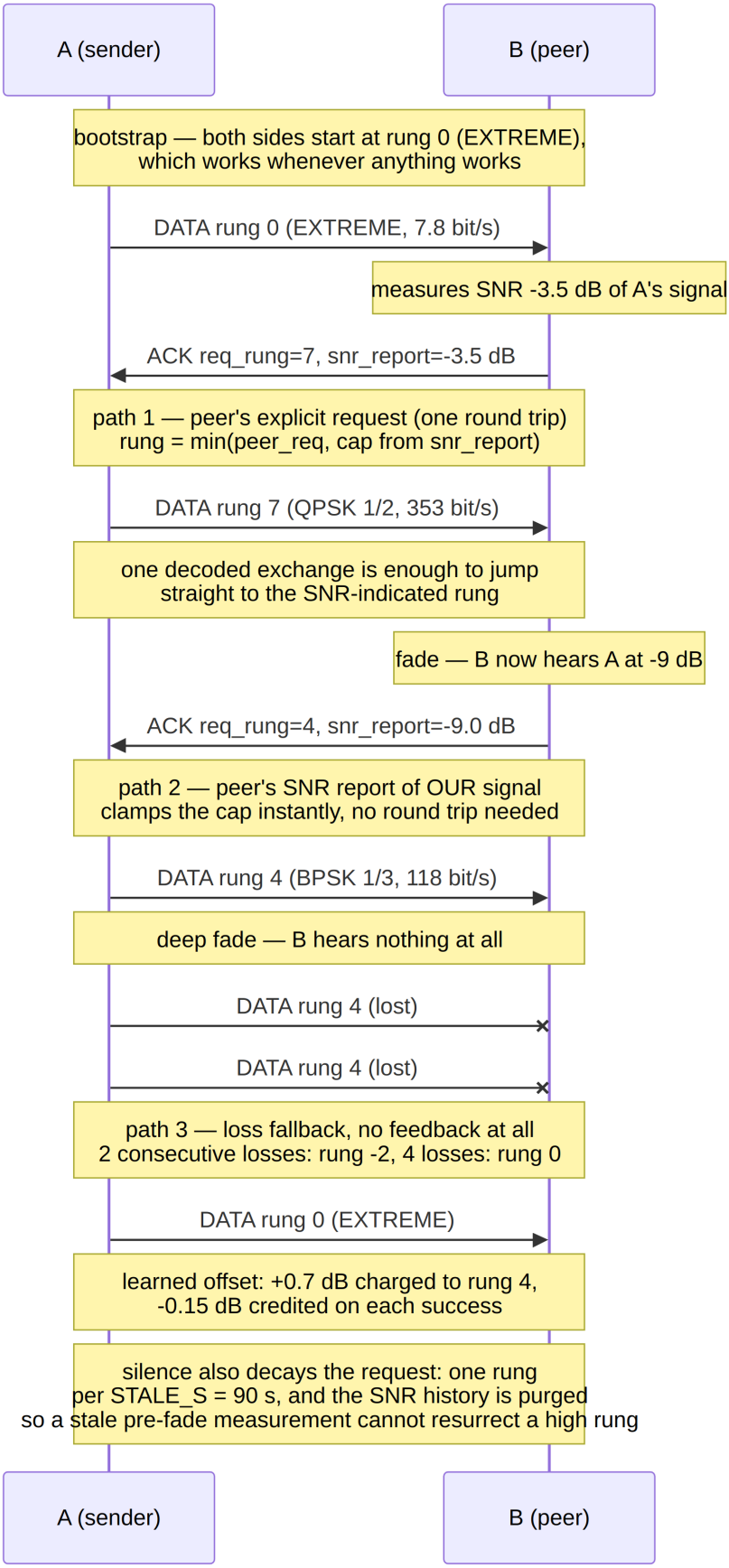


Figure 19: Rung selection across modes: bootstrap at rung 0 (EXTREME), the jump to the SNR-indicated rung after a single decoded exchange, a fade clamped by the peer's report, and the loss fallback when no feedback arrives at all. Silence is not neutral — it decays the request by one rung per 90 s and purges the SNR history.

On the transmit side, the rung is the minimum of the peer's request and a cap derived from the peer's SNR report of our own signal, adjusted by *learned per-rung offsets* (+0.7 dB on loss, −0.15 dB on success — correcting the static table per deployment), with loss fallback (2 consecutive losses → −2 rungs, 4 → rung 0) and staleness decay. Bootstrap is trivial by construction: both sides start at rung 0 (EXTREME), which works whenever anything works, and one decoded exchange carries enough measurement to jump directly to the SNR-indicated rung.

7.4 ARQ, QoS, and Preemption

ARQ is stop-and-wait with piggybacked ACKs (the 2-bit sequence number suffices), enhanced by the HARQ combining of Section 6. An invariant worth recording: sequence numbers 0–7 are all legitimate, so "nothing received yet" is represented by `None`, never a magic value — the initial implementation used 7 as a sentinel and deadlocked the moment a real frame carried sequence 7.

QoS is policy, not machinery: three classes (control > interactive > bulk) with strict priority; per-class air-time caps size fragments (a 27-byte EXTREME frame is 43 s of air time, so latency budgeting happens at segmentation); control and ACK traffic rides one rung below bulk for extra margin. The priority stream *preempts* an in-progress bulk message at fragment boundaries — the priority-stream LC flag selects one of two reassembly buffers — so an urgent message injected mid-transfer arrives ∼23 s before the bulk completes instead of after it.

7.5 Burst Windows and Streaming

Bulk transfers use selective repeat: a window of fragments goes out back-to-back, answered by one bitmap acknowledgment (NACK-by-omission). Where the PHY offers streaming (Section 2.3), the whole window is sent behind a *single* preamble and header rather than one preamble per fragment. The packets are byte-identical to per-frame burst fragments — one spare bit in the sub-header's index byte marks a fragment as streamed — so a receiver without streaming support still decodes the first block of every burst as an ordinary fragment. That property is what makes the fallback safe rather than fatal.

The window is chosen once, at engage, as min(operator ceiling, buffer cap, fragment count), then clipped by an air-time cap so that a single transmission can never outlast a plausible fade — everything in a streamed window is exposed before any of it is acknowledged. Sizing the window to the transfer is what makes a short transfer cost exactly one acknowledgment, which is the deferred-NAK arrangement of LTP and CFDP arrived at through a constant rather than new wire format. Measured on a 14 kB file over the virtual channel: window 4 takes 14 transmissions, window 8 takes 9, window 16 takes 6.

Resizing the window *during* a transfer was implemented and then reverted on the evidence. Halving it on each streamed-window timeout, rather than striking out of streaming, collapsed the window 16 → 8 → 4 → 2 → 1 with no path back — it grows only on a fully acknowledged window, which never happens inside a fade — and produced 196 transmissions with 72 timeouts against 134 and 9 for the strike-out path. On a clean channel the two were indistinguishable. When a fade is what breaks a burst, the correct response is to stop streaming, not to stream less.

Three conditions end streaming for a transfer, and they are not the same kind of statement. A bitmap that acknowledges at most one fragment of a window of three or

more is the signature of a peer decoding only the first block: that is a statement about the *peer*, and it is remembered across transfers, because a receiver unable to follow a stream will not learn to between them. Repeated timeouts and a transmitter-side build refusal describe the *channel* and the local buffers respectively, and deliberately do not stick — a fading channel raises timeouts against a peer that streams perfectly well, and making that sticky would disable streaming for the session on a capable link. The peer verdict is not permanent either, since a deep fade can forge the one-fragment signature: streaming is probed again after eight transfers, which bounds a wrong verdict to one wasted window per retry period while a genuinely incapable peer costs two windows per period instead of two per transfer.

On the C stack this probing later became the fallback rather than the mechanism: a three-leg capability handshake (Section 12.14) lets a peer *declare* streaming, window ceiling and message size before the first window flies, and the NOACK inference above remains only as the channel's veto.

7.6 The Reply Timer

A station that has transmitted and expects an answer must decide how long to wait. The budget splits along what is knowable. The reply's *air time* is computable exactly from the rung and the expected reply length, and it swings by a factor of forty across the ladder, so smoothing it would be meaningless. Everything else — the peer's turnaround, its decode time, its wait for the channel to clear, its scheduling, and the error in our guess of which rung it will answer at — is latency that can only be measured.

The air term is therefore computed and the overhead is estimated in the manner of RFC 6298: a smoothed mean and variance ($\alpha = 1/8$, $\beta = 1/4$) with the budget set at $\mathrm{srtt} + 4\,\mathrm{rttvar}$. Karn's rule keeps ambiguous exchanges out of the estimator — a reply that follows a retransmission cannot be attributed to a particular transmission — and a timeout doubles the timer until a clean exchange clears the backoff. Against a peer that consistently takes 0.40 s, the 2.30 s bootstrap guess converges to 0.40 s.

This replaces a fixed margin that had already caused a misdiagnosis. When the host could not sustain the demonstration harness's 25× time compression, the two stations' sample-derived clocks drifted apart — the mostly-transmitting station stayed current while the one decoding three receivers fell behind — and the transmitter timed out before the receiver had finished the burst in its own timeline. Every timeout landed on one station and none on the other, with the reply arriving immediately after each: a signature that reads exactly like a protocol failure and is not one.

7.7 Simplex Channel Access

Both stations share one frequency and are deaf while transmitting. Figure 20 shows the access state machine. The load-bearing rules: carrier sense before keying; the listen window is sized from the *expected reply's* air time at the peer's rung; a timeout on a *busy* channel is not a loss (the peer may be replying at a slower rung); random 1–6 s backoff breaks the symmetry of two stations keying together; and a station replies only to frames that carried data — no ACK-of-ACK loops.

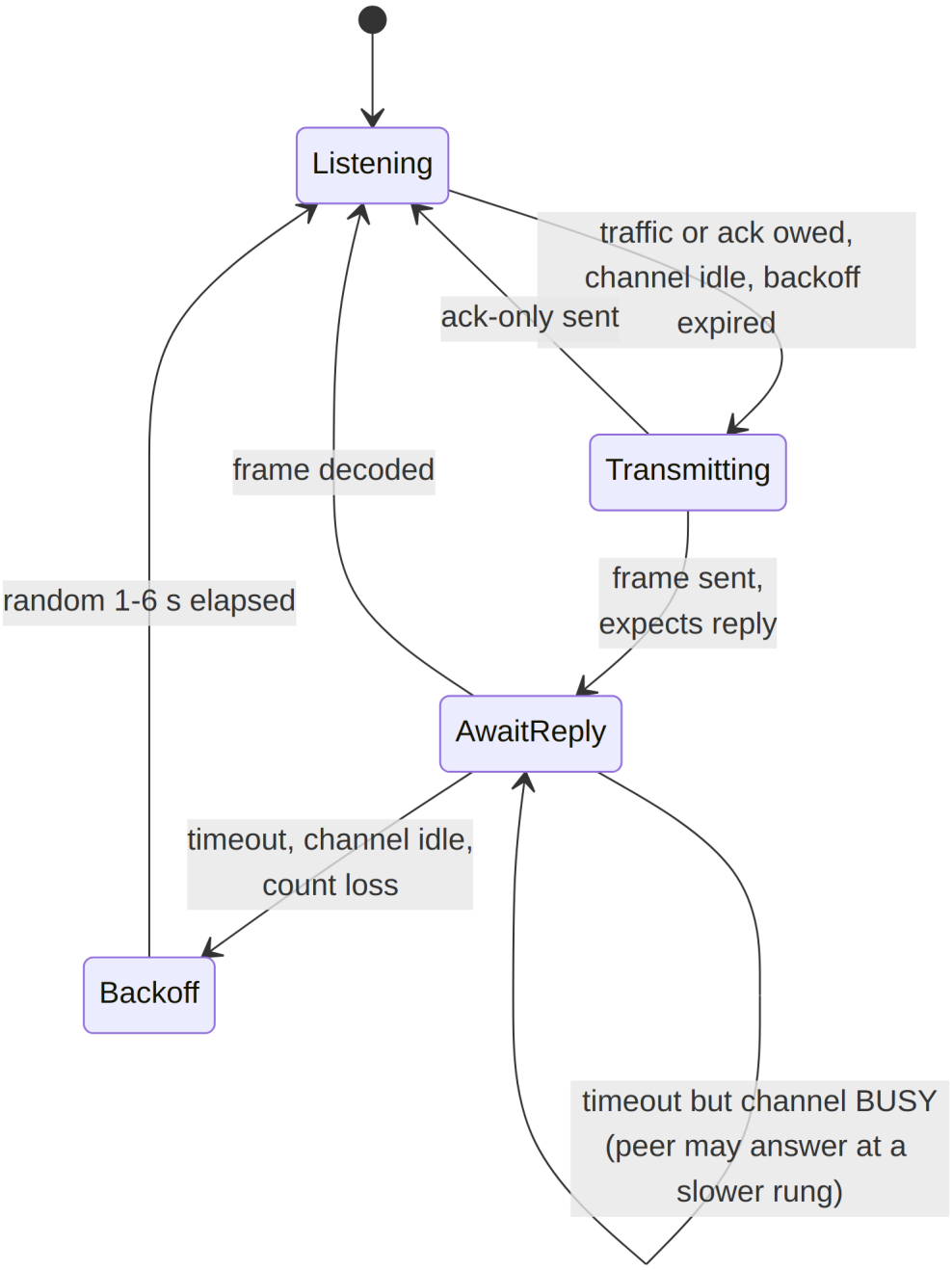


Figure 20: Simplex channel-access state machine (`LinkStation.poll_tx`).

The whole-system test (`experiments/simplex_session.py`) runs two stations with bidirectional QoS traffic and a −14 dB fade in the middle of the bulk transfer: all messages deliver bit-exact in QoS priority order, the fade costs four timeouts and one fallback-and-recover cycle, and the session terminates without deadlock at 11–22 bit/s effective on the shared channel. Figure 21 shows the controller-level simulation: bootstrap at EXTREME, a fast climb to the QPSK rungs, loss-driven fallback within two frames of a sudden fade, and recovery.

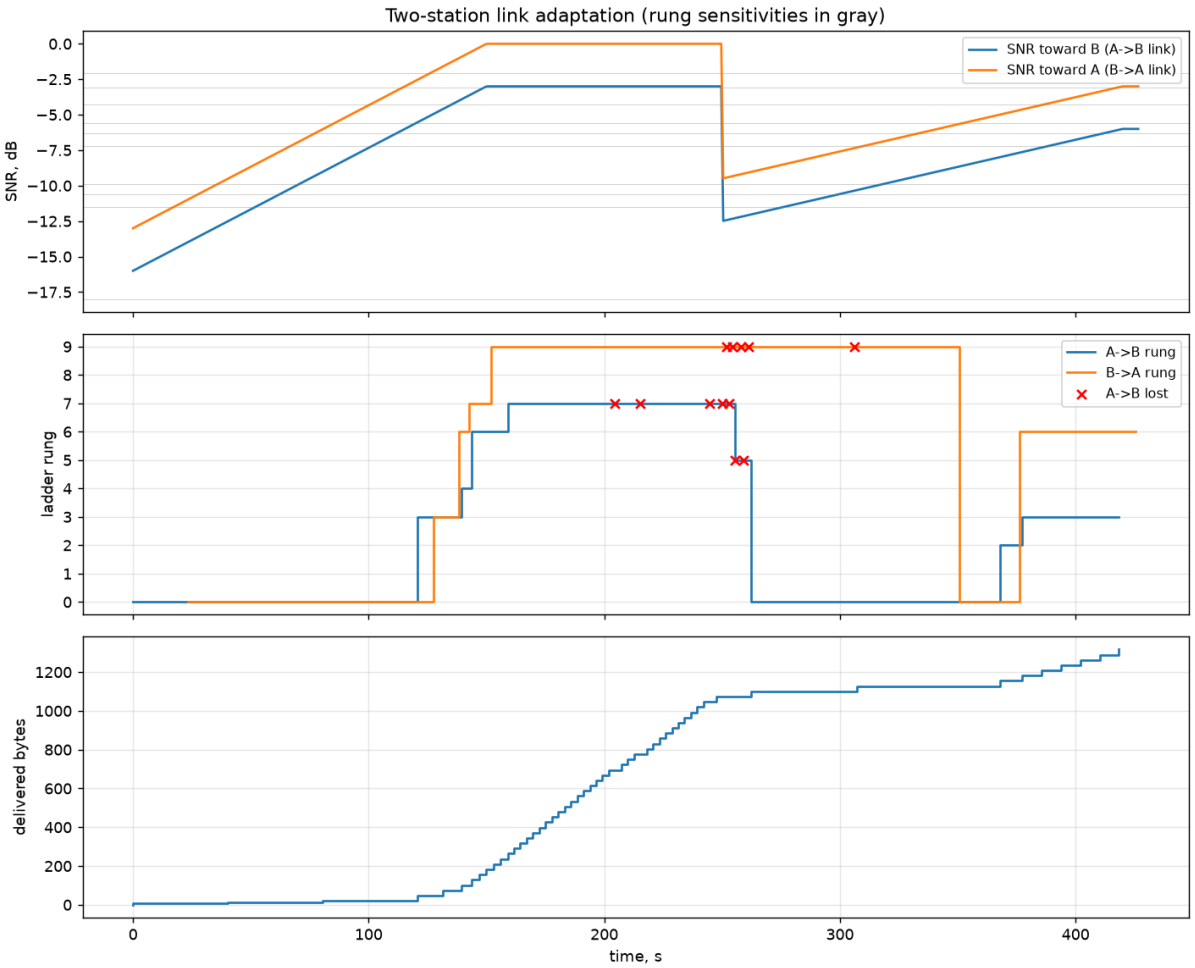


Figure 21: Controller-level two-station simulation (`experiments/link_adaptation.py`): asymmetric channel, −16 dB start, sudden fade; 1290/1500 bytes delivered at 24 bit/s effective goodput including ACK overhead and turnarounds.

7.8 Broadcast: Delivery Without Acknowledgment

Speech and telemetry cannot wait for a retransmission, so they need a mode in which nothing is repeated and the receiver's only contribution is a report that it was heard. The transport for this already exists: a broadcast is a streamed burst (Section 2.3) with the acknowledgment machinery subtracted — no acknowledgment request, no window, no selective repeat, no reply timer. What has to be added is the one thing a burst does not need. A burst has exactly one listener, who was already there; a broadcast must let a receiver that tunes in mid-transmission join. Every group therefore re-sends the preamble and opens with a SYNC frame carrying the stream descriptor, so a late listener acquires at the next group boundary rather than waiting for the broadcast to end.

Framing is two bytes per frame: a SYNC bit, an EOS bit, a six-bit sequence number used *only* for loss statistics, and the count of valid payload bytes in that frame. The obvious model to copy would be RTP, and copying it would be a mistake: its twelve-byte fixed header is 91 ms of air time at the top rung, so a 20 ms-packetised stream would carry 1.75 bytes of payload behind 12 bytes of header. What survives the transplant is what a receiver cannot work out for itself — sequence, stream boundaries and payload type. What does not is the timestamp (this link's delay is deterministic, so a frame's position

in the stream *is* its timestamp), the synchronisation source identifier, and the per-packet payload type, which rides the SYNC frame instead.

Carrying the length in *every* frame rather than only in the EOS frame costs about 4 % at 26-byte frames and buys a property that only matters when nothing is retransmitted: every frame is self-delimiting, so losing the one frame that would have carried the length does not mis-size the payload.

The rate ladder decides what can be carried at all. Codec2 at 700 bit/s fits only rungs 11–12 (941 and 1 059 bit/s user rate, +2.6 and +4.7 dB); Codec2 450 fits rungs 8–9; below rung 8 this mode carries telemetry only, down to EXTREME's 7.8 bit/s. That envelope is set by the waveform, not by the protocol above it.

Table 7: Broadcast delivery, 200-byte payload in groups of four (`experiments/broadcast_demo.py`, 20 trials per point). Nothing is retransmitted, so "recovered" is payload bytes reassembled against bytes sent.

	all recovered	≥90 %	breaks down
speech, rung 12 (16-QAM 3/4)	+9.5 dB	+5.0 dB	+2 dB
telemetry, rung 7 (QPSK 1/2)	+1.5 dB	−3.0 dB	−6 dB

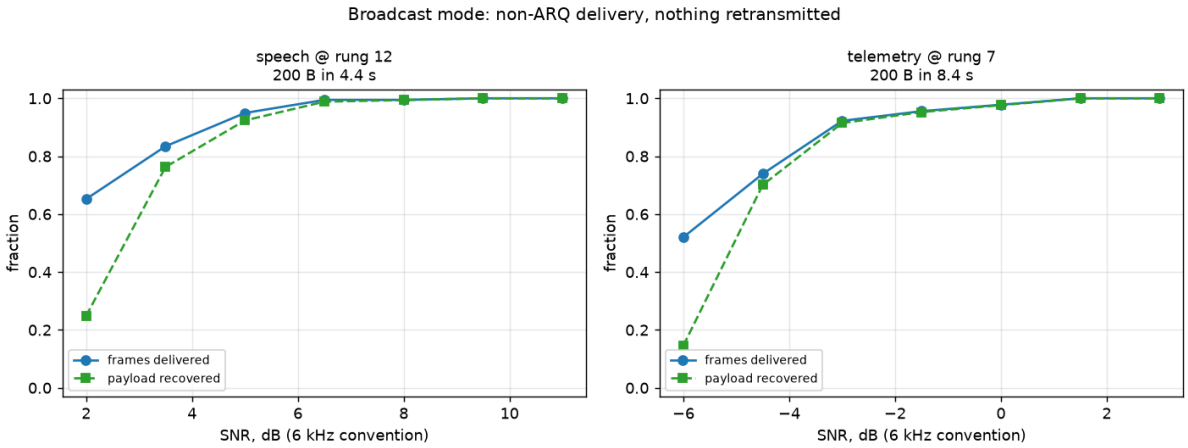


Figure 22: Non-ARQ delivery against SNR. Frames delivered runs above payload recovered at the low end because a frame lost from the middle of a group costs its own bytes while still being counted in the span.

7.8.1 What a Broadcast Exchange Actually Looks Like

Figure 23 traces a 14162-byte file broadcast over the virtual channel. Three properties of the sequence are worth drawing out, because each was established by a failure rather than by design.

The rung is fixed at engage. A broadcast cannot renegotiate, so it runs at whatever the link last established — and the link must be established first: below rung 4 a broadcast cannot be carried at all, so the sender probes with a control frame and waits rather than refusing outright.

The listener stays silent. On a simplex channel, transmitting is precisely what stops a station hearing, and a broadcast is a train of transmissions with gaps of well under a

second between turns. Those gaps read as an idle channel to carrier sense, so a station holding a pending reply fires into one and is deaf as the next transmission's preamble goes past. Losing a group's preamble costs the entire group: measured, a single 0.8 s reply cost exactly 16 frames on a +20 dB channel. A station that is hearing a broadcast therefore treats the channel as busy, which suppresses the transmission *and* stops the link layer scoring the resulting reply timeouts as losses.

EOS ends the walk, not the group descriptor. The final group of a broadcast is normally short, but the descriptor can only advertise the nominal size — it carries $\log_2$ of the group. A receiver that trusts the descriptor walks blocks that were never transmitted, and while stepping through blocks it is not running the preamble detector: 15 phantom CRC failures, each costing a block-time of deafness. The EOS bit is the stop signal, with a consecutive-failure bound for the case where the EOS frame is itself lost.

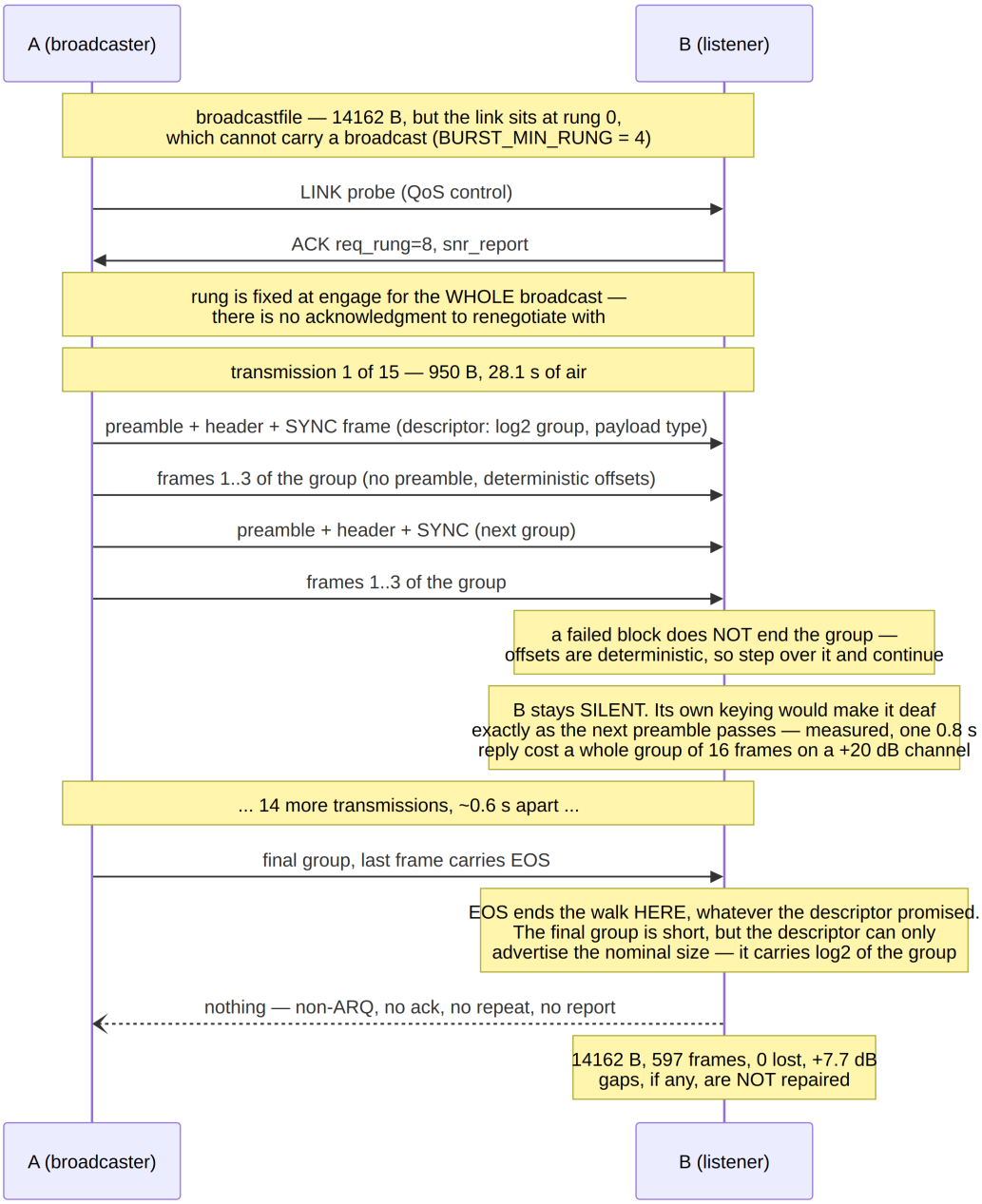


Figure 23: A file broadcast: link probe, then transmissions carrying one preamble per group, with no acknowledgment of any kind in return. The listener's silence is a protocol requirement, not an omission.

7.8.2 Group Size Under Fading

Group size is the one free parameter of the mode, and on a clean channel it looks free in both directions: a 14162-byte file arrives complete and byte-identical at groups of 4, 8 and 16 alike. That result says nothing, because it exercises only half of the trade. One preamble amortised over more frames is the benefit; the cost is that a missed acquisition takes the whole group with it, and a broadcast retransmits nothing. The cost term only fires when acquisitions are actually missed, which on a clean channel they are not.

Figure 24 runs the same three group sizes against SNR with Rayleigh fading enabled (0.5 Hz, roughly a 2 s fade period), 30 trials per point, all three sizes facing the same

fading realisation at each point so the comparison is paired rather than three independent samples of a noisy quantity.

The left panel is the metric that matters, and it is flat: mean payload recovery is 64.4 %, 62.5 % and 62.3 % for groups of 4, 8 and 16. The spread between them is smaller than the point-to-point scatter, so on this evidence a larger group buys nothing.

The right panel is where the trade becomes visible. Frames *decoded* does rise with group size — there are fewer preambles to miss — but payload recovered does not follow it, and the gap between the two is the payload lost behind a missed acquisition. That gap is monotonic in group size and well outside the scatter: 4.8, 8.6 and 15.5 points on average. A group of 16 at +2 dB decodes 89.3 % of frames while recovering 65.0 % of the payload.

The practical reading is that group size trades *granularity* of loss, not quantity. Above +5 dB all three are equivalent and the larger group is free: with the same 0.5 Hz fading and a +20 dB nominal channel, the 14162-byte file still arrived complete and byte-identical at a group of 8, the fading having pulled the measured SNR to +2.4 dB and the link down one rung. Below that the larger group concentrates the same loss into fewer, longer holes. For telemetry reassembled whole this is strictly worse; for speech, where one long dropout is more objectionable than several short ones, worse again.

The demonstration application caps the group at 8, which is the ceiling its static receive buffer imposes rather than a value this figure argues for — on these measurements 4 is the better default whenever fading is expected, and the choice should follow the channel rather than the buffer.

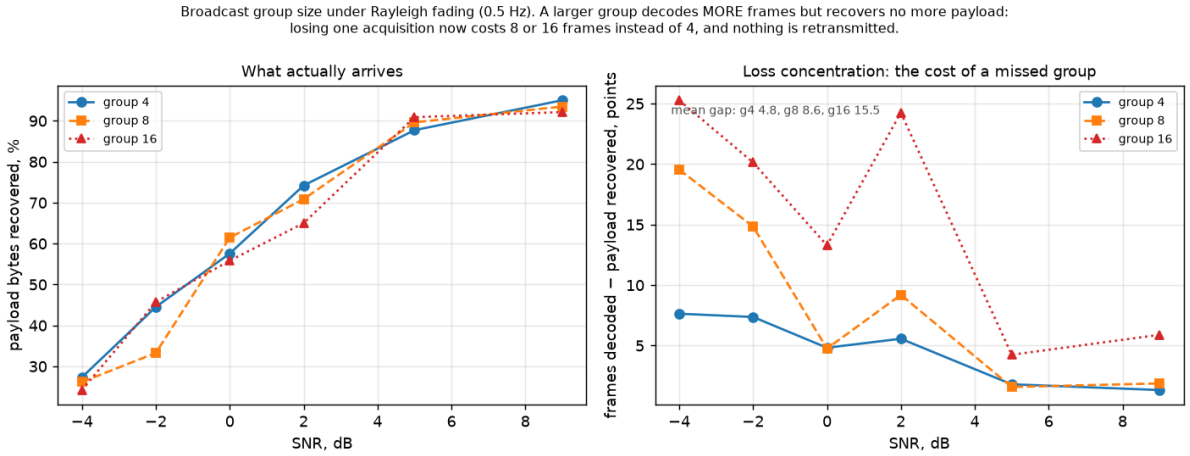


Figure 24: Broadcast group size under 0.5 Hz Rayleigh fading (`experiments/broadcast_fading.py`, 30 trials per point, paired realisations). Payload recovery is indistinguishable across group sizes; what changes is how the loss is distributed.

7.8.3 A Reception Report: Measured and Rejected

The gap this leaves is real. A 14162-byte broadcast ran its entire 450 s at rung 8 while the listener was decoding it at +7.7 dB — enough for rung 12, which would have cut each turn from 28.1 s to 5.1 s. Worse, broadcast frames deliberately never reach the ARQ path, so during a broadcast *neither* station's controller sees traffic and the staleness decay walks both ladders down: after 454 s of broadcasting, the next file transfer opened at rung 4 and spent 37.0 s on a frame that rung 12 delivers in 5.1 s.

A listener report was therefore implemented, in a quiet window left by the sender, and then removed. Table 8 is why.

Table 8: Listener reception reports, 14162-byte broadcast, 597 frames sent, +20 dB channel. The mode repairs nothing, so every lost frame is lost for good.

	frames lost	turns	rung
no report (run 1)	0	15	8 fixed
no report (run 2)	4	15	8 fixed
report per group	32	10	$8 \rightarrow 11$
report per turn	28	16	$8 \rightarrow 7$

Rate-limiting the report to one per turn removed the storm and bought nothing: the same turn count, 28 frames lost permanently, and the rung moved *down*. The feedback is self-defeating in a way tuning cannot fix — the listener's own transmission causes losses, the losses make the controller conservative, and the broadcast gets slower. The root cause is that the listener infers "the sender has paused" from quiet on its own clock, while it decodes *behind* the sender (lag of seconds; 11.7 s was measured on a 16-block group), so its reply lands inside the next transmission whatever threshold is chosen, and a window wide enough to absorb the lag costs more air than the rung climb saves.

Doing this properly requires the *sender* to signal an explicit report slot on the wire — a flag on the last frame of each turn, so the listener answers a marker instead of inferred silence. That is a frame-format change across the float, fixed and C twins and their golden vectors, and it is left as future work rather than approximated at the application layer.

Telemetry reaches 90 % delivery at ≈ -3.1 dB against rung 7's -5.3 dB acknowledged-mode sensitivity. That $\approx$2 dB is the honest price of non-ARQ: a frame that fails is simply gone, and a group whose preamble is missed takes four frames with it. A receiver that tuned in half-way through the first group of seven recovered 90 % of the remainder — it cannot, of course, recover what it never heard.

Two findings from building this generalise beyond broadcast.

The first concerns the detector. `detect_preamble` takes a *global* argmax over whatever slice it is handed, so a slice containing several groups makes it lock the strongest preamble rather than the first, and every group before that one is silently lost — 300 samples of lead-in were enough to skip a whole group. The slice must therefore be bounded on *both* sides so that it holds exactly one preamble. Starting the slice on a preamble is not sufficient; nor can the far edge be found by searching forward for the next preamble, since from inside a group the detector false-locks on data. The bound is geometry — preamble plus header plus blocks — and because the first decode necessarily runs before that geometry is known, the receiver learns the extent from it and then restarts the walk with the bound in place. Anything that scans a recording containing multiple frames has the same exposure.

The second concerns LLR calibration (Section 5), which matters more here than under ARQ for a simple reason: a frame the decoder cannot recover is not retried, it is lost. Enabling the measured reliability map moved telemetry from 30.6 % to 69.8 % of payload recovered at -4.5 dB and from 3.4 % to 12.0 % at -6 dB, while changing nothing above -3 dB where delivery is already near perfect. It is therefore on by default in this mode. The map is gated to $\mu \leq 2$ because it was trained on BPSK/QPSK statistics, and the

speech column of the sweep came back *bit-identical* with the map enabled — an incidental but clean confirmation that the gate does what it claims.

What broadcast gives up is the feedback path. The rate-adaptation mechanisms of Section 7 all depend on the peer replying, and here it does not. Either the sender schedules periodic listening gaps in which receivers return the statistics above, at the cost of the continuous transmission that makes broadcast cheap, or it does not adapt at all: announce a rung in the descriptor and hold it. STANAG 5066's non-ARQ delivery mode takes the second course, and it is the right default — a broadcast has no single listener to adapt to.

8 AFC Frequency Netting

The receiver measures the peer's carrier offset on every decoded frame anyway (`stats.cfo_hz`), so the link closes the loop: the 5-bit LC field carries "shift your carrier by $-X$ Hz" requests (± 120 Hz per frame, 8 Hz steps, ± 12 Hz deadband), and the peer trims its *shared* reference oscillator. Trimming the shared reference moves TX and RX together — which is exactly why asking the peer to correct beats retuning locally (Section 9). Corrections are applied at **half gain**: both sides act on measurements that are one frame stale, and full-gain corrections would swap the offset between the stations each turn instead of converging (Figure 25).

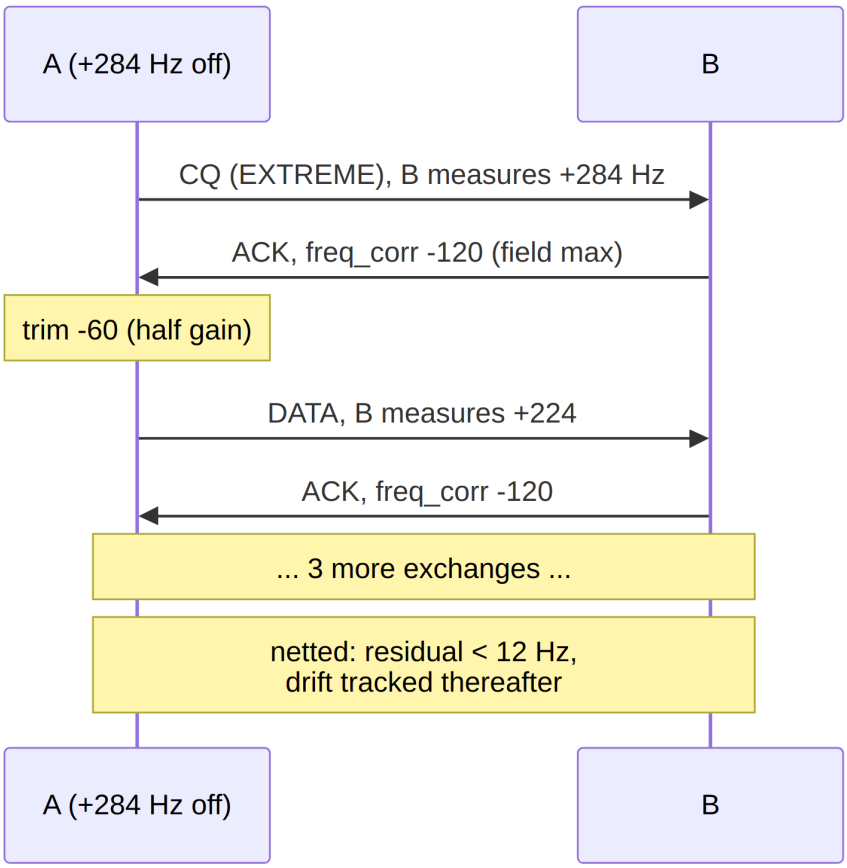


Figure 25: AFC netting from a +284 Hz initial offset.

8.1 Guard Rails

The loop observes only the *differential* offset, so without bounds the pair's common frequency random-walks over a long session — out of the channel slot, into bystanders' passbands, or off the rig's trim range. Two mechanisms bound it:

- `afc_max_trim_hz` (default ±150 Hz) hard-clamps each station's *cumulative* trim. Requests beyond the budget are partially honoured or refused, the peer's own headroom carries the rest, and any residual is left to the modem's ±300 Hz CFO tolerance — the link keeps working, just without full netting.
- `afc_anchor=True` designates a station (e.g. the CQ caller) that never trims, pinning the absolute frequency entirely; the peer does all the correcting.

8.2 Measured Behaviour

`experiments/afc_netting.py` runs two worn-TCXO rigs on 14.2 MHz starting +284 **Hz apart** (near the modem's ±375 Hz limit) with opposing thermal drifts: netted to within the deadband in ≈5 correction exchanges (≈104 s — EXTREME bootstrap frames dominate the wall clock), steady-state $|\mathrm{CFO}| \leq 11$ Hz under continuing 0.09 Hz/s relative drift, 400/400 bytes delivered. The guard scenarios behave as designed: default budgets net fully (final |CFO| 3 Hz, trims clamped at $-150/+144$ Hz); deliberately tight ±40 Hz budgets stop at their limits leaving a 222 Hz residual *by design*, with the link still delivering on CFO tolerance alone; an anchored station A forces B to absorb the full +292 Hz and the pair's absolute frequency never moves. Beyond robustness margin, a netted link could shrink EXTREME's per-symbol frequency-search range (compute) and eases mask-grid detection at the extremes.

9 RF Layer and Channel Models

Two channel models coexist. The audio-domain simulator (`channel.py`) is the article's model — timing offset, CFO with quadratic drift, optional Rayleigh fading, multipath $[1, 0, 0.4, 0, 0, 0.2]$, AWGN, BSC block flips ($p = 10^{-3}$) and BEC block erasures ($p = 2 \cdot 10^{-2}$) — and drives most sweeps. The RF layer (`rf.py`) models the actual transceiver chain, so carrier offsets *emerge from oscillator physics* instead of being injected as parameters.

9.1 The SSB Chain

Simulation runs at a 48 kHz IF rate with a 12 kHz carrier standing in for the RF carrier; LO errors are specified in ppm of the nominal RF frequency (e.g. 7.1 MHz), so audio-band offsets come out physically correct (Figure 26).

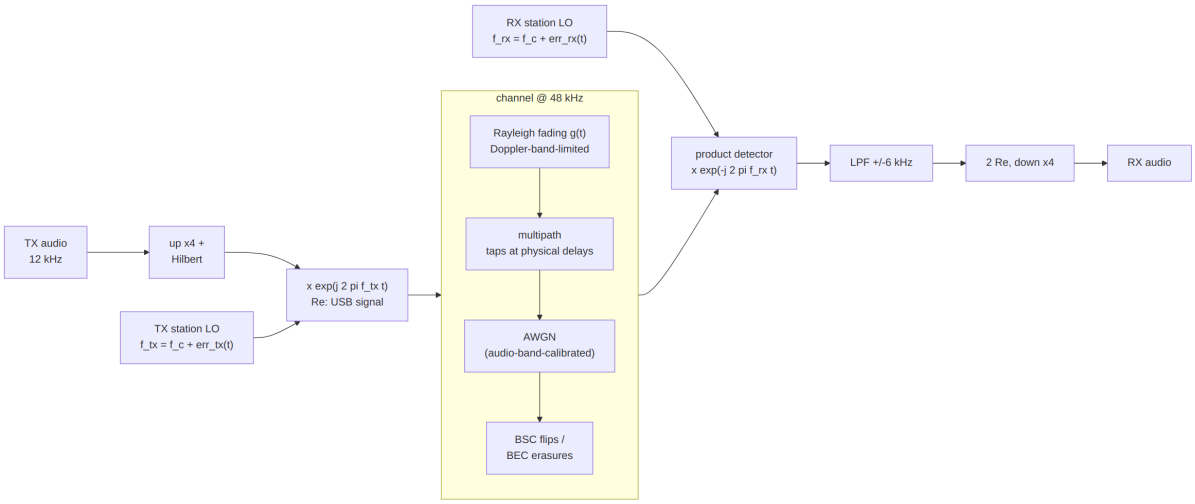


Figure 26: The RF chain: USB modulator, 48 kHz channel (fading, multipath, AWGN, BSC/BEC), product detector.

9.2 The LO Model

Each `StationRF` has *one* reference oscillator shared by its TX and RX, as in a real transceiver:

$$\mathrm{err}(t) \;=\; f_{\mathrm{rf}} \cdot \mathrm{ppm} \cdot 10^{-6} \;+\; \mathrm{drift}_{\mathrm{Hz/s}} \cdot t \;+\; \mathrm{trim}_{\mathrm{Hz}},$$

and the audio CFO between two stations is $\mathrm{err}_{tx}(t) - \mathrm{err}_{rx}(t)$ (Figure 27). Validated against the modem's own CFO measurements to 0.1 Hz at +99.4, −106.5, and +284.0 Hz scenarios. `trim_hz` is the AFC hook of Section 8.

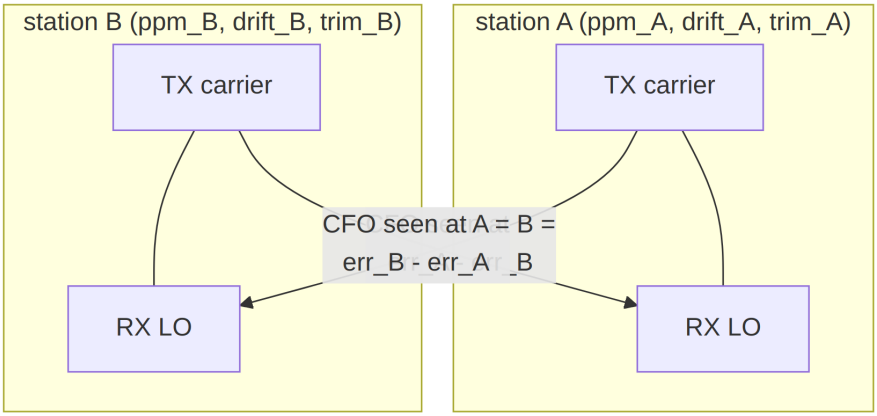


Figure 27: Per-station shared-reference LO model: where CFO comes from.

9.3 Calibration Subtleties

Three traps are documented in code comments because each silently costs dB if gotten wrong:

- **AWGN sizing.** The product detector's Re(·) halves *uncorrelated noise* power but not the coherent signal, so the RF noise variance factor is $2 \cdot P_{\mathrm{sig}} \cdot 10^{-\mathrm{SNR}/10}$ — not the naive bandwidth ratio, which is 3 dB off. With the correct factor the audio-band

SNR convention matches the audio-domain simulator, so all measured sensitivities carry over (9/10 decodes at −5 dB through the full RF chain).

- **Rayleigh fading.** Complex Gaussian band-limited to the Doppler bandwidth (0.1–0.5 Hz, typical HF QSB), unit mean power, applied to the analytic signal; AWGN is sized from the *average* faded power, so instantaneous SNR swings around the nominal value.
- **Multipath at RF.** The audio-domain tap delays are physical, so taps map to interpolation-factor-spaced positions at the RF rate.

9.4 The Recorded Demonstration

`experiments/demo_wav.py` records a complete two-station session through this chain: station A at +6 ppm and B at −8 ppm on 7.1 MHz (A→B CFO ≈ +99.4 Hz, handled invisibly by the protocol). The output WAV is the audio of a *passive monitor* receiver with its own +2 ppm LO — A's frames appear at ≈ +28 Hz and B's at ≈ −71 Hz in the blind-decoded transcript (Appendix C), per-station CFO fingerprints with the thermal drift visible across the session.

10 Fixed-Point RTL Reference Model

`ofdm_phy/fixed/` is an integer-only twin of the modem, structured the way an FPGA/ASIC datapath would be. It is the golden reference a future RTL implementation verifies against; `experiments/fixed_point.py` (24 checks, Appendix B) cross-validates it against the float model continuously.

10.1 Receiver Datapath

Figure 28 shows the complete integer receive path, from int16 audio to decoded packet. The model covers the *full* frame family: convolutional and LDPC FEC, BPSK/QPSK/16-QAM, all three link modes, HARQ chase combining, and the optional calibrated-LLR mode — on the transmit side as well: `build_frame(fec="ldpc")` emits `ver=2` frames (the LDPC accumulator encoding is already pure integer, so only the header/codec switch was needed), and the Q15 constellations cover BPSK, QPSK, and Gray 16-QAM (per-axis levels $\{\pm 1, \pm 3\}/\sqrt{10}$).

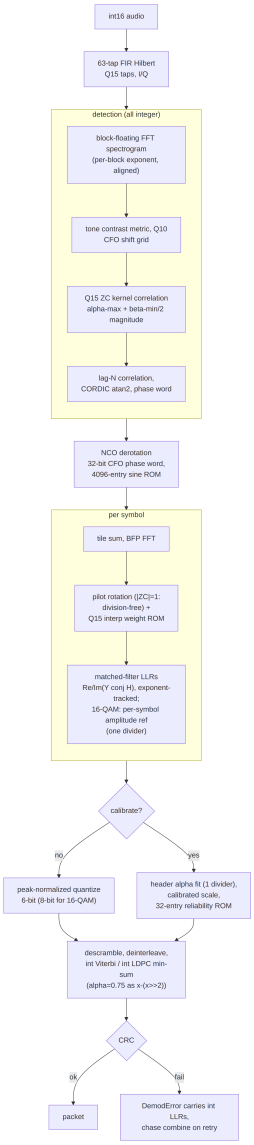


Figure 28: Fixed-point receiver datapath (all integer).

10.2 Primitive Blocks

Table 9: Fixed-point primitives and their measured quality.

Block	Structure	Quality
FFT	radix-2 DIT, Q15 twiddle ROM, per-stage $\gg 1$ (built-in $1/N$), BFP wrapper	59 dB SQNR
Hilbert	63-tap type-III FIR, Q15	61 dB SQNR
NCO	32-bit phase accumulator, 4096-entry Q15 sine ROM	-67 dBc
CORDIC	16-iteration vectoring, angle in phase-word units	< 0.001 rad
Viterbi	6-bit LLRs (8-bit for 16-QAM data), ± 1 expected symbols (adds only), int32 metrics	exact
LDPC	integer normalized min-sum, $\alpha = x - (x \gg 2)$	matches float min-sum

10.3 RTL-Oriented Conventions

- **CFO is a 32-bit phase-increment word** end-to-end (one turn $= 2^{32}$); Hz exists only at API boundaries.
- **Divisions are minimised.** Channel estimation is division-free ($|\text{ZC pilot}| = 1$, so the estimate is a rotation); pilot interpolation is a Q15 weight ROM. The remaining dividers: the calibration α fit (one per frame), the 16-QAM amplitude reference (one per symbol), and the detection metrics.
- **Scale tracking.** BFP exponents are carried through every energy and LLR comparison (scale $\propto 2^{2 \cdot \text{exp}}$); ZC correlation magnitude uses the α-max + β-min/2 approximation.

Two intentional deltas versus the float model: NORMAL-mode tone detection uses a 256-point FFT (float: 128) so the coarse CFO grid is fine enough for an unambiguous lag-N residual; and the demodulator uses the slew-limited frequency-search tracker for *all* tile factors — the float model's polynomial phase fit is not RTL-friendly, and measurably weaker at the edge anyway.

10.4 Measured Results and One Surprise

The fixed TX waveform correlates 0.99998 with the float waveform; the full fixed/float TX×RX cross-matrix decodes; NORMAL-mode PER is within $\approx$0.5 dB of the float receiver, and ROBUST/EXTREME decode at $-11/-17$ dB. The surprise: the default 6-bit peak-normalized LLR quantization already acts as a *compressive recalibration* (Section 5), and combined with the frequency-search tracker the fixed receiver **matches or beats the recalibrated float chain at $-9/-10$ dB** (22 vs. 20 and 11 vs. 5 of 30 trials). The calibrated-LLR mode (integer header α fit plus a 32-entry reliability ROM measured on the fixed chain itself) therefore buys a *stable cross-frame LLR scale* — what makes HARQ combining legitimate — rather than extra PER.

The 16-QAM rungs were swept separately (`experiments/fixed_qam16_sweep.py`, 48 packets/point, fixed TX with identical samples into both receivers). The first sweep exposed a ∼2 dB gap on the puncture-weak high rates (+4.6/+6.3 dB versus float +2.6/+4.5 at rates 2/3 and 3/4): 16-QAM max-log LLRs span a much wider dynamic range than BPSK (inner/outer bits times per-carrier gain), and the 6-bit peak-normalized quantization compresses them. The fix is a *per-modulation quantizer*: data blocks with $\mu = 4$ use an 8-bit peak-normalized quantizer (±127), while $\mu \leq 2$ keeps the 6-bit path, whose compression is precisely what helps at the BPSK edge. Peak and mean normalization measured equivalent at 8 bits — the win is the resolution — so the RTL cost is two extra LLR bits on the 16-QAM data path only. Re-swept, the fixed receiver measures +1.5/ + 2.9/ + 5.3 dB for rates 1/2, 2/3, 3/4: within 0.3–0.8 dB of the float receiver everywhere.

Robustness note: the tiled chain survives up to 65 % contiguous audio erasure (the 4× tiles keep partial symbols alive); the HARQ path is validated by a deterministic test with two 75 % complementary erasures whose union covers every sample at least once.

10.5 Gated Two-Stage Frequency Search

The first-symbol residual-CFO search — 275 full-symbol hypotheses at EXTREME, the demodulator's dominant burst in a firmware budget — was restructured as coarse-then-fine: a quarter-length coarse pass (tiles/4 accumulation, whose 4× wider frequency mainlobe tolerates a 4× coarser grid) ranks hypotheses, and only the top-3 ±5-step windows run at full precision, ≈5.5× fewer derotated samples. The coarse metric is ∼6 dB noisier than the full symbol, and a hard coarse decision measurably costs ∼0.5 dB at the EXTREME sensitivity edge — so the fast path is taken only when the coarse top-1/median contrast clears a measured gate (2.25×; edge frames sit ≤ 2.2×, frames with 5 dB of margin ≥ 2.7×), and below it the exhaustive grid runs unchanged. The same gated structure was then back-ported to the *float* demodulator's per-symbol frequency search — where it pays on every symbol, not just the first, because the float chain has no slew-limited tracker. Figure 29 shows the noise-sensitivity A/B (`experiments/coarse_search_ab.py`, 36 packets/point, identical channel realizations per arm), including the ablation that justifies the gate: the quarter-length coarse pass *without* the gate (dotted) shifts the EXTREME waterfall right by 0.5–1 dB (fixed RX at −18 dB: PER 0.31 vs. 0.11), while the gated search (dashed) is *identical* to the exhaustive grid at every measured point, both models — lossless by construction, with the full grid reduced to a low-margin fallback. ROBUST tolerates even the ungated coarse pass at parity: its coarse stage (4 of 16 tiles) retains enough detector SNR at that mode's shallower edge, so the gate specifically protects the deepest mode.

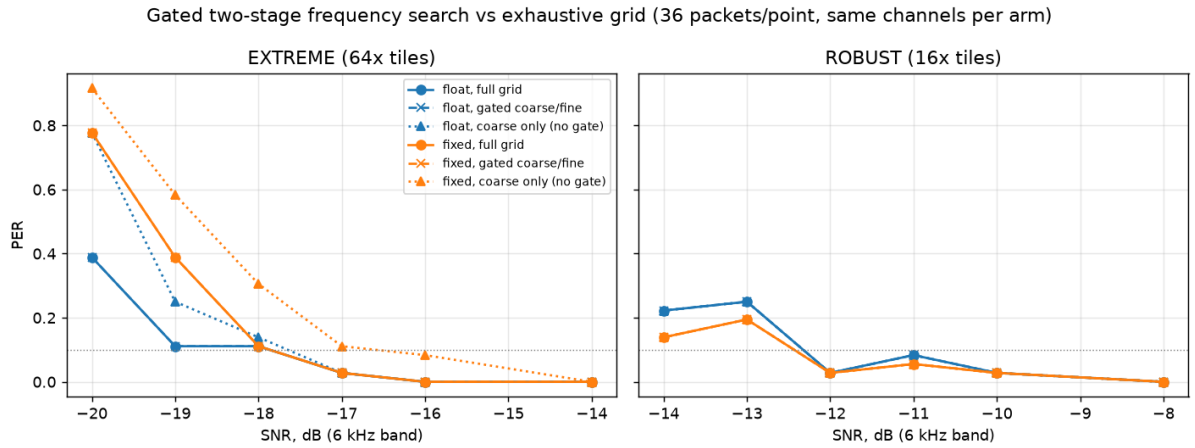


Figure 29: PER versus SNR for the exhaustive frequency grid (solid), the gated two-stage search (dashed), and the ungated coarse-only ablation (dotted); float and fixed-point receivers, same channels per arm. Gated overlays full exactly everywhere; the ablation's edge loss in the EXTREME panel is what the contrast gate prevents.

10.6 Integer SNR Estimator and Full-System Integration

Rate adaptation is driven by a per-frame SNR estimate the fixed receiver originally lacked. The added estimator is data-aided and divider-free: per-column first/second moments of the LLR stream against the known ± 1 bits (the re-encoded header always, plus the decoded data block for $\mu \leq 2$), an integer $\log_2$ via bit-length and a 16-entry mantissa ROM, minus the tiling gain, plus one measured global calibration constant ($-7.2\,\mathrm{dB}$, constant across all modes and modulations). Two design points were forced by measurement. First, grouping moments *per column* removes the multipath $|H|^2$ spread across carriers. Second, symbol rows must be *gain-weighted* by their mean |LLR|: a tiled EXTREME frame spans several QSB fade cycles, and both naive poolings failed — unweighted moments count the fading itself as noise (measured 5–15 dB pessimistic, which parked the rate ladder at rung 0), and equal-weight gain-normalized pooling yields a harmonic-mean-flavored estimate still dominated by fade valleys. Gain weighting reproduces the signal-energy-weighted average the float estimator reports — the statistic the ladder was tuned against — and reduces to the plain estimator when the gain is constant. Final accuracy: $\leq 1.5\,\mathrm{dB}$ error over $-17 \ldots +8\,\mathrm{dB}$, every mode and modulation (slightly pessimistic on the tiled modes, the safe direction).

With the estimator in place, a thin adapter (`FixedPHY`) gives the fixed TX/RX the float `Transceiver`'s link-facing interface — including blind mode auto-detection by trying the three per-mode receivers — and `LinkStation` accepts it via a `phy` parameter. The complete adaptive system then runs on the integer pipeline end to end: `demo_wav.py --phy fixed` records the two-station QSO of Section 11 with both stations *and* the blind monitor decoding on fixed point. At $-2/-1\,\mathrm{dB}$ the session bootstraps at EXTREME, netting and climbing exactly as the float system does, and the monitor blind-decodes 37 of 47 frames; at $+5\,\mathrm{dB}$ the ladder climbs into 16-QAM (rungs 9–10 on the air, requests up to rung 11) and the monitor decodes 26 of 40 (Appendix D).

10.7 Coarse-CFO Ambiguity and the Two-Lag Unwrap

The integer detector carried a defect that cost ≈8 % of *all* acquisitions and stayed hidden for a long time because acknowledged mode simply retransmits what it loses. Broadcast (Section 7.8) has no retransmission, so it surfaced there as whole groups vanishing, and the diagnosis is worth recording in full because the symptom pointed three layers away from the cause.

The observable was a timing outlier: the fixed receiver is sample-accurate on 22 of 24 noisy trials, and on the other two it locks +33 or +34 samples late. Thirty-three is one more than the 32-sample cyclic prefix, so those frames die of inter-symbol interference — and the rate was flat against SNR, which is the signature of a discrete decision going wrong rather than of noise.

The chain of causation runs backwards from there:

1. The coarse mask-shift search is a near-*tie* between adjacent frequency bins. Measured on a failing trial, the correct shift scored 36.7×10^6 and its neighbour 38.0×10^6 — 4 % apart, so noise picks the wrong bin often enough to matter. On its own this is harmless: the residual estimator exists to absorb exactly this.

2. It cannot. The residual is the lag-N phase of the tone field, which wraps beyond $\pm f_s/2N = \pm 46.9\,\text{Hz}$ — and that is precisely *one coarse bin*. With a one-bin coarse error the residual must supply +49.9 Hz, so it wraps to −43.8 Hz and the total lands 93.75 Hz ($= f_s/N$, one subcarrier) away.

3. A frequency error *cyclically shifts* a Zadoff–Chu correlation in time. The 93.75 Hz error moves the ZC peak by ≈34 samples, and the fine stage duly reports it: the ZC stage was faithfully answering for the frequency it had been handed.

So the defect is a **phase-unwrap failure**, not aliasing and not peak selection. The float chain never had it because it resolves the residual with a full-block FFT peak (unambiguous over ±1.5 bins) before refining with the lag-N phase; the integer model had only the lag-N step.

The fix restores the missing disambiguation at integer cost: take the phase at lag $N/2$ as well, which is unambiguous over $\pm f_s/N$ — wide enough to cover a one-bin coarse error — and use it to choose which lag-N cycle is the correct one,

$$\hat{\omega} = \omega_N + \text{round}\left(\frac{\omega_{N/2} - \omega_N}{f_s/N}\right) \cdot \frac{f_s}{N}.$$

One extra correlation, no FFT, no new tables. Applied identically to `ofdm_phy/fixed/rx.py` and `cport/src/rx_detect.c`.

Table 10: Effect of the two-lag unwrap on the integer detector.

	before	after
CFO on the two failing trials (truth +3.0 Hz)	−90.9, −87.0 Hz	+3.8, +5.7 Hz
timing outliers, 24 trials × 3 SNRs	6 at +33/ + 34	**none**
locks within ±16 samples	21–23 of 24	**24 of 24**

The golden vectors were regenerated against the corrected model and the C detector re-verified: that `test_rx` still passes is the meaningful check, since it asserts the two

implementations agree bit-exactly with the new unwrap rather than merely that both work.

End to end, the fix closes a gap that had looked like a sensitivity difference. Broadcast delivery (Section 7.8, 200-byte payload, 12 trials per point) ran 15–20 percentage points behind the float chain before it, because one lost acquisition costs a whole group of four frames; afterwards the integer chain matches float everywhere and is *ahead* at the two lowest points — the same compressive 6-bit quantization advantage reported above, no longer masked by lost acquisitions.

Two coincidences make this defect what it is, and both are visible in Figure 30. The first is numerical: the coarse bin spacing is $f_s/B = 46.875\,\text{Hz}$ and the lag-N wrap boundary is $f_s/2N = 46.875\,\text{Hz}$ — *the same number*, because $B = 2N$. A one-bin slip therefore demands a residual sitting exactly on the wrap, where the phase is at $\pm\pi$ and the decision is a coin toss. That is why the failures cluster at multiples of the bin spacing and vanish in between: 20–25 % of locks ruined at 0 and 46.9 Hz, none at all between 10 and 40 Hz.

The second coincidence is operational, and it is the one that matters. CFO ≈ 0 is not an arbitrary point on that axis — it is where the AFC netting loop of Section 8 drives a linked pair. The worst-affected operating point was the one the system actively steers toward, so a *well-netted* link paid the highest price. In acknowledged mode that price is invisible in the delivery statistics and shows up only as extra transmissions: at zero offset, 1.25 per delivered frame before the fix against 1.00 after, or 25 % of the link's transmissions spent re-sending frames the detector had mistimed.

The left panel measures the cause and the right the effect, and at zero offset they are exact complements — 20 % of locks off by more than half a cyclic prefix, 20 % of frames lost — so every mistimed lock cost precisely one frame. Note also what an A/B at a single frequency would have concluded: at 20 Hz the two curves are identical, and the first throughput measurement taken for this section, which happened to use that offset, reported no improvement whatsoever.

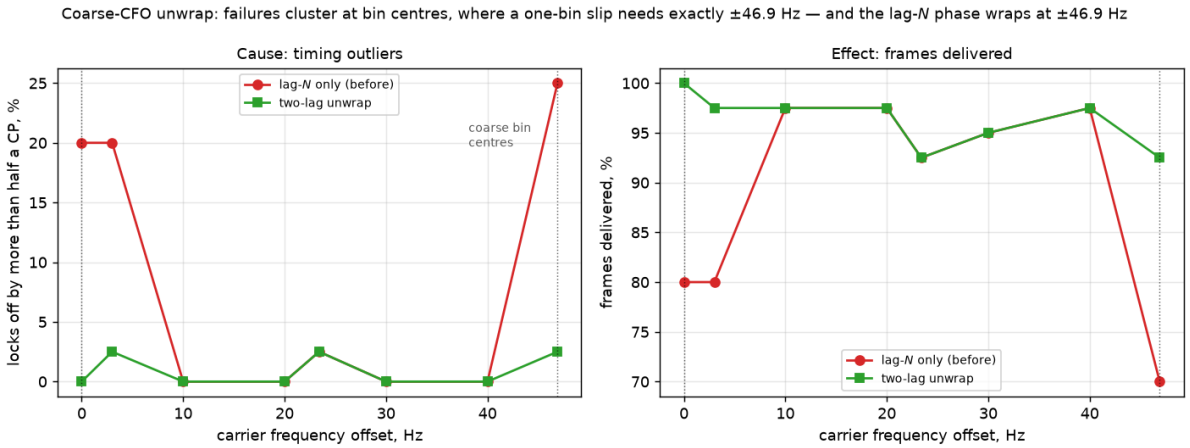


Figure 30: The coarse-CFO unwrap swept across one bin (`experiments/cfo_unwrap.py`, QPSK 1/2 at −3.5 dB, 40 frames per point). Dotted lines mark the coarse bin centres.

Table 11: Broadcast payload recovered, before and after the unwrap fix.

SNR	float	fixed (before)	fixed (after)
−4.5 dB	76.2 %	60.2 %	**87.1 %**
−3.0 dB	98.0 %	78.8 %	**100 %**
−1.5 dB	100 %	86.1 %	**100 %**
0.0 dB	100 %	90.1 %	**100 %**
+3.0 dB	100 %	100 %	100 %

10.8 Where the Fixed Model Differs from the Float Chain

The integer model is not a transliteration; several places diverge deliberately, and a caller that mixes the two must know which.

Table 12: Deliberate float/fixed divergences.

	float	fixed
Detection FFT (NORMAL)	128 bins	256 bins, so the coarse grid is fine enough for an unambiguous lag-N residual
Residual CFO	full-block FFT peak, refined by lag-N phase	lag-$N/2$ phase selects the lag-N cycle (Section 10.7)
CFO representation	Hz throughout	32-bit phase-increment word everywhere; Hz only at the boundaries
Analytic signal	`scipy.signal.hilbert`, zero delay	63-tap FIR with a **31-sample group delay** — detector positions are in analytic index space, not sample space
LLR quantization	float LLRs, ±20 clip	6-bit peak-normalized for $\mu \leq 2$; 8-bit for $\mu = 4$, whose wider dynamic range the 6-bit path compressed
LLR recalibration	measured monotone map	32-entry reliability ROM trained on the fixed chain itself
SNR estimate	decision-directed E_s/N_0	gain-weighted symbol rows, −7.2 dB measured constant; reads low outside the regime it was calibrated in
FFT scaling	unscaled	$1/N$ carried as a per-stage $\gg 1$, with BFP exponents tracked through every energy and LLR comparison

The group delay is the one that bites hardest in new code. Inside `FixedReceiver.receive()`

it cancels — detection and demodulation index the same analytic arrays — but any caller that detects in one space and then slices, bounds or advances in sample space accumulates 31 samples per step. It is the first thing to check when an integer-chain walk drifts.

10.9 Debug Lore

Three integer-domain bugs cost real time and are recorded so an RTL implementation does not repeat them: the ZC metric's Q-scaling must account for *both* 2^{-15} factors the Q15 kernel introduces (a perfect correlation scored 0.006 with one missing); scipy's `remez(..., type="hilbert")` returns taps producing $-\sin$ for a cosine input, so the taps must be negated for a positive-frequency analytic signal; and BFP energies compare only after shifting by $2\,\Delta\text{exp}$.

11 Validation and Results

11.1 Regression Suites

Every layer has a self-verifying suite (Table 13); all pass at the state documented here.

Table 13: Verification suite summary.

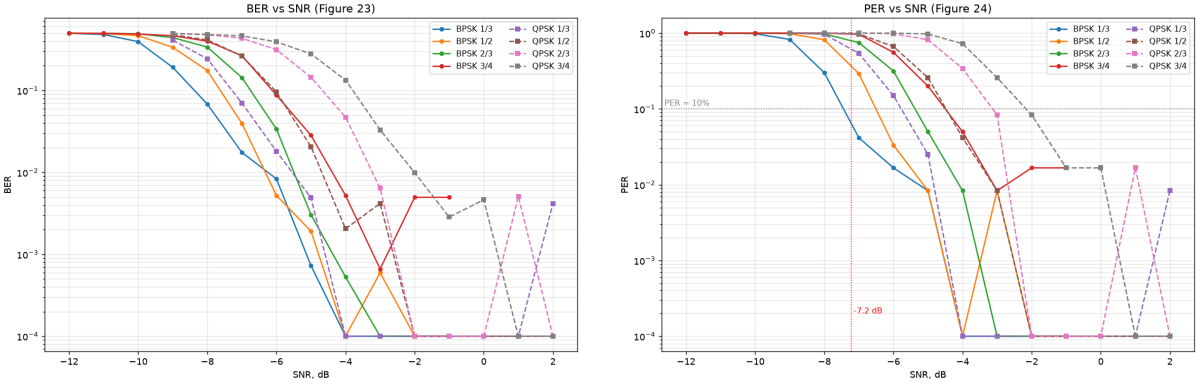
Suite	Checks	Scope
`verify_article.py`	44	bit-exact article worked examples: CRCs, Base38/QTH, conv-code outputs at all rates, scrambler/interleaver sequences, ZC ACF (Appendix A)
`smoke_e2e.py`	10	end-to-end TX→channel→RX incl. CFO, modes, STF, auto-mode detection
`fixed_point.py`	24	fixed-point primitives, TX/RX parity, LDPC and 16-QAM TX and RX, HARQ, calibration, SNR estimator (Appendix B)
`rf_channel.py`	6	SSB transparency, LO-derived CFO to 0.1 Hz, decode through the fading RF chain
`afc_netting.py`	—	netting convergence + trim-budget and anchor guard scenarios (PASS/FAIL)
`simplex_session.py`	—	whole-system two-station test, bit-exact delivery assert (PASS/FAIL)

11.2 Article Reproduction

Table 14: Reproduction versus the article.

Metric	Article	This implementation
Worked examples	—	44/44 bit-exact
Sensitivity, BPSK 1/3	≈ -7.5 dB	-7.2 dB raw RX; -7.6 dB genie-synchronized
BER/PER curves (Figs. 23–24)	simulated	reproduced (Figure 31), both preambles

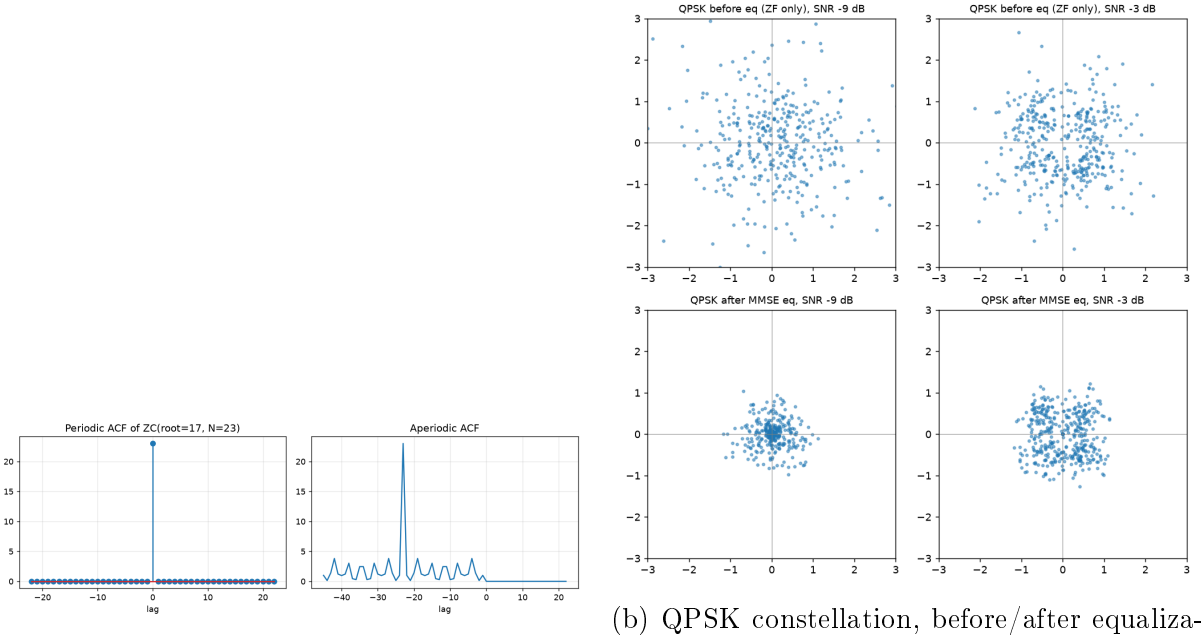
The remaining 0.3 dB between the raw and genie-synchronized receiver is residual synchronization loss at low SNR (occasional missed detections and one-symbol timing slips), not the decode chain — i.e. the reproduction gap is closed to within measurement noise of its diagnosed cause.



(a) BER versus SNR.

(b) PER versus SNR.

Figure 31: Reproduction of the article's Figures 23–24: eight modulation/rate configurations over the article channel model (multipath $[1, 0, 0.4, 0, 0, 0.2]$, BSC 10^{-3}, BEC $2{\cdot}10^{-2}$, random ± 100 Hz CFO with drift, random timing), 120 packets per point.



(a) Zadoff–Chu autocorrelation.

(b) QPSK constellation, before/after equalization.

Figure 32: Article-style diagnostic figures (`experiments/figures.py`).

11.3 Measured Feature Gains

Table 15 consolidates every A/B-measured improvement. All A/B comparisons use fixed seeds (same channel realizations per arm); sensitivity points use 40–120 trials.

Table 15: Feature gains, A/B measured.

Feature	**Measured effect**
LLR recalibration	+1.5–2 dB on BPSK/QPSK rungs (−9 dB: 6/40 → 29/40); regresses 16-QAM → gated to $\mu \leq 2$
HARQ chase combining	2nd attempt 15/20 vs. 10/20 at −8.5 dB (noise-limited regime)
LDPC (IRA 768/256) vs. conv	+0.7 dB AWGN at BLER 10 %; ≈ parity end-to-end (front-end LLR quality binds)
STF preamble variant	equal to Newman everywhere (±0.14 dB over 8 configs, 2σ agreement per point)
AFC netting	+284 Hz → < 12 Hz in 5 exchanges; steady ≤ 11 Hz under 0.09 Hz/s relative drift
Fixed-point RX vs. float	≤ 0.5 dB loss at −7 dB; *ahead* at −9/ − 10 dB (22 vs. 20, 11 vs. 5 of 30, against float+recal); 16-QAM within 0.3–0.8 dB after per-modulation LLR quantization (8-bit for $\mu = 4$)
Integer SNR estimator	≤1.5 dB error, −17 . . . + 8 dB, all modes; gain weighting essential (equal-weight pooling was 5–15 dB pessimistic on faded EXTREME frames)

11.4 The System Demonstration

The recorded demonstration (`results/system_demo.wav`, Figure 33, transcript in Appendix C) is the whole system at once: two stations over the fading SSB RF chain with realistic LO errors, EXTREME-mode negotiation, rate climb, AFC netting visible in the transcript's CFO column, HARQ, and bidirectional message transfer — blind-decoded from the monitor's audio by `demod_frame_auto` with no side information.

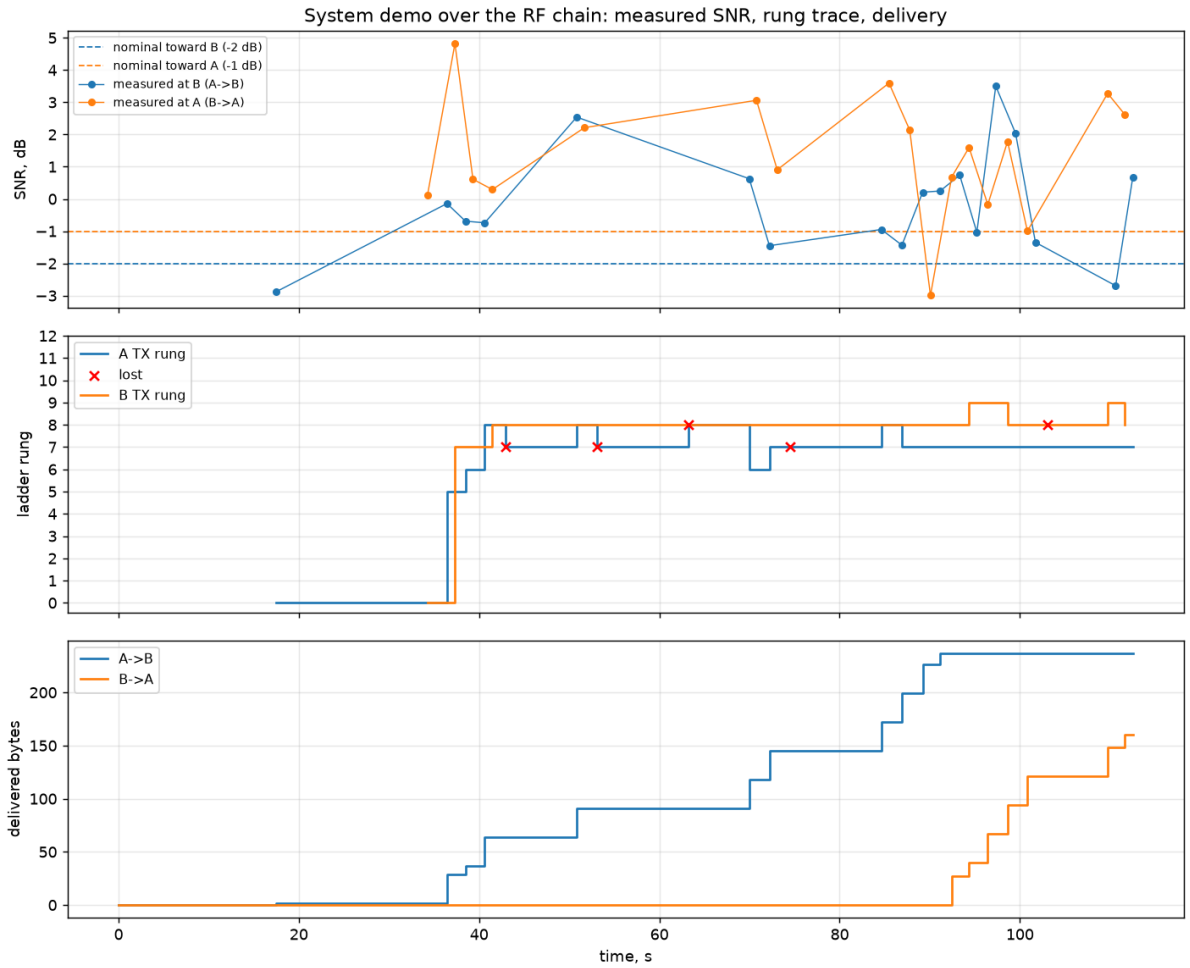


Figure 33: System-demo timeline: per-frame SNR, ladder rungs, and delivery over the session.

The same demonstration also exists on the *fixed-point* pipeline (`--phy fixed`, Section 10): at $-2/-1$ dB (`system_demo_fixed.wav`, 37/47 frames blind-decoded) and at $+5$ dB, where the ladder climbs into 16-QAM (`system_demo_fixed_qam16.wav`, 26/40; Figure 34 and Appendix D).

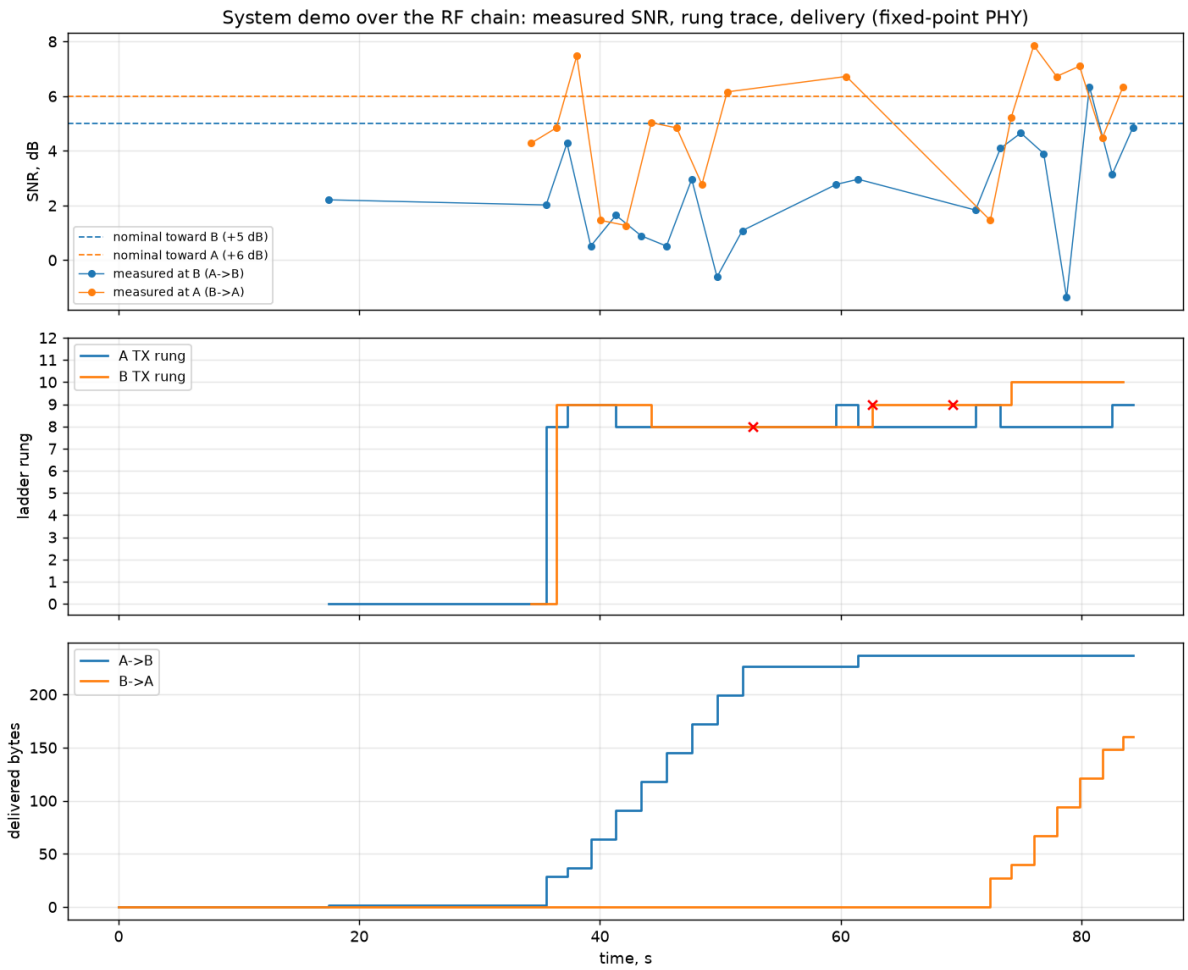


Figure 34: Fixed-point-pipeline demo at +5 dB: the integer SNR estimates drive the ladder into the 16-QAM rungs.

11.5 Link Budget

In radio terms, the EXTREME operating point (−17.5 dB planned, −17.9 dB measured, 6 kHz reference) needs $S/N_0 \approx +20.3$ dB-Hz, i.e. ≈ -13.7 dB in the ham-standard 2.5 kHz bandwidth — about 7 dB less sensitive than FT8 [7] and ∼20 dB better than SSB voice. Against ITU-R P.372 noise levels [12] with 0 dBi antennas: on 80 m at 10 W, 500–2000 km reliably on typical nights and 3000–5000 km from quiet rural sites; on 40 m at 10 W, 300–1500 km all day and worldwide multi-hop on quiet winter nights; 100 W adds roughly one ionospheric hop to everything. A planning caveat: EXTREME's 0.69 s coherent integration assumes $\lesssim$0.1 Hz Doppler spread — on disturbed or auroral paths ($\geq$1 Hz) the coherent gain collapses at any power and ROBUST is the better choice, one more argument for adaptive mode selection.

11.5.1 Measured Against a Reference Source

Everything above is derived from a *relative* sensitivity and an *assumed* noise figure. With the SDR receiver of Section 12.20 on the bench, the assumption can be replaced by a measurement: a tinySA's calibration output, a reference tone at a documented level, was fed into the radio through a 40 dB attenuator and the receive chain characterised end to

end (`demoapp/sdr_rx_calibrate.py`; `results/sdr_rx_calibration.csv`). At 10 MHz with −65 dBm at the antenna port:

Table 16: Receive chain measured against a reference source (HackRF One, LNA 40 dB).

Quantity	**Measured**
Frequency error	−33.1 Hz at 10 MHz = −3.31 ppm, repeatable to 0.0 Hz (−23 Hz at 7 MHz)
Linearity	1.002 dB/dB over 16 dB of gain, 0.03 dB residual
Receiver noise	−95.5 dBm in the 6 kHz reference band (NF ≈ 40 dB)
Absolute sensitivity = measured noise floor + the rung's required SNR:	
rung 0 EXTREME BPSK 1/3 7.8 bit/s	−113.4 dBm
rung 4 NORMAL BPSK 1/3 117.6 bit/s	−103.1 dBm
rung 12 NORMAL 16-QAM 3/4 1059 bit/s	−90.8 dBm

Two results matter for planning. First, the receiver's own noise is *not* what limits this link: against ITU-R P.372 atmospheric noise in the same 6 kHz band, external noise still dominates by +9 dB at a quiet rural site, +17 dB at a typical rural one and +24 dB in residential noise — so the externally-limited assumption behind the ranges above holds, and it holds with about 9 dB of margin at the quietest sites. That margin is precisely where a preselector and a low-noise amplifier begin to pay for themselves, and it is worth noting that the 40 dB noise figure measured here belongs to a wideband, 8-bit, unpreselected SDR — an ordinary HF receiver is far better, so this is close to a worst case.

Second, the frequency error is negligible for this waveform: 3.31 ppm against an acquisition range that tolerates roughly 50 ppm at 7 MHz. A free-running SDR needs no external reference to *acquire* a frame; a disciplined oscillator matters only for EXTREME's 0.69 s coherent integration, where drift *within* a 44 s frame is the constraint rather than absolute accuracy. The absolute levels in Table 16 inherit the calibration output's nominal −25 dBm; the relative results — linearity, frequency error, and the comparison against atmospheric noise — do not.

12 C Implementation and Infrastructure

The fixed-point reference model of Section 10 answers *whether* integer arithmetic preserves the link performance; the C port (`cport/`) answers whether the system *fits* a DSP or microcontroller. It is portable C99 with no `malloc` and no floating point on the signal path (doubles remain on the control plane — dB values and timers — as in the model), and covers the entire feature family: both FEC families, all three modulations and link modes, the gated two-stage frequency search, HARQ combining, calibrated LLRs, the integer SNR estimator, the full link layer, and a streaming receiver architecture for MCU deployment.

12.1 Bit-Exact Porting Methodology

Every module is validated against *golden vectors* generated from the Python fixed-point model (`gen_vectors.py`): inputs and expected outputs are dumped as C headers and

compared with `memcmp` — no tolerances anywhere. Whole transmit frames (up to 260 k samples for EXTREME) are compared through 64-bit FNV-1a hashes over every `int16` sample, with the first 64 samples embedded verbatim for debuggability. Two rules keep the comparison honest:

- **ROMs are dumped, never recomputed.** Twiddles, preamble blocks, pilot symbols, LDPC graph, ladder tables and detection kernels all come from the Python model; a C-side reimplementation of table generation would risk silently diverging in rounding.
- **Width findings are documented, not patched over.** Every intermediate that exceeds a naive width estimate (Viterbi path metrics, BFP energy sums, the ZC metric's Q-scaling) is recorded in `WIDTHS.md` with its measured range, so an RTL implementation inherits the analysis.

The suite currently runs **79 checks** across six binaries (primitives, bit pipeline, TX frames, RX demodulation, streaming receiver, link layer). RX genie vectors are self-hosting: clean frames are rebuilt by the bit-exact C transmitter inside the test, so only the Python detection genie (start sample, CFO word) and expected bits are dumped.

12.2 Streaming Receiver

The frame-at-once reference receiver is impractical on an MCU (it wants the whole capture in memory), so `rx_stream.c` implements a ring-buffer state machine (SEARCH → ZC → HEADER → DATA) fed in arbitrary chunks. The tone-detection stage is necessarily causal and diverges from the model in two documented ways: block exponents and the noise floor are windowed (a streaming receiver cannot know the whole capture's minimum exponent, and the model's whole-capture *median* floor is acausal), and detection commits on a local metric peak — when the above-threshold region ends or the argmax stays stable for a full window span — instead of a global argmax. On the golden corpus the streaming receiver lands on the identical tone anchor for clean frames, making everything downstream bit-identical to the frame-at-once path.

Memory is dominated by sample history, and two measurements shaped the final architecture:

- **The ring only needs to cover the Zadoff–Chu re-anchor lookback, not the preamble.** The tone stage is incremental (each 512-sample block folds into a ≈540-byte summary; raw tone samples are never re-read). A permanent high-water-mark diagnostic (`rxs_ring_hwm`) measured the deepest lookback over the corpus at 8 510 / 32 318 / 124 478 samples per mode.
- **One shared raw ring serves every instance.** All mode-instances listen to the same audio (the preamble root self-labels the mode), so concurrent instances write identical samples and a single `int16` ring of 147 456 samples (288 KB) replaces per-instance analytic rings. The analytic signal is reconstructed on extraction (Hilbert-on-read), bit-identical to the former write-time FIR — and measured *faster*, because the 63-tap filter now runs only on consumed regions rather than on every sample in every instance.

12.3 Protocol Extensions

Three extensions beyond the Python reference address the fixed per-frame overhead (preamble + header ≈0.64 s in NORMAL mode) that dominates air time at the top rungs:

- **Selective-repeat burst ARQ.** Bulk transfers send up to a window of frames back-to-back, answered by one bitmap acknowledgment (NACK-by-omission). The two flag combinations impossible in the legacy protocol serve as burst frame types, so the legacy path is untouched.
- **Extended frames.** The article's header `len` field counts packet *bits* (255 max → 27-byte payloads). An unused header type codepoint reinterprets `len` as payload *bytes*, allowing 255-byte payloads (2 076-bit packets) behind a single preamble. Burst fragment size scales with the engage-time rung (200 B at rung ≥10, 100 B at ≥7, else 25 B).
- **Streamed burst windows.** A whole selective-repeat window is transmitted behind one preamble and header (Sections 2.3 and 7.5) instead of one preamble per fragment. `tx_build_burst` waveforms are bit-exact against the fixed-point model, and both receivers can follow a burst: the frame-at-once path re-runs the recording through `rxd_receive_burst` and re-locks on each Zadoff–Chu block, while the streaming receiver steps to the next block at its deterministic offset via `rxs_continue_burst`.

A streamed burst carries only full-size fragments; the short tail of a transfer always travels as its own frame, because the receiver derives the message length from that final fragment's own length. The acknowledgment request rides on the burst's *first* block as well as its last, so a peer that cannot follow the stream still answers and the sender learns by bitmap rather than by waiting out a timeout.

That receive-side continuation is also where the sharpest bug of this work appeared. While the streaming receiver is stepping through blocks it is not running the preamble detector, so a continuation that does not know where to stop makes the receiver deaf for one block-time per phantom block. An early version re-armed its block counter on every successful block and therefore never learned that a burst had ended: it chased up to seven blocks that were never sent, ≈12 s of deafness landing squarely on the peer's next burst, which timed out and retried indefinitely. The transmitter marks the last block of a burst with the acknowledgment request, which is the stop signal — qualified as "set *and* not the first block of this stream", since the first carries it too.

Measured on the virtual channel: a 250-byte transfer completes in 3 frames instead of ≈20 for stop-and-wait, and a 5 000-byte file drops from 211 transmitted frames (legacy) to 45, holding the top QAM16 rung throughout. Streaming the window compounds this: a 250-byte transfer at 25-byte fragments takes 4 transmissions streamed against 13 after falling back to per-frame bursts, and a 14 kB file over a +20 dB channel takes 76 transmissions streamed against 187 per-frame.

A fourth extension sits above the protocol rather than inside it. Nothing in the chain compressed, while every comparable HF system does — Winlink's B2F, VARA, and PACTOR all compress before transmitting. Files are now DEFLATEd whole and the compressed stream is what gets split into parts; compressing each part separately would discard most of the ratio, since the dictionary never warms up. A distinct envelope

magic marks the compressed form, so a peer that predates it sees an unknown message rather than misparsing a structurally valid one, and the transmitter falls back to sending raw whenever compression fails to shrink the payload. Compression lives at the application layer precisely so the C port stays dependency-free: an embedded build substitutes miniz or heatshrink, with plain DEFLATE on the wire either way. Measured on a 14 kB configuration file: 14 162 bytes to 5 015 on air, 2.82×. The honest qualification is that transmissions fell only 1.4× and wall-clock delivery 1.6×, because acknowledgments and the rate-ladder bootstrap do not compress — the ratio applies to the part of an exchange that is payload, not to the exchange. Supporting endpoints added alongside: a diagnostic event stream (every TX/RX, timeout, rung change annotated with the controller inputs that caused it, burst state transitions) plus a controller-decision snapshot — which immediately located a real bug (burst acknowledgment windows systematically timing out and poisoning the rate controller toward its hard rung-0 fallback; fixed by forgiving a window's first miss and budgeting the reply air time for the actual bitmap at a conservative rung) — and an external oscillator fine-tune endpoint (VCTCXO DAC / CAT clarifier actuator driven by the AFC netting loop, with budget clamping, an anchor role, and a manual override).

12.4 Memory and Flash Engineering

Table 17 shows the measured flash footprint (`arm-none-eabi-gcc -mcpu=cortex-m7 -O2`, const tables count into `.text`). Three reductions brought the total from an initial 242 KB to ≈57 KB with zero runtime cost and bit-exactness enforced by the golden tests:

- **Preambles stored as their unique periodic blocks** (183 KB → 1.3 KB): each mode's preamble is exactly a 32-sample tone block tiled $2T\times4$ times, a 64-sample tone block tiled $T\times2$ times, and one 128-sample ZC tile (whose tail doubles as the cyclic prefix) repeated `sym_tile` times — 224 unique samples per mode, re-expanded by modulo indexing.
- **Single-table trigonometry** (18.2 KB → 8 KB): the NCO sine table and all six FFT twiddle arrays are exact index-arithmetic views of one 4096-entry cosine table ($\sin[i] = \cos[(i + 3N/4) \bmod N]$; twiddles at stride N/n); the generator asserts the identities at dump time.
- **Packed Viterbi traceback** (1 bit per state per step instead of a predecessor byte — the predecessor is $(s\gg1) + (\text{bit}\ll5)$), and `int32` depuncture buffers: extended-frame Viterbi state fell from 181 KB to 42 KB, bit-identically.

Table 17: Measured Cortex-M7 flash footprint (all three modes).

Object	Flash	Dominant tables
`fft.o`	9.4 KB	8 KB shared cosine ROM
`rx_detect.o`	10.5 KB	ZC kernels + detection masks
`rx_stream.o` + `rx_demod.o`	16.9 KB	frequency-search tables
`ldpc.o`	7.2 KB	code graph
`link.o` + `station.o`	7.2 KB	rate-ladder ROM
`tx.o`	3.5 KB	1.3 KB preamble blocks
`dsp.o` + `conv.o` + others	3.0 KB	—
Total	**≈57 KB**	

12.4.1 RAM: Making the Cost Follow the Signal, Not the Capture

The host reference buffers whole captures because a workstation can afford to: the transmitter assembled a complete frame before clipping it, the detector materialised its entire search window, and the demodulator held a whole recording. Linked for Cortex-M7 that came to roughly 10 MB of `.bss`, which is not an optimisation problem so much as a statement that the reference had never been asked the question. Five changes took it to 699 KB, each verified bit-exact against the golden vectors on both x86 and Cortex-M7.

The transmitter generates on demand (4.3 MB → 10 kB). The waveform is nothing but short periodic blocks repeated: tone field A is a 32-sample block, tone B is 64, ZC is 128, and a data symbol is one 128-sample IFFT tile repeated `sym_tile` times behind a CP that is its own tail. So a generator needs one tile, not one frame. The obstacle is that the clip threshold is a *whole*-waveform RMS — the invariant that makes a burst not the concatenation of separately built frames. Rather than buffer for it, the generator runs twice: once to accumulate energy, once to emit. It is a pure function of its inputs, so the second pass reproduces the first exactly.

Detection pulls instead of being handed a window (2.4 MB → 220 kB). The ZC scan sweeps up to 70 688 samples at EXTREME — 5.9 s of audio — but its *live* span is only preamble_len+1: at position p the correlation reads $[p, p+\text{klen})$ and the energy term reads $m = p-(ng-1)\text{klen}$ and $m+\text{preamble\_len}$, which is the same leading edge because $ng \cdot \text{klen} = \text{preamble\_len}$ exactly. Likewise the two lag correlations use lags of 128 and 64 — not a fraction of the segment — so they need a 128-sample delay line however long the segment is. A fetch-by-index source lets the scan read from the raw `int16` ring that already holds the audio, derotating on the way out (the phase at index k is $k \cdot cw \bmod 2^{32}$, which is what makes random access possible).

One arena for scratch that is never co-resident (646 kB → 129 kB). Classifying buffers by *lifetime* rather than by name: the raw ring, the block summaries and the per-symbol LLR accumulators carry state across calls and cannot be shared, but everything else is call-scoped, and the phases are strictly sequential — detection finishes before the first symbol is demodulated, demod before decode. One arena sized by the *largest* phase replaces their sum. The subtlety worth recording is that demod is the largest, not detection: `eval_hyp`'s derotation scratch is live *while* the symbol samples still are, because it runs inside the symbol demodulator. Sizing from the buffer list rather than the actual nesting would have produced silent corruption.

Narrowing the sample and soft-decision types. Measured peaks across the whole suite justify each: analytic samples 43 077 (17 bits → `int32`), LLRs 1 211 062 (21 bits),

correlation magnitudes 780 982 (20 bits). Products of two narrowed values are widened explicitly — on Cortex-M7 that is `SMULL`/`SMLAL`, the same instruction count as `MUL`/`MLA`, so exactness is free.

Per-instance tone summaries (204 kB → 94 kB). Every receiver instance had been given EXTREME's 128-block ring, but an instance only ever scans its own mode's window: 15/30/120 blocks.

Table 18: Measured Cortex-M7 RAM (`-Os`, `-gc-sections`, linked image referencing only the streaming APIs).

Component	RAM	Sized by
Shared raw `int16` ring	160 KB	EXTREME re-anchor lookback (67 134)
Scratch arena (RX 3 phases + TX)	128.5 KB	EXTREME ZC geometry
Tone summaries, 3 instances	94 KB	each instance's own mode
LLR buffers (`int32`)	24 KB	192 KB with EXT frames
Message store (pooled)	3 KB	12 slots, peak 6 measured
Everything else	152 KB	—
Total, three modes	**561 KB**	793 KB with EXT frames
Transmit-only image	27 KB	

Target fit, corrected. An earlier estimate put the three-mode station at ≈453 KB and concluded it fitted the STM32H723xG (≈496 KB usable data RAM) with margin. The linked image does not: 699 KB without extended frames, 929 KB with. The estimate had costed per-mode summaries while the code gave every instance the EXTREME count, and assumed a build without EXT frames. Both were addressed; three further reductions followed, described below and in Section 12.5, bringing the figure to 561 KB. The conclusion is unchanged in kind — an H743-class device, not the H723xG — but the margin is now comfortable rather than absent.

Nor is a single-mode build the way out, though the arithmetic is tempting. Rung 0 of the ladder is EXTREME, and rung 0 is where both stations bootstrap, where the controller drops after four consecutive losses, and where staleness decay lands; rungs 1–3 are ROBUST. A build without EXTREME cannot acquire a link at all, nor recover one after a fade — which is exactly when 64× tiling earns its cost. Per-mode sizing is a capability decision, not a memory knob.

Three later reductions are worth separating because each has a different character. **The transmitter joined the arena** (26 992 B): a station is half duplex, so the streaming generator's state — the one transmit buffer live across calls — can never overlap a receive phase in time, and the arena did not have to grow to hold it. The assumption is checked rather than trusted: receive entry points stamp an owner tag and `txs_pull` refuses to continue a generator whose state a receive phase overwrote. **Message storage was pooled** (161 280 B at the demonstrator's settings): `station_t` had given each of the 42 positions that *could* hold a message its own `ST_MSG_MAX` bytes — 24 queue slots, 16 delivered-log entries, two current messages — which sizes RAM by the number of addressable positions rather than by how many messages can exist at once. The positions now hold a slot index into one store of 12; measured peak occupancy is 6, on a 14 KB file transfer. **And the raw ring nearly halved** (128 KB), for reasons that are a signal-processing story rather than a bookkeeping one, told in Section 12.5.

What the exercise changed is less the total than its *shape*. Nothing in Table 18 now

scales with frame length, capture length, or search-window length; every remaining entry is sized by a property of the waveform. The arena is visibly balanced — its receive phases sit within 6 kB of each other — which is the signal that the union has no slack left: it costs the maximum, so no single phase can be shrunk profitably alone.

12.5 On-Target Measurement, and What It Corrected

Every processor figure quoted so far in this report was a *projection*: analytic multiply–accumulate counts scaled against an assumed sustained throughput of "SMLAD-class, 2× overhead margin: Cortex-M7 at 480 MHz ≈ 400 MMAC/s". That assumption was the weakest line in the document, and it was wrong by a factor of 2.7.

Getting onto the silicon. The measurements below were taken on an STM32H743 over JTAG, with an ESP32 development board as the debug probe: it bit-bangs OpenOCD's `remote_bitbang` protocol, one ASCII character per pin edge, bridged from a serial port to a socket. This is slow — 42 236 edges per second, so a 52 KB image takes 84 seconds to download — but the probe is the part of the loop that does not need to be fast, and OpenOCD keeps its own target support. The benchmark runs entirely from RAM: OpenOCD loads it, points the core at the entry symbol, and reads a result structure back out of DTCM afterwards. The target's flash is never written.

Reading the part first settled the memory map, which the datasheet (DS12110 Table 7) gives only for the H742 and defers otherwise: `DBGMCU_IDC` = `0x20036450` (revision V), 2048 KB of flash, 512 KB of AXI-SRAM with `0x24080000` faulting, and — the figure that mattered — SRAM1/2/3 confirmed *contiguous* across 288 KB, which is what permits a single `g_raw` to live there. A trap worth recording: D2 and D3 SRAM are clock-gated and reset to *off*, so every read of `0x30000000` fails in a way that reads as absent memory rather than a disabled clock.

Primitive costs. Table 19 gives the measured cycle counts. The derived MAC throughput is 147 MMAC/s at 480 MHz, against the assumed 400 — so every load projection built on that figure scales up by 2.7×. They still fit: the continuous front end moves from under 1 % of a 480 MHz core to 1.7 %, and EXTREME acquisition from a projected 10 % to a measured 27 %.

Table 19: DWT cycle counts, STM32H743, code in ITCM.

Primitive	Cycles	Per unit
`fft_bfp`, 128-point	91 376	714 / bin
`hilbert_analytic`	3 970 631 / 4096	969 / sample
`nco_derotate`	159 763 / 4096	39 / sample
`cordic_atan2`	365	—
Viterbi, 255 bits rate 1/3	2 465 972	9 670 / bit
LDPC min-sum, 128 bits, 60 iters	21 885 598	—
Complex MAC (correlation inner loop)	—	3.27 / MAC

A memory-attribute trap that looked like an algorithm problem. The same correlation, over byte-identical data, measured 3.27 cycles per MAC from DTCM and D2

SRAM but 7.52 from AXI-SRAM — and disabling the data cache changed the AXI figure not at all. The cause was not the part: the firmware already resident on the board left an MPU region over AXI-SRAM with `TEX=0 C=1 B=0 S=1`, that is, *shareable*. A Cortex-M7 has no cache-coherency unit, so a shareable Normal region is effectively uncached. With that MPU disabled all three memories measure identically. STM32H7 projects mark AXI-SRAM shareable routinely to sidestep DMA coherency, so this is a live hazard: a misconfigured MPU costs 2.3× on the acquisition path while presenting as slow code.

The same experiment killed a proposal. Splitting the scratch arena so that the detection phase lived in DTCM had been designed and costed at +126 KB before it was measured; the measurement showed it would buy exactly nothing, because the sliding correlation re-reads 512 samples per offset and advances by one, so its reuse is ≈99.8 % and the 16 kB data cache absorbs it from any backing SRAM. It was not implemented.

12.5.1 The ZC scan, and where its cost actually went

Acquisition dominates the receiver's processor budget, and within acquisition the Zadoff–Chu correlation dominates everything else. It is worth setting out exactly what the scan does, because the eventual 4× saving came from *where* it looked rather than from how it computed.

A frame begins with a tone comb of $3T{\cdot}N$ samples (T tone tiles, $N = 128$ bins; 61 440 samples at EXTREME), followed by one ZC symbol of CP + $z_L{\cdot}N$ samples. Detection runs in two stages. The tone stage folds each incoming 512-sample block into a ≈540-byte summary — a band energy and, for each candidate frequency shift, the dot product of the block's spectrum against two masks — and tracks the arg-max of that metric. It never re-reads a raw sample. The ZC stage then correlates the received signal against the known root sequence at every candidate time offset, taking the peak as the frame boundary.

The critical detail is *when* the tone stage can commit. Partial overlap crosses the detection threshold roughly a full window before the true peak, so a decline-from-best rule fires too early; the detector instead takes the arg-max over the whole above-threshold region and commits when that region ends. The region ends about one window after the peak, and the peak is itself one window after the tone field starts. So at the moment the detector announces a candidate start c, the write head is already *two tone fields* beyond it.

That is a latency, not a cost — but two consumers turned it into both. Both anchored at c:

- the lag-N residual carrier estimate, which correlated the tone segment $[c,\, c + 2TN)$ — 40 960 samples at EXTREME, all of them re-read;
- the ZC scan, which searched forward from c across the entire tone field, 62 497 candidate offsets, to find a symbol that lives at the field's *end*.

Both were unnecessary, and each yielded to a different observation.

For the carrier estimate: derotating a signal by a constant per-sample phase θ multiplies every lag-L product by the same constant $e^{-j\theta L}$, because $x'[k]\,\overline{x'[k-L]} = x[k]\,\overline{x[k-L]}\,e^{-j\theta L}$. The correlation's *angle* therefore simply shifts by θL. So the correlation can be accumulated on *raw* samples, incrementally, one block at a time as the tone stage already walks past them, and the coarse frequency word removed at commit as a single angle subtraction. Nothing is re-read. Checked against

the original path on a real EXTREME preamble across ±120 Hz of coarse word, the worst disagreement was 0.0018 Hz against a 93.75 Hz coarse bin.

For the scan: the candidate c is accurate to a fraction of a block. Measured offsets between c and the true frame start were +37, −219 and −220 samples for the three modes — against a search that swept 62 497 offsets. The ZC symbol is not somewhere in that span; it is at a *known* place, the tone field's end, give or take however wrong c is. So the window is anchored there instead:

$$\text{anchor} = 3TN - mB \tag{19}$$

$$\text{window} = (\text{CP} + z_L N) + 2mB \tag{20}$$

where B is the detection block length and m is `ZC_ANCHOR_MARGIN_BLK`.

What the margin is, and why it is a tuned constant. m is the slack allowed on either side of the expected ZC position, counted in *detection blocks* rather than samples so that it tracks B, which is what the candidate's quantisation error tracks: c is a block boundary, so it is wrong by at most a block from that alone, plus whatever the tone arg-max contributes. Measured total error is well under one block in every mode. The shipped value is $m = 8$.

Eight is not a large safety factor in the obvious sense — it is roughly 8× the observed error — but the constant is bounded on *both* sides, and the bounds are mode-dependent, which is what makes it delicate. Table 20 shows why.

Table 20: The anchoring at $m = 8$ blocks. The reduction is worth far more at EXTREME than at NORMAL, and NORMAL is also the mode that constrains m from above.

Mode	Tone field	mB	Offsets, was	now	Ratio
NORMAL	3 840	2 048	4 641	4 129	1.1×
ROBUST	15 360	4 096	16 417	8 225	2.0×
EXTREME	61 440	4 096	62 497	8 225	7.6×

Below the lower bound the window stops covering the ZC reliably and acquisitions are simply lost: $m = 4$ fails the streaming suite. Above the upper bound something less obvious happens. The margin is subtracted from the tone field, and NORMAL's tone field is only 3 840 samples — so at $m = 16$, $mB = 4\,096$ exceeds it, the anchor clamps to zero, and the window reverts to searching forward from c while also being wide enough to reach past the preamble into the data symbols. Data carries no ZC but does carry structure, and a correlator given more places to find a peak will eventually find a spurious one. $m = 16$ fails the suite too.

So the valid range is bounded below by the candidate's accuracy and above by NORMAL's tone field, and the two bounds are only about a factor of two apart. That is narrow enough that m should be treated as measured rather than chosen: the sweep below is the instrument, and it should be re-run if the value is ever moved.

Table 20 also explains where the saving actually comes from. The reduction is worth 7.6× at EXTREME and essentially nothing at NORMAL — which is exactly the right shape, because EXTREME is the mode whose acquisition dominates the processor budget, and NORMAL's was never the problem.

What this bought, and what it did not. Table 21 gives the whole-frame measurement, taken by piping the streaming transmitter directly into the streaming receiver on the target so that no buffer ever holds a frame.

Table 21: One EXTREME frame (43.5 s of audio) on an STM32H743, before and after the two re-anchorings. Decoded CRC-clean in both cases.

Phase	Before	After	% of a 480 MHz core
Tone search	94.6 Mcyc	103.6 Mcyc	3.7
Acquisition	**4 050.4**	**1 018.0**	**41.8**
Demodulation	809.0	856.8	5.5
Whole frame	**4 954.0**	**1 978.5**	**9.5**

Two consequences, and one clarification that matters.

The processor cost of a frame fell from 23.7 % of a 480 MHz core to 9.5 %, and acquisition specifically by 4×. The tone stage pays 10 % more — 9 Mcycles, for the per-block correlation it now maintains — to save 3 032. More importantly the burst now runs in *real time*: before, acquisition needed 8.44 s of processor inside a 5.08 s window and fell behind, catching up afterwards out of the ring; now it needs 2.12 s and keeps up with 2.4× headroom.

The ring shrank with it, and for two reasons at once. Its depth is set by how far back the receiver reaches, which was two tone fields and is now one plus the search margin: 124 478 → 67 134 samples at EXTREME, measured identically on host and target. And because the processing lag is gone, that reach-back no longer has a backlog added to it — previously the two summed to 164 810 against a 147 456-sample ring, an overrun; now 67 134 sits inside 81 920 with 14 786 spare. 288 KB of RAM became 160 KB.

The clarification: this is not a decoding improvement. It is tempting to read a 4× acquisition saving as better reception, and it is not. The frames that decoded before still decode, bit-identically; the frames that failed before still fail. Nothing in the demodulator, the equaliser, the LLR calibration or the decoders was touched, and no sensitivity figure in this report moves. What changed is the *cost* of finding a frame — processor cycles and the RAM to buffer the search — not the probability of finding one. The honest statement of the benefit is that the same reception now fits on a smaller, cheaper part with headroom to spare, which is a deployment result rather than a link-budget one.

There is a plausible argument that narrowing the search should also *reduce* false locks, since a spurious correlation peak now has 8 225 places to hide instead of 62 497 — but that is a hypothesis, not a measurement, and it is not claimed here.

Does the narrower search cost sensitivity? Cutting a search from 62 497 candidate offsets to 8 225 is exactly the kind of change that trades sensitivity for cost without announcing it, and the existing C suite could not answer the question — its only noisy case is NORMAL at −5 dB against stored samples. The A/B was therefore built: both arms compile from one source (a switch restores the original anchor and window) and run over *byte-identical* EXTREME waveforms — same seed, same Gaussian noise, same carrier offset — so the comparison is paired rather than two independent samples of a noisy quantity.

Over a wide sweep at 60 trials per point the two arms decoded 291 frames out of 480 each: identical. At the knee, where the curve is steepest and any loss would show, Table 22 gives 250 trials per point.

Table 22: Paired A/B of the ZC anchoring at the EXTREME sensitivity edge, 250 trials per point, identical waveforms.

SNR	**Original**	**Anchored**	**Δ**
−11.5 dB	223 (89 %)	222 (89 %)	−1
−12.0 dB	195 (78 %)	193 (77 %)	−2
−12.5 dB	138 (55 %)	133 (53 %)	−5
−13.0 dB	58 (23 %)	61 (24 %)	+3
Total	**614/1000**	**609/1000**	−5

Half a percentage point, with deltas of mixed sign. The curve falls ≈43 points per dB through that region, which bounds any sensitivity difference at **≈0.01 dB** — two orders of magnitude below the ≈0.5 dB effects this report treats as real, such as the gated coarse search or the streaming resync. And **noise-only false alarms were zero in 250 runs for both arms**, against the 0/20 the original detection thresholds were tuned to (Section 3).

The SNR reference here is the full-band RMS of the transmitted waveform, which reads roughly 9 dB pessimistic against the convention used elsewhere in this report; it is irrelevant to a paired comparison, where what is measured is two arms on the same input rather than an absolute sensitivity.

One qualification survives. The margin is a *tuned* constant: eight blocks passes every streaming test, four fails, and sixteen fails as well — at sixteen the anchor clamps to zero for NORMAL and the window re-admits data symbols, which is a false-lock risk. The value is not arbitrary and should not be moved without re-running the sweep.

12.6 The Modem as a USB Device

A modem that reaches the PC over a USB-to-serial bridge appears as one more `/dev/ttyACM*` among however many adapters are attached, numbered by enumeration order — so the host has to guess which port is the modem, and guesses wrong after a reboot or a re-plug. The C port therefore presents the STM32's own USB peripheral as a device with an *identity*: a vendor-specific interface (class `0xFF`, so no operating system binds a driver of its own), two bulk endpoints, a fixed VID:PID, and a serial string that is the part's 96-bit unique ID. Two modems on one host are then always distinguishable, and a udev rule gives each unit a stable `/dev/ofdm-modem-<serial>`. The wire format is length-prefixed frames behind two sync bytes and carries no checksum, because USB already CRCs and retries every packet; the parser's resynchronisation count is therefore a fault counter rather than a statistic, and it read zero throughout everything below.

The protocol codec is deliberately free of any USB, station or platform dependency — bytes in, frames out — so one implementation serves the firmware, the host driver, and an *emulator* that runs the real device-side C over a pipe. The host driver was tested against that emulator (11 checks, plus 11 on the codec itself, and 40 MB of random input under the sanitizers) before any hardware was involved, which is what made the hardware phase a search for hardware problems only.

Bring-up. TinyUSB's dwc2 port drives the H743's `USB2_OTG_FS` core; the image runs from RAM over the JTAG probe of Section 12.5, with a progress beacon at a fixed DTCM address, so the board's flash is never written. Figure 35 shows enumeration. Three things cost real time, and all three present as something other than what they are:

- **The USB supply has two sources needing opposite settings.** A board feeding `VDD33USB` from its own 3.3 V rail wants `USB33DEN` alone; one deriving it from VBUS also wants `USBREGEN`. Enabling the regulator on the first kind holds `USB33RDY` low forever: `PWR_CR3` read `0x03000042`, then `0x05000042` the instant `USBREGEN` was cleared. The firmware now tries the external supply first and falls back, recording which worked. It also no longer spins on the flag: an unbounded wait there hung before anything could report why, and looked exactly like a crash.
- **A USB-C-to-C cable does not work on most development boards.** They omit the CC pull-down resistors that mark the board as a device, so a C-to-C host never detects an attach and never sources VBUS — every register on the device reads correct and nothing appears on the bus. A C-to-A cable needs no CC negotiation.
- **A RAM-resident image has no vector table.** With VTOR still pointing at the resident firmware, the first USB interrupt — and any fault — vectored into another program's handler and vanished. The image now installs its own table; within a minute it caught an unhandled SysTick nobody had switched off (`ICSR` `0x80F`, `CFSR` and `HFSR` zero). Genuine faults stop and report; unexpected interrupts are masked, counted and survived.

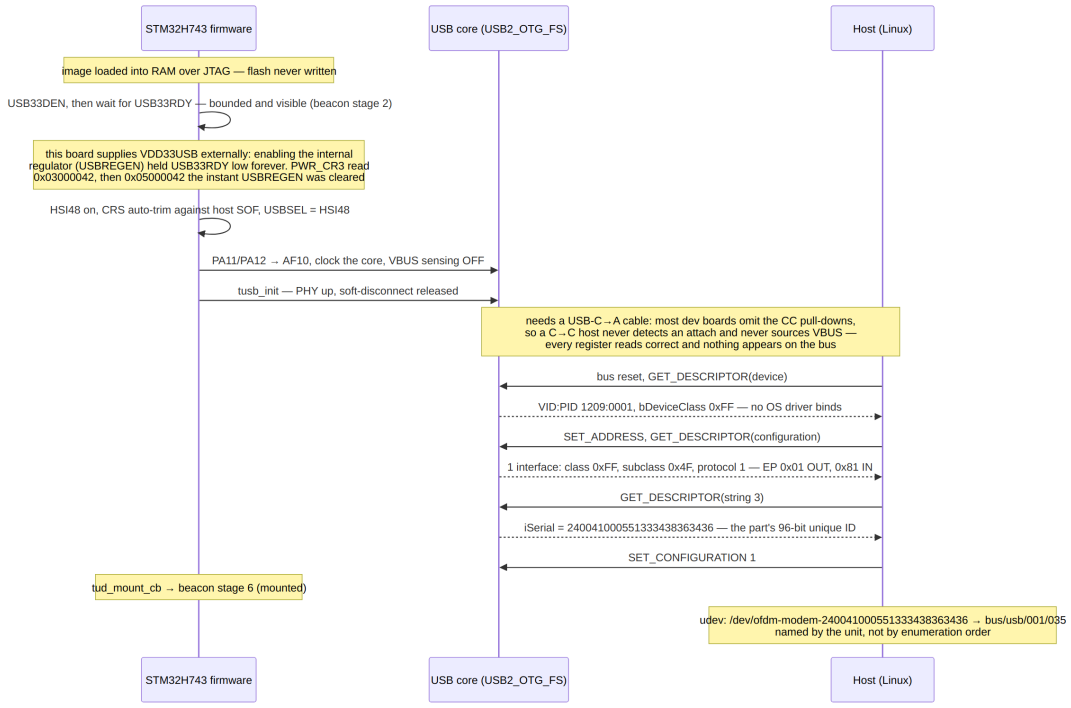


Figure 35: Enumeration: supply detection, clocks from HSI48 (trimmed against the host's SOF, so the firmware is correct whatever the board's crystal), and the descriptors that give the device its identity.

A session. Figure 36 traces a host session against the real link layer: the station's queues, rate ladder and reply timers run for real behind the endpoints, with a stub PHY that accounts air time and transmits nothing — the honest shape of a modem with its antenna disconnected. Status is pushed unprompted twice a second whenever the device is mounted; the diagnostic event stream is opt-in and yields to everything else, because a lost diagnostic costs nothing and a lost reply costs the session.

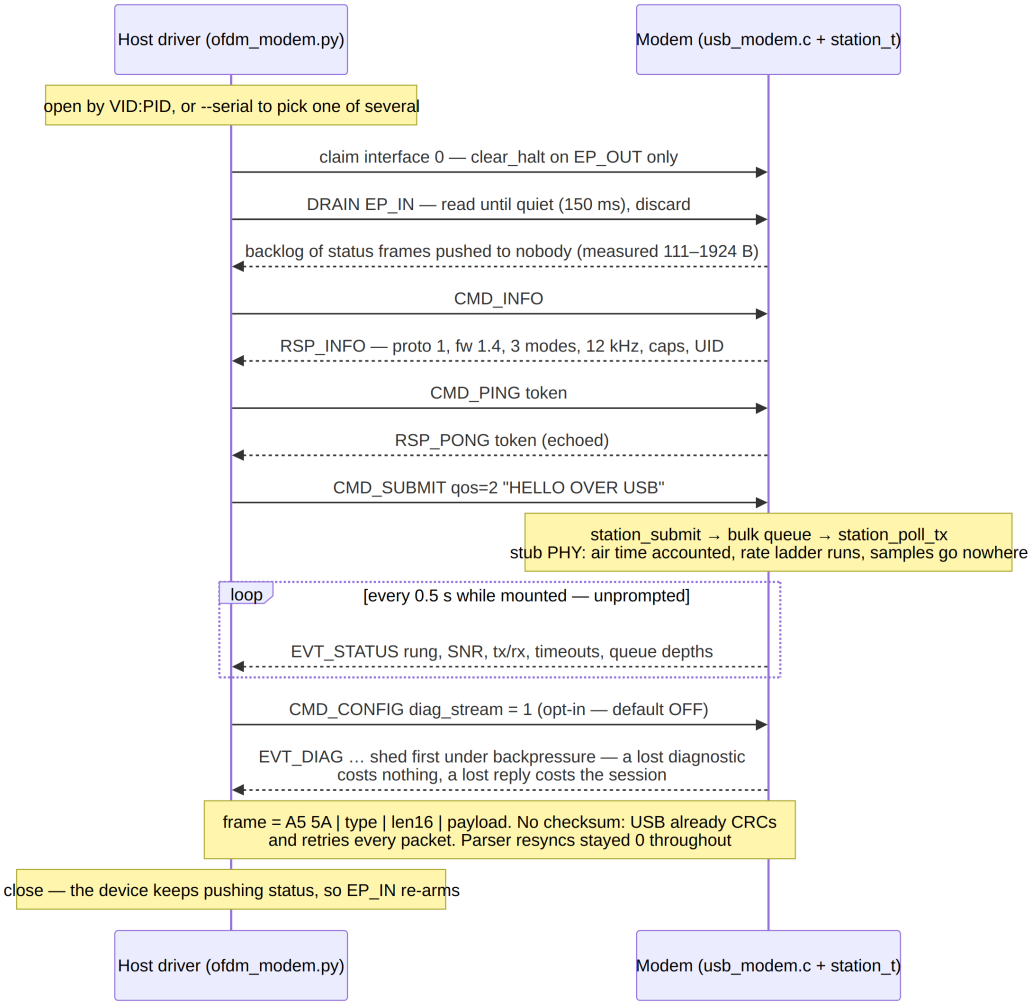


Figure 36: A host session: open, drain, identify, command and reply, unprompted status, opt-in diagnostics.

The wedge, and why the host must drain rather than reset. The station image enumerated correctly and then, for most of a day, answered nothing and fell silent after a single packet — deterministically, across four builds. Five hypotheses were disproven by measurement (a diagnostic firehose, loop starvation under polling, cache against DMA, polling against interrupts, the clock), and three unrelated bugs fixed on the way, before the instrumentation at the moment of the stall read: staging queue empty, 150 bytes free of a 512-byte TinyUSB transmit FIFO, 399 bytes handed over. That is 362 bytes sitting *inside* TinyUSB unsent — 29 (the `RSP_INFO` reply) plus nine 37-byte status frames, exactly — and $399 - 362 = 37$: precisely one frame ever reached the wire. The device was not silent; it was mute.

The cause, confirmed from TinyUSB's source and shown in Figure 37, is an interaction between an ordinary host habit and the library's clear-stall handling. Because the device pushes status unprompted, its IN endpoint is normally *armed* with a frame when a host opens it. The host driver called `clear_halt` on that endpoint on open — a common recovery idiom, and one added during this work. `usbd_edpt_clear_stall` drops its software BUSY flag unconditionally (its own comment calls this "long-standing behavior"), while the dwc2 `dcd_edpt_clear_stall` only clears the STALL bit and resets the PID; nothing disarms the hardware. The next flush sees "not busy" and re-arms the endpoint on top of a live transfer — undefined on the core — so one packet escapes, the completion no longer matches what the library started, and BUSY never clears. The level-triggered FIFO-empty interrupt then storms: `isr_count` reached 90 million.

It was confirmed by prediction rather than by fix. Holding status frames until the host had spoken, so the endpoint was idle at open, made the *first* open work — and every later open still wedged, exactly as the mechanism requires, since by then the device was pushing again.

The fix is on the host: consume the armed transfer instead of resetting the endpoint. The driver reads the IN endpoint with short timeouts until it goes quiet and discards what it finds; `clear_halt` is kept for the OUT endpoint only, where three opens against an armed endpoint measured harmless. The device is unchanged — pushing status while mounted is the right thing for a modem to do, and is the case a driver must handle. Verified on the part: three consecutive opens (draining 1924, 185 and 111 stale bytes) and a session killed mid-read then reopened (draining 37 bytes: the one frame armed at the kill), all answering, submitting and reading with zero resynchronisations; `isr_count` 293 against 90 million wedged, no faults, no drops.

Two qualifications. VID:PID `1209:0001` is the pid.codes *test* identifier, correct for development and wrong for anything shipped: a released unit needs an allocated product ID, or it collides with every other prototype using the same one. And the device has not been made robust to `clear_halt` on an armed endpoint itself; that is TinyUSB's clear-stall path rather than this project's, so the contract is stated instead — any client for this device must drain, not reset.

12.6.1 The host side, and the 2.3-second stall

The drain-not-reset rule above is one instance of a wider mistake, and the rest of it surfaced only once the boards were doing real work. A host naturally assumes the device is always ready to talk. This one is not: a single `rxs_push` can occupy the processor for 2283 ms at the end-of-frame commit (Section 12.8, the same measurement that sizes the capture FIFO), and for that whole window USB is simply not serviced. Every host-side defect found after the two-board stand came up is a deadline shorter than that number.

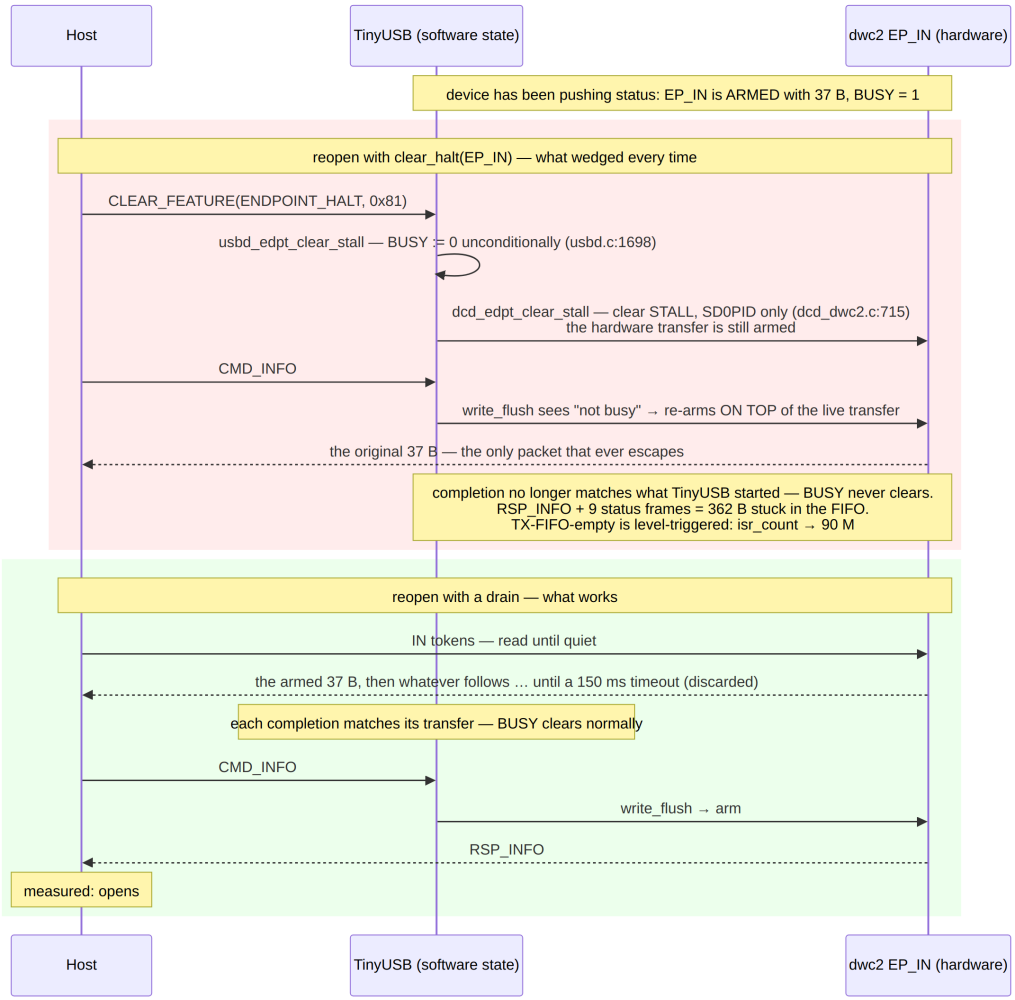


Figure 37: Reopening against a device that is already pushing. Above: `clear_halt` on an armed IN endpoint desynchronises TinyUSB's software state from the hardware and wedges it after one packet. Below: draining the endpoint consumes the armed transfer through the normal path.

Table 23: Host timeouts against the device's worst blocking decode. Each was chosen before that figure was measured, and each failed in a way that looked like something else.

Operation	Was	Now	Symptom
Bulk write, Python	1000 ms	5000 ms	console died mid-transfer with a traceback out of its 1 Hz heartbeat
Bulk write, C	2000 ms	5000 ms	"usb write failed", frame lost
Serial string, C	4×50 ms	10×300 ms	intermittent `<unreadable>` in `-list`
Serial string, Python	none	10×300 ms	`ValueError: no langid`, 3 opens in 12

The last of those is worth more than its size, because the obvious fix did not work. Adding retries left the *first* open in twelve still failing, and failing in 3.7 s — which is 3.7 s of not touching the bus, because pyusb caches a failed language-ID fetch as an empty tuple and every subsequent string request then raises immediately, forever, on that device object. A retry that does not clear the cache is decoration. With the cache reset between attempts: 60 opens of both boards, no failures, and a cold open after fifteen idle minutes — the reported case — takes 0.17 s. The distinguishing evidence was the shape of the failure rather than its message: the *first* open failed and the next eleven succeeded in 0.16 s each, which is a cache being poisoned, not a bus being busy. The error text says "permission issue", and it is not one.

Two further host defects were not about timing at all, and both are the same shape as the sentinel family of Section 12.12: a value that had drifted from its definition. The Python host's maximum frame size was still 1024 bytes, the protocol's original, while `UP_MAX_PAYLOAD` had grown to 3336 for whole-message transfers — so its parser discarded every frame declaring more, and the Python console received small traffic perfectly while file parts vanished, from a board that had decoded and acknowledged them. And `pyusb` was never in `requirements.txt`, only in a README line, so the environment the project is normally run from could not talk to a board at all. Both are now pinned by tests (`host/test_modem.py`), which is the only thing that keeps a constant duplicated across two languages honest.

12.7 Two Boards on a Wire

Everything to this point validated the chain against something the project itself controls: golden vectors, a simulated channel, a virtual audio daemon, and an SDR whose transmit and receive halves share one oscillator. One impairment was never exercised by any of them — two *independent* clocks. A single-board loopback cannot supply it either: playing to `DAC1_OUT1` and recording `ADC1` on the same board runs both converters off one TIM6, so the sample-rate offset is exactly zero by construction, and the receiver's tolerance to it is untested no matter how much copper the signal crosses.

The stand in Figure 38 removes that last shared assumption. Board A's DAC drives board B's ADC through one signal wire and one ground wire; each board clocks its converter from its own crystal and its own PLL, and the two never exchange a timing signal — not even to start. One ESP32 debugs both boards at once: JTAG daisy-chains, so `TDI`/`TDO` thread through board A into board B and back to the probe, which costs

exactly one wire beyond a second board's worth of the existing harness. The three push-pull control lines are duplicated onto a driver pin per board rather than spliced; because a line and its copy sit in the same GPIO bank, the firmware switches both in one `w1ts`/`w1tc` store, so there is no skew between the boards at all. That is a wiring convenience rather than a fix: at the probe's measured 42 236 edges/s a TCK half-period is 25 µs, against nanoseconds of everything else.

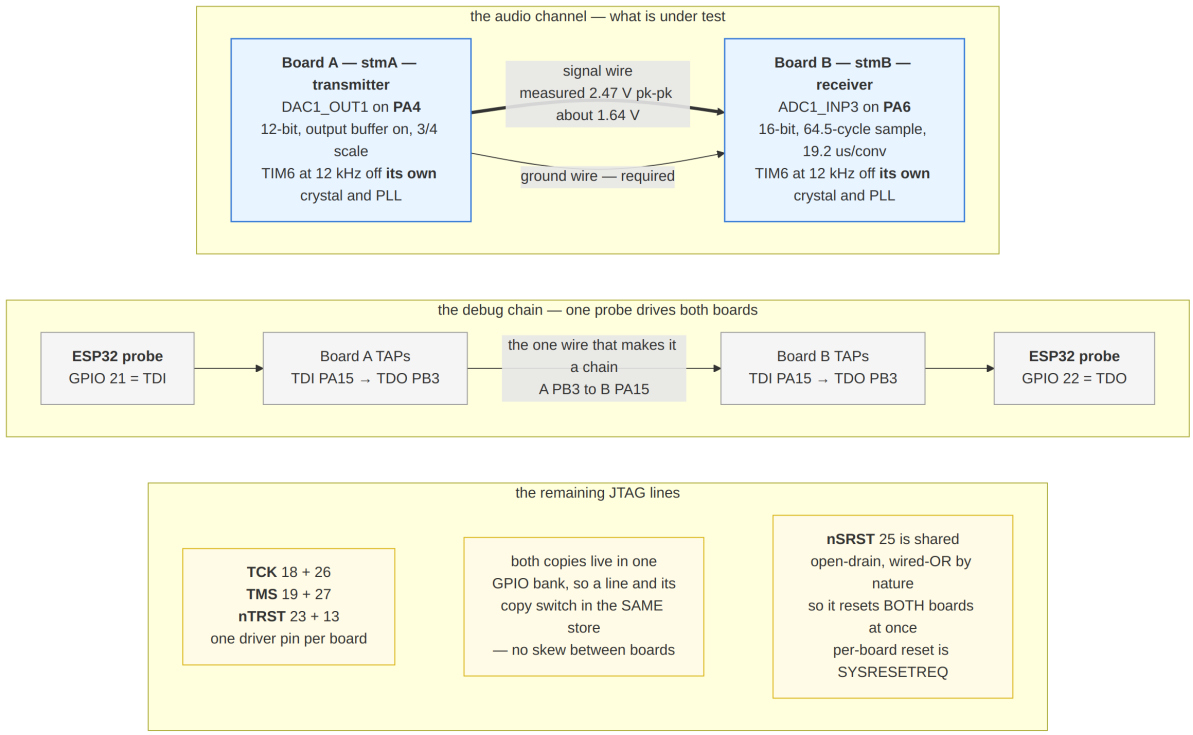


Figure 38: The two-board stand. The audio channel is one signal wire and one ground wire between converters that share no clock; the debug chain is a single probe threaded through both boards' TAPs. `nSRST` is deliberately not duplicated — it is open-drain and wired-OR by nature, and duplicating it would not buy per-board reset anyway, since `remote_bitbang` drives both resets from one pair of commands.

One image serves both roles, selected over JTAG before the CPU is released, because a station is half duplex and its transmitter shares the receiver's arena — no board ever does both. The transmitter builds one NORMAL QPSK frame and replays it; the receiver records 54 000 samples and only then runs the streaming receiver over the recording, so the analog path can be characterised independently of whether the decoder liked it. The host releases the transmitter *first*, which is what removes the need for a trigger: the whole capture window falls inside a transmission already in progress.

Table 24: Two boards, one wire, independent clocks. Measured on the stand; the payload check is `memcmp` against the transmitted packet, so "bit-exact" is exact.

Quantity	Measured	Note
Complete frames in window	2	54 000 samples at 12 kHz
Decoded bit-exact	**2 of 2**	`memcmp` vs. the transmitted packet
ADC swing	2.472 V pk-pk	about 1.643 V, the DAC's mid-rail
Residual CFO	+0.01 Hz	after preamble correction
ADC conversion	19.2 µs	64.5-cycle sample, `per_ck`/8
Receiver ISR	19.5 µs	of an 83.3 µs period, i.e. 23 %
Transmitter ISR	0.11 µs	one DAC store

What it corrected: a vector table in cached SRAM. The receiver role hung on its first run — stage "running", its TIM6 interrupt count stuck at zero, the run loop spinning until the three-minute timeout. The state read off the part was contradictory: TIM6 counting with its update flag set, the interrupt *pending* (`ISPR1` = `0x400000`) but *not enabled* (`ISER1` = 0), `PRIMASK` clear — and the vector table in memory holding the correct handler. The status block explained the disable: the first interrupt had been taken by the *stray* handler, whose job is to mask an unexpected source rather than let it wedge the device, so it had cleared the enable for IRQ 54.

The vector table lives in ordinary cached SRAM, and `vectors_set()` wrote the entry and executed `DSB` — which orders accesses but does not clean. On a Cortex-M7 an exception vector fetch reads *memory*, not the D-cache, so the new entry sat in a dirty line while the hardware still fetched the old one. Reading the table over JTAG showed the correct handler only because the line had been evicted by then, which is precisely what made the failure look impossible. `SCB_CCR` read `0x00070200` on the stalled board, `DC` set — inherited from the USB firmware the RAM image had displaced. The transmitter role never hit it because `build_frame()` runs between installing the handler and the first interrupt and evicts the line in time: the same binary on the same board, differing only in eviction timing, which is why the bug presented as role-dependent. Both entry points now clean by MVA. This had been latent under every RAM bench in the project.

Two smaller hazards came from the same root — a RAM image inherits the machine state of the firmware it replaced. `ISER3` bit 5 (`OTG_FS`) is still set when the image starts, so an inherited enable can deliver a stale interrupt as the first one and get its source masked; `vectors_irq_enable()` therefore clears the pending bit before setting the enable, and the bench masks interrupts for the whole of bring-up.

What was not measured. The obvious way to report the sample-rate offset — compare the two boards' measured TIM6 clocks — is structurally incapable of showing it, and would have produced a confident *0.0 ppm*. Each board measures TIM6 against its own DWT cycle counter, and both are derived from the *same* PLL, so that ratio is exact on each board and identical between them however far apart the crystals are. The offset is instead read off the wire, from the spacing of consecutively decoded frames against the number of samples transmitted between them. With only two complete frames in the capture window the span is 17 920 samples, and a start position good to one sample makes that a *bound* rather than a figure: the two clocks agree to within ±56 ppm. That is consistent with the +0.01 Hz residual CFO, and it is enough to say the link decodes across independent clocks — but it is not a measurement of their offset. A tighter number

needs a much longer capture, which needs the receiver to decode as it goes rather than into a buffer; that is the next step for this stand, not a result of it.

12.8 A Board That Is a Radio

Section 12.6 gave the board an identity and a host protocol, but bound the station to a *stub* PHY: queues, rate ladder, ARQ and reply timers all ran for real, and nothing reached the air. Section 12.7 gave the opposite half — frames crossing copper between two boards' converters — with no link layer above them and no host beside them. This section is the two in one image: a board a host drives over USB and a peer hears over a wire.

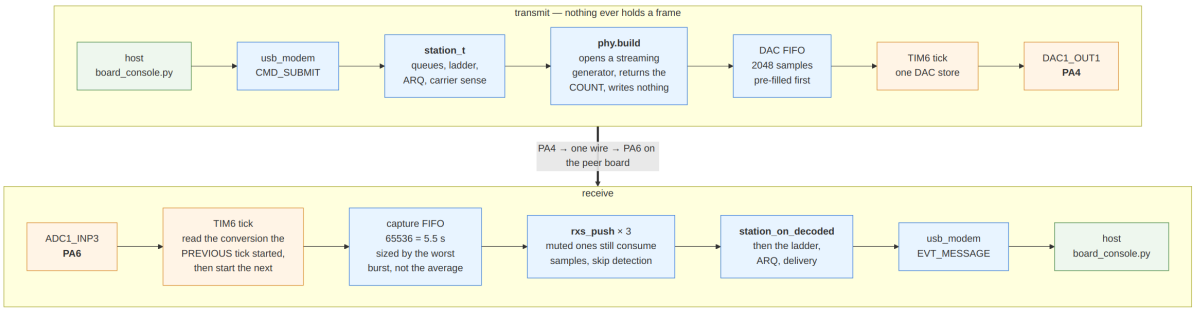


Figure 39: The radio firmware. Half duplex here is a contract rather than a simplification: the streaming generator's state lives in the shared arena *across* `txs_pull` calls while the receiver's scratch is call-scoped, so `rxs_push` must never run during a transmission — and does not, because the tick feeds only one FIFO at a time.

The transmitter never renders a frame. `station_phy_t::build` is frame-at-once: it is handed a buffer and returns a sample count. An EXTREME frame is about 456 000 samples — 912 kB — which this part cannot hold, and demoapp's answer (a 600 000-sample buffer) is not available. It is also unnecessary. The station never reads or writes those samples: `out` and `out_cap` appear nowhere in `station_poll_tx` except where they are passed through to `build`, and the count is what the caller acts on. So `build` here opens a *streaming* generator and returns the total it reports without writing a sample, and the main loop pulls from that generator into a 2048-sample FIFO as the tick drains it. The waveform is identical either way — a one-block burst with no resync is bit-for-bit a frame, which both test suites already assert.

The 12 kHz tick never blocks. The analog bench of Section 12.7 started a conversion and waited for it inside the tick, 19.2 µs of an 83.3 µs period; that was free when the decoder ran afterwards and is not free when the decoder has to keep up. Here the tick reads the conversion the *previous* tick started and immediately starts the next, so the sample is one tick old — a constant delay indistinguishable from cable length.

What it corrected: the flash images never enabled the caches. The receiving board dropped 1 522 686 captured samples — 64 % of everything its ADC produced — and decoded nothing, while the transmitting board underran its DAC 375 times. `SCB_CCR` read `0x00040200` on the running part: both L1 caches off. `startup_flash.c` had never enabled them, so code ran from flash at two wait states with no instruction cache and

every SRAM access uncached, which is five to ten times slower than the same code out of ITCM. It had not mattered while the flash images only ran a USB endpoint and a link layer, and the RAM benches never showed it because their `.text` lives in ITCM and needs no cache. Enabling both (the data cache invalidated by set/way first) cut overruns by 61×. Two consequences follow and are handled rather than discovered later: a debugger reading over the AHB-AP does *not* see dirty lines, so the beacon is cleaned explicitly before each read; and a RAM image that now inherits `DC`=1 needs its vector-table writes cleaned, which is what Section 12.7 already had to fix.

Three smaller faults surfaced with it. The flash linker script listed `.bss` *before* `.d2_bss`, and `.bss`'s `*(.bss*)` matches `.bss.g_raw` too — so the D2 lines matched nothing, the 160 kB sample ring sat in AXI, and adding the receiver overflowed AXI by 100 kB while D2 stood completely empty (`stm32h743_usb.ld` always had the right order, which is why only the flash images were affected). `TIM6_DAC`, IRQ 54, had no entry in the vector table: designated initialisers leave every other slot `NULL`, so the first tick would have vectored to address zero. And arming an empty DAC FIFO underran it once per transmission — 166 underruns across 3 frames, each one a mid-rail sample on the air — which pre-filling before starting the carrier reduces to zero.

Sizing the capture FIFO: the worst burst, not the average. The decoder's cost is wildly uneven. Quiet blocks are cheap; the commit at the end of a frame — acquisition, demodulation of every tiled symbol, Viterbi — is one long blocking call, measured at 2283 ms inside a single `rxs_push` against a 19.5 s frame. As an average that is about 12 % duty, comfortably real time. What matters is not the average but whether the FIFO can hold the samples that keep arriving *through* the burst: 16 384 samples (1.37 s) could not, dropping 33 036 samples mid-frame and decoding nothing, while 65 536 (5.5 s) decodes with no overruns at all. The beacon carries `cap_overruns` and `push_ms_max` so this is checked on the part rather than reasoned about.

The receiver follows the negotiated rung. Each open receiver runs its own detector over every sample and the EXTREME one is much the most expensive; three concurrently do not fit real time on this part. They cannot simply be opened selectively either, because every instance shares one raw sample ring and indexes it by its *own* sample counter — feed a subset and they write each other's history at the wrong offsets. So all three are opened and the unwanted ones are *muted*: a muted instance still consumes every sample, keeping the ring and its own timeline coherent, and skips only the per-block detection, which is where the cost is. Unmuting rearms the search rather than resuming, since a state machine that was mid-frame when it went quiet would otherwise decode a frame whose middle it never saw.

Which are active follows `ladder_mode(ctl.my_req)` — the rung *this* station asked the peer to transmit at — plus two others that are not optional. EXTREME always, because it is rung 0 and therefore the bootstrap, it is where the ladder decays to after a spell of silence, and it is what a peer that cannot hear us falls back to; muting it would make the link undiscoverable and unrecoverable exactly when recovery is needed. And the mode last decoded on, because `my_req` is a request rather than an observation: between raising it and the peer acting on it, the peer is still transmitting at the old mode, and dropping that mode immediately would make the station deaf for precisely the exchange that would have confirmed the change — timeout, request decays, link oscillates. In the settled bootstrap state, both terms are EXTREME and exactly one detector runs.

Table 25: Two boards, one wire, driven from two host consoles. Every figure read off the boards' beacons or the consoles.

Quantity	**Measured**	**Note**
First message, end to end	36 s	EXTREME bootstrap, ≈19.5 s of air
Second message	**15 s**	after the ladder climbed
Frames / decodes, each board	1 / 1	B decoded, replied, A decoded the reply
Capture overruns	**0**	both boards, throughout
DAC underruns	**0**	both boards, after pre-filling
Worst single decode	2283 ms	of a 19.5 s frame, ≈12 % duty
Idle listening set	EXTREME	`my_req` 0, one detector
After the climb	NORMAL+EXTREME	`my_req` 9 (A), 7 (B)
SNR reported	+15.0 / +9.7 dB	A and B

Streamed bursts — a whole selective-repeat window behind one preamble — are the one part of the link this section does not settle: their first implementation on the boards failed in a way that every plausible theory misdiagnosed, and finding out why is a story worth a subsection of its own.

12.9 Debugging the Streamed Bursts

The streaming continuation itself is small: after a decoded block whose packet carries the link layer's streamed marker, the receiver takes the next block from the deterministic offset past this one instead of hunting for a preamble, and stops on the ack-request bit of the last block. (`phy.receive_burst` stays `NULL` by design — the station only reaches it from the frame-at-once `receive` path, which is handed a whole recording this firmware never has.) The same machinery passes every host test and runs demoapp's transfers. On the boards it failed totally: block 0 of a stream decoded, and *every* continued block failed its data CRC, on every attempt, at every rung — while the same file crossed byte-exact as per-frame fragments.

The failure looked exactly like a channel problem, and the two boards are the first stand in the project where the two ends share no sample clock — so clock drift was the obvious suspect, with quantization noise and the resync geometry close behind. All of it was wrong, and what kept the search honest was the discipline in Figure 40: reproduce on the *host* first, where a cycle costs seconds against the seven minutes of a build–flash–transfer round on the boards.

Step 1: exonerate the physics. A host harness (`bench/burst_repro.c`, `make burstrepro`) builds the *firmware's* streamed waveform — station-style `EXT_DATA` blocks through the streaming generator — and walks it with the firmware's own burst logic, under the board's impairments applied one at a time: the 12-bit DAC re-read by the 16-bit ADC at 3/4 scale, the mid-rail DC offset, and a sample-rate offset between the ends. It decoded 8 of 8 blocks under all of them at once, up to 56 ppm — worse than the wire, whose +0.01 Hz residual CFO puts the real offset near 5 ppm. It also replayed the DAC FIFO's exact variable-sized pull pattern (sample-identical to a flat render) and interleaved transmit-generator arena traffic mid-walk (harmless). Five plausible theories died in one afternoon without touching a board, and what remained was a certainty

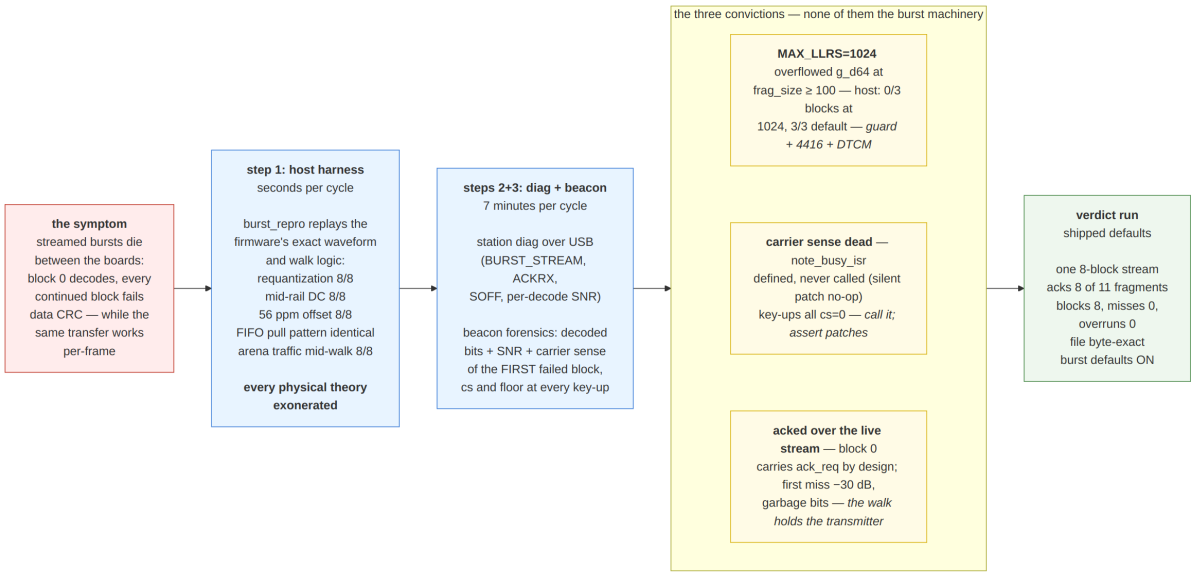


Figure 40: The diagnostic ladder. The host harness exonerates the physics before any board cycle is spent; the board-side instruments then carry the burden of proof, and each of the three convictions ends with a measured signature, not an argument.

worth more than any of them: the bug was in the firmware's *interaction*, not in the shared DSP.

Steps 2 and 3: make the boards testify. Two instruments carried the rest. The station's diagnostic event stream (`UP_EVT_DIAG`, readable from the host console) reports the link layer's own decisions — windows engaged, streams sent, bitmaps received, streaming abandoned and why. And the firmware's beacon grew *forensic* fields: for the first failed continued block, the decoded bits of its leading 44 bits (the LC word and sub-header are predictable, so near-correct bits mean "aligned but noisy" while garbage means the walk ran off the waveform), the SNR the demodulator saw, and the carrier-sense level — plus a recorder of the last four transmitter key-ups with the carrier-sense mean and floor at each. Reading them over JTAG needs neither a debugger halt nor instrumentation in the signal path.

The three convictions. None was the burst machinery.

First, the radio build had inherited `-DMAX_LLRS=1024` from the small-frame bench images. A streamed block's LLR store is sized by the padded symbol count, and any fragment size above roughly 100 bytes overflows `g_d64` — silently, because nothing guarded it, and invisibly on the host, where the overflow lands in the neighbouring receiver instance's rows and reads back self-consistently. The harness convicted it cleanly: 0 of 3 blocks decode at 1024, 3 of 3 at the default. The receiver now *refuses* a header whose announced frame exceeds its buffers — the header was valid; this build cannot decode what it describes — and the radio build sizes `MAX_LLRS` for the largest EXT frame, with the store placed in DTCM by the linker script.

Second, carrier sense was dead — and had been since the commit that claimed to move it into the converter tick. That patch defined the tick-side function and never inserted the call: the scripted edit's pattern had the wrong indentation, and a text replacement with no assertion silently replaced nothing. The measured mean stayed zero, `channel_busy()`

answered idle forever, and the commit's fix had never run. The key-up recorder caught it in one glance — `cs=0` at every key-up, where a quiet wire reads about 2×10^4, since the idle peer's DAC holds mid-rail. The repair is one call; the lesson is a rule: every scripted patch replacement gets an assertion.

Third, the station answered block 0 *over the top of the rest of the stream.* A stream's first block carries the ack request on purpose, so a peer that cannot stream still replies; with carrier sense dead, nothing stopped the receiving board keying its bitmap ack two seconds into the peer's 9.75 s transmission — and key-up drops its own capture. The forensics showed precisely that: the first failed block decoded at −30 dB with garbage lead bits — the signal was simply gone — while the transmitting board's counters showed a complete, clean carrier. The structural fix does not lean on carrier sense: a receiver whose walk still expects blocks holds its own transmitter, with a deadline so a peer dying mid-stream cannot mute the station forever. The ack then leaves when the walk ends, which is the contract the link layer wanted all along.

The verdict run. With all three fixes and *no* host-side configuration — the burst defaults are now on — one 8-block stream behind a single preamble acked 8 of 11 file fragments in a single bitmap, the receiver's beacon read `blocks 8, misses 0`, capture overruns and DAC underruns stayed zero on both boards, and the 1200-byte incompressible file crossed byte-exact. The 224-second-frame hazard that had earlier argued for shipping bursts off (fragment size is fixed at engage, and a rung collapse mid-transfer keeps sending fragments sized for the rung it engaged at) turned out to be a symptom of these same bugs poisoning the rate controller, not a property of the machinery.

The instruments outlived the hunt: the harness is a build target, the beacon parser is a checked-in tool (`bench/radio_beacon.py`) whose field list lives in one place beside the append-only beacon struct, and the triage ladder itself is recorded where the next investigation will find it.

12.10 Hardening: the Stress Campaign and a DC-Blind Front End

With streamed bursts working, an 8 kB incompressible transfer — 35 console parts, the largest the boards had attempted — was put through the link as a stress test, and sustained load surfaced what a six-part file never could. Three more failures fell out, each again convicted by measurement, and the campaign ended in a change of philosophy rather than another patch: the receiver's front end is now DC-blind by construction, and its reliability is enforced by a test gate instead of by luck.

Frames longer than every carrier-sense time constant. Fragment size is fixed when a burst transfer engages, so a transfer engaged at rung 10 with 203-byte fragments that collapsed to rung 0 sent *224-second* EXTREME frames. Carrier sense cannot survive that: the noise floor's climb crosses the busy threshold after about 176 s of continuous signal, and the rebase re-baselines at 300 s — either way the peer eventually declares the channel idle and keys over the frame, which times it out, keeps the rung at zero, and makes the next frame another 224 seconds. A death spiral with both boards perfectly healthy. The fix is structural: `poll_tx` now *disengages* a burst whose next fragment would exceed 45 s of air at the rung the link actually has (diagnostic `BURST_REFRAG`); the message falls to the legacy path, whose payloads are air-time capped, and the burst re-engages with

honestly sized fragments when the ladder recovers. Streams get the same refusal, and the rebase window rose to 300 s — above the worst frame the station can now emit, not above demoapp's.

The boot latch. Gating only the carrier-sense *feed* for ADC warm-up left the measured mean at zero, and the busy check — called every millisecond from boot — read that zero and snapped the noise floor to its clamp; a genuinely quiet wire then read "busy" forever. The evidence was exact to the millisecond: both boards' first key-ups ever landed at `ms=300031` and `ms=300029` — the rebase window precisely, and a collision, two stations going deaf-mute together at boot. The warm-up gate now covers the consumer too.

The wire that was never broken. The campaign's longest detour was a diagnosis this report must retract as instructively as it was made. The raw capture ring showed one board's ADC flatlined at a constant 0.81 V — and the obvious call, a failed breadboard contact, fit every symptom: levels that wandered between reads, a link that healed mid-run and relapsed, a power cycle that "fixed" it. The wire was innocent. The tick only writes the DAC *while transmitting*, so at the end of every frame the pin held the frame's *last sample* — anywhere within ±0.9 V — until the next transmission. Decoding never noticed, because the demodulator is DC-blind (bin 0 is unused); carrier sense reads mean *square*, and a DAC parked 0.83 V off mid-rail put a constant 2.7×10^8 on the peer's carrier sense, at a level that changed with every frame's final sample. One line parks the DAC at mid-rail at transmit end — and the 8 kB transfer promptly completed, byte-exact, in 255 s.

The lesson, made structural. A receiver that any input operating point can break is a lab demo, not a receiver. Two changes make the property hold by construction rather than by bug-fix (Figure 41):

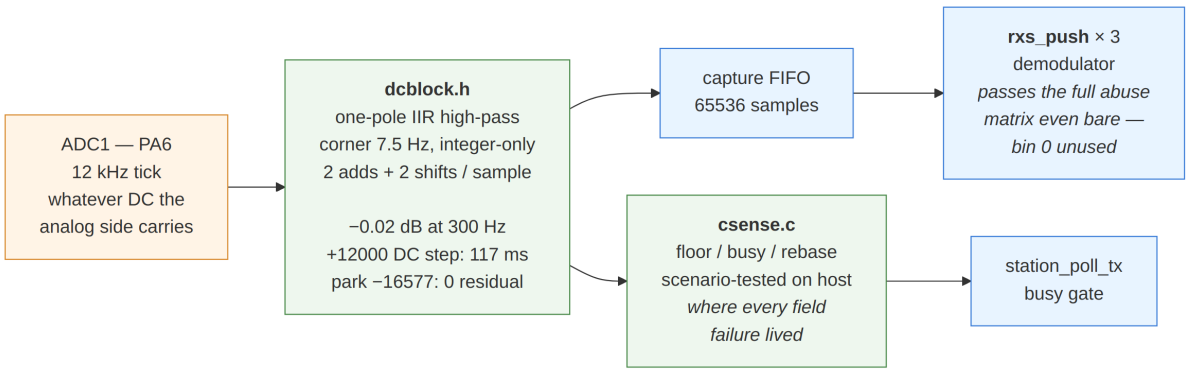


Figure 41: The DC-blind front end. One integer high-pass at the input makes every consumer indifferent to the analog operating point; carrier sense — where every field failure of the campaign lived — is a shared module with a host scenario suite.

First, every ADC sample passes a one-pole IIR high-pass (`dcblock.h`: a leaky-integrator DC tracker subtracted from the input, integer-only, two adds and two shifts per sample) *before* the capture FIFO and carrier sense. The corner sits at 7.5 Hz, two decades under the band: measured, the filter costs −0.02 dB at 300 Hz, absorbs a +12 000-LSB DC step in 117 ms, and leaves zero residual on the measured park level after

one second. Whatever DC the analog side carries — a parked peer, the bias network that AC-coupling capacitors would need, a ground offset between boards — is now a non-event, which also clears the way for the capacitor hardware option.

Second, carrier sense moved out of the firmware into a shared module (`csense.c`) precisely so it could be *tested*, and `make robust` is the resulting reliability gate. Its decode half drives the burst harness through an input-abuse matrix — static DC to ±18 000 LSB, DC steps, 2.5× clipping, 50 Hz hum, impulse clicks, stuck-converter dropouts, all at once, at 5 ppm of clock offset — and its verdict reframes the campaign: *the demodulator passes the entire matrix even without the blocker.* Every field failure lived in carrier sense. So carrier sense gets the scenario half: boot into a quiet wire must read idle within 5 s (the latch), the parked DC must read idle through the blocker and demonstrably latches bare (why the blocker is not optional), a 45-second frame — the new maximum — must hold busy end to end, frame/gap cycles must track, and a DC step may cost at most a 2-second transient. Seven scenarios, each a replay of a measured failure; the gate fails on every bug this campaign found, had it existed a day earlier.

Table 26: The stress campaign, by the numbers. All figures measured on the boards or by the host gate.

Quantity	Measured	Note
8 kB transfer, final firmware	**241 s, byte-exact**	35 parts, 36 bitmap acks, 5 timeouts
Same, before the DC blocker	255 s	8 timeouts
Worst pre-fix frame	224 s of air	203-byte frag at rung 0
Floor climb crosses busy	≈176 s	why frames are capped at 45 s
Boot-latch key-ups	`ms` 300031 / 300029	the rebase window, both boards
DC blocker passband	−0.02 dB at 300 Hz	corner 7.5 Hz
DC step absorption	117 ms	+12 000 LSB
Abuse matrix, demodulator bare	**8/8 blocks, all rows**	DC ±18k, clip, hum, clicks, dropouts
CS scenario suite	7/7	each a measured field failure

12.11 Calibrating the SNR Estimate

The chat transcripts from the two-board stand exposed one more defect, subtler than anything the stress campaign found because nothing *failed*: messages were delivered, but the rate ladder whipsawed — `rung 10 → 0 → 8` with zero losses — and every bootstrap climb crawled. The tell was in the diagnostics: EXTREME frames reported +0.5 dB on a wire that NORMAL frames reported at +12 dB.

A host sweep of all seven (mode, modulation) combinations the ladder emits found the mechanism (Figure 42, left). The integer estimator derives SNR from LLR moments, and LLRs *rail* at high SNR — at roughly the same raw ratio in every BPSK mode. The estimator then subtracts the mode's tiling gain (6.02 / 12.04 / 18.06 dB), which is correct near the sensitivity knee where the ratio still moves, but applied to a railed ratio it simply spreads the ceilings apart: predicted separations 12.04 and 6.02 dB, measured 11.9 and 6.1. Modulation adds its own spread. The consequence is quantitative: the ladder compares these readings against one `LADDER_SENS` table, and EXTREME's +0.5 ceiling sits below $\texttt{SENS}[10..12] + \texttt{MARGIN\_UP} = +3.2..+7.2$ dB while NORMAL's +12.4 does not — so a station that had just decoded an EXTREME frame could never request the top rungs, however clean the wire, and one dip to rung 0 dragged both stations down after it.

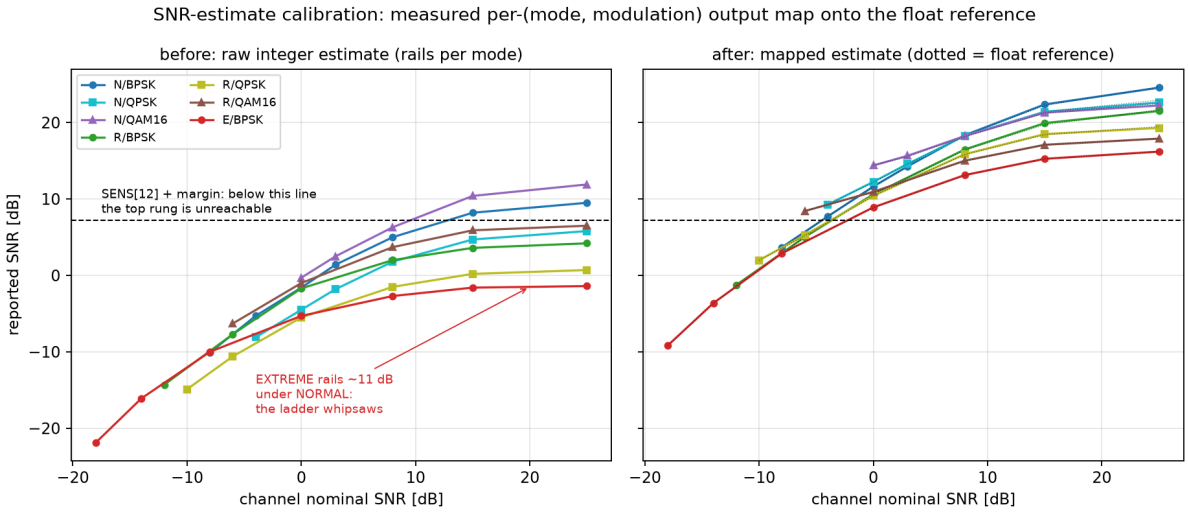


Figure 42: The calibration, before and after. Left: the raw integer estimate rails at a per-(mode, modulation) ceiling, and EXTREME's ceiling falls below the threshold the top rung requires. Right: the mapped estimate, which tracks the float reference (dotted) to ∼0.3 dB across the grid.

The fix is a measured map, not a touched constant. Both twins were fed the *same* noisy waveforms across `simulate_channel`'s own nominal convention — seven combinations, five to seven nominals each — and the resulting (integer estimate, float estimate) pairs became piecewise-linear output maps. The target is the float estimator deliberately: it is the reference the rate ladder was system-tested against, and near the knee the two estimators already agreed (the fixed-point suite asserted ±2 dB there), so the map is close to identity exactly where the existing calibration was right and lifts only the railed region. The knots live in one place (`FixedReceiver.SNR_MAP`), the generator emits them into `rom_modes.h`, and `rxd_snr_map()` applies them in both C receive paths — the parity between twins is by construction, and the fixed-point suite now asserts the new contract: mapped-integer tracks float within 3 dB at nominals −7/0/+15 (measured medians 0.5 / 0.3 / 1.2 dB).

On the boards the change is immediate: EXTREME frames read +16.2 dB and NORMAL +20.9..+22.5 on the same wire — consistent to ∼6 dB across modes instead of 12 apart, the residual being the estimators' genuine mode differences — and the ladder now climbs 0 → 11 in a *single* exchange where it previously stepped for minutes. The staleness decay's conservative re-probe after long silence remains, by design.

12.12 A Sentinel on the Air

Calibrating the estimate fixed what the receiver *measures*. What it reports when it has measured nothing took longer to find, because every symptom pointed at the channel.

An ordinary two-board chat session, read afterwards from both consoles, showed replies arriving 18–21 s after they were queued on a wire running at +20 dB, interleaved with replies that arrived in one second. Both stations reported healthy counters, no timeouts and no retransmissions. The rung display made it worse: it read `rung 11` at attach and the very next frame went out at rung 0.

The instrumented exchange settles it in one screen. Four messages, 75 s apart, diagnostics on:

```
A 11:13:56 send "one"   .  rung 11 -> 0 (losses 0, cap 12)  tx rung 0
B 11:14:15 « "one"      .  rung 11 -> 0 (losses 0, cap  0)  tx rung 0
A 11:15:11 send "two"   .  rung  0 -> 11 (losses 0, cap 12) tx rung 11
B 11:15:12 « "two"      .  rung  0 -> 11 (losses 0, cap 12) tx rung 11
A 11:16:26 send "three" .  (rung unchanged)      tx rung 11
B 11:16:27 « "three"    .  rung 11 -> 0 (losses 0, cap  0)  tx rung 0
```

`losses 0` throughout, so no loss fallback; A's cap is 12 throughout, so A's view of the channel is correct. B's cap collapses to 0 on the exchanges whose gap exceeded 60 s — and 60 s is `SNR_MAX_AGE_S`, the age at which the SNR filter empties.

The mechanism is three lines of arithmetic. A frame's link-control word carries `ctl_filtered_snr()` evaluated when the frame is *built*; with an empty history that function returns its "no measurement" sentinel, −99 dB. `lc_pack` quantised that as if it were a measurement, $\mathrm{rint}((-99+24)/2) = -38$, and clamped it into the field's low rail, code 0. The peer unpacked code 0 as a genuine −24 dB report, and `ctl_tx_rung` caps the transmit rung by the peer's report: no rung's sensitivity clears −24 dB, so the cap became 0. A station that had merely been quiet for a minute was telling its peer *"I hear you at −24 dB"*, and the peer answered a one-byte acknowledgement with a 19-second EXTREME frame. Conversational pacing straddles 60 s constantly, so this was the common case rather than an edge: three of four replies in the run above.

The repair is to give the field's code 0 the meaning it was already carrying by accident. Code 0 now means *no measurement*; genuine reports occupy codes 1–15, −22 to +6 dB, and a real reading below the floor clamps to code 1 — "I hear you badly" must stay distinguishable from "I have not heard you", which is the entire content of the bug. `ctl_tx_rung` applies the report cap only when a report exists; a peer that has genuinely gone away is still handled by the request's staleness decay, which was doing that work all along. Choosing the code the old sentinel already clamped to makes the change compatible in both directions: an unpatched sender's absent measurement lands in code 0, so a patched receiver repairs the exchange unilaterally, and an unpatched receiver reads code 0 exactly as it did before. (Argued from the encoding; both boards on the stand are patched, so the mixed case is not measured.)

Re-running the identical script gives B's reply rungs as $0, 11, 10$ where they were $0, 11, 0, 0$: acknowledgement air time falls from ≈19 s to ≈1 s on every exchange that was broken. The one remaining slow frame is the first after twelve idle minutes, which is the request's own decay and is intended.

Two displays were repaired with it, and the reason is not cosmetic. The diagnostic printed `cap 0` for "no report" — a cap of zero and the absence of a cap rendered identically — which is precisely what made the defect expensive to read; it now prints `no peer report - request stands`. And `status` reported `stats.last_rung`, the rung of the last frame actually transmitted, which after an idle night is a memory rather than a state: it is what read `rung 11` three seconds before transmitting at rung 0. The status frame now carries the rung the *next* frame would use, and the consoles show the last transmitted one only when the two differ (`rung 0 (last tx 11)`).

The general lesson is worth more than the fix. A sentinel encoded as a number survives every layer that treats it as data: it printed as −99.0 dB in a status line, travelled the air as −24 dB, and was consumed by a control law that had no way to tell a measurement from its absence. The same day's audit turned up five instances of the same shape — the SNR display, the peer report, the -10^9 "never" timestamps, the cap, and this one, the only one that had escaped onto the wire. Where a quantity can be absent, absence needs

its own representation, and every consumer needs to be unable to mistake it for a value.

12.13 Broadcast on the Boards

Broadcast (Section 7.8) is delivery without acknowledgment: a streamed burst with the ARQ machinery subtracted, plus a SYNC frame at the head of every group so a receiver that was not there at the start can join. Both models implement it as a *recording* walk — `bc_receive()` takes a slice of audio and scans it for groups — and that is exactly what a board cannot do. The firmware never holds a recording: samples arrive 256 at a time out of a capture FIFO and leave through a streaming receiver that emits one event per decoded (or failed) block.

So the boards needed a fourth path. `usb_radio_main.c` builds one group per keying and walks the received group with `bc_advance()` over that streaming receiver — the same `rxs_continue_burst()` stepping burst ARQ uses (Section 12.9), with the acknowledgment machinery absent rather than removed. Figure 43 traces one broadcast across the wire. Three properties had to be right that no host model ever faced.

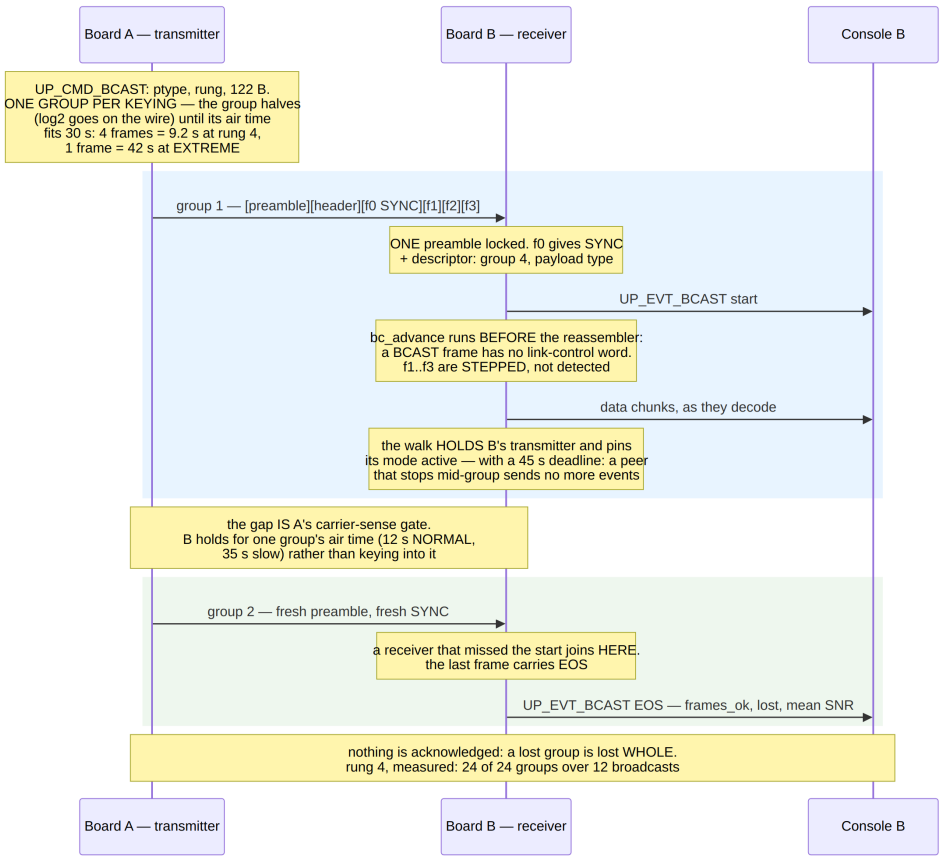


Figure 43: One broadcast between the two boards. The transmitter keys one group at a time; the receiver locks a single preamble per group, reads the descriptor from the SYNC frame and then *steps* the remaining blocks without detecting again — holding its own transmitter while it does. Nothing is acknowledged, so the EOS frame's statistics are the entire feedback path.

The dispatch order is load-bearing. A BCAST frame is Data-shaped — same header, same CRC, same block geometry — but its reserved field is not a link-control word. `bc_advance()` therefore runs *before* `burst_advance()` and the station's reassembler; a broadcast frame reaching the latter would be read as an ARQ fragment with a garbage sequence number and a garbage acknowledgment request.

A walk that holds the transmitter must be able to expire. While the walk runs, the receiver is not running its detector at all — it is stepping to known block boundaries — so the walk pins its receiver instance active (muting one stops detection, and unmuting rearms the search rather than resuming it) and gates the station's key-up, for the reason Section 12.9 measured: answering over a peer's live stream costs the rest of it. That leaves the failure the burst walk already had to solve. The walk advances only on events, and a peer that stops mid-group produces none: without a deadline the board stays mute for good. With one — 45 s, a group's air-time cap plus margin — it recovers on its own.

One group is one keying. Its air time is therefore bound like everything else the station emits, and for the same reason the 224-second frames of Section 12.10 were fatal. The group size *halves* — $\log_2$(group) is what travels in the SYNC descriptor, so it must stay a power of two — until the group fits 30 s: four 26-byte frames are 9.2 s at rung 4, while at EXTREME a single frame is already 42 s. The yield between groups is the carrier-sense gate itself, and a station that has heard broadcast traffic recently holds its own transmitter for about one group's worth of time (12 s at NORMAL, 35 s at the slow modes) rather than keying into the gap.

The rung decides who can hear it

A broadcast has no peer to negotiate with, which makes the rung a choice rather than a measurement — and the choice decides whether anyone is listening at all. Which modes a board keeps active follows the rung it has negotiated, and decays to EXTREME alone after a couple of minutes of silence (Section 12.8); a broadcast sent at a mode the intended listener has already dropped is inaudible, at any signal level.

The first default got this wrong, and the wire said so within two commands. It took `stats.last_rung` — what this station last *transmitted* at — and floored it at rung 4 on the reasoning that leaving the rung unspecified cannot mean "take four minutes". On a link running at EXTREME that floor turns "this link is at EXTREME" into "broadcast at NORMAL": the sending board keyed a perfectly good NORMAL group, and the receiving board, whose ladder had decayed to EXTREME-only listening, recorded a carrier at 1.1×10^8 of mean square — full strength — with zero capture drops, zero events and nothing decoded. Twice, with every counter on both boards healthy.

The default is now `ctl_tx_rung()`: the rung this station would send the peer an ordinary frame at, which *is* the peer's own request and therefore the one rung the peer is certainly receiving on. Nothing is floored — a broadcast nobody can hear is worse than a slow one, and the air-time cap keeps the slow case honest by collapsing the group to a single frame. An explicit `-r` is still honoured exactly as given, which is how a beacon reaches stations that have never been heard from: EXTREME is the only mode an idle receiver is guaranteed to have kept. And because "nothing arrived" looks identical from the host whatever the cause, the board now logs the rung it chose:

```
board:  broadcast:  12 B at rung 0 (EXTREME), 1 frame(s) per group,
42.1 s each
```

The measurements bracket both cases. A board freshly booted and having never exchanged a frame — listening on EXTREME alone — received a `bcast -r 0` byte-exact at +16.2 dB, in two groups of one frame each; the same transfer over a link *running* at EXTREME, which the floor used to break, now goes out at rung 0 and arrives. At rung 4 with a 122-byte payload (two groups per broadcast, four frames then two), a twelve-broadcast run delivered **24 of 24 groups**, 72 frames, and reported zero losses at +24.7 dB; across the whole campaign 38 of 39 groups arrived.

Pinning the one loss on the air, not the code

The single missing group is worth the paragraph, because a whole group vanishing looks exactly like a defect. It went *whole*: not one frame of it decoded, including the SYNC block that follows the preamble. The host bench that answered the question builds the same group with the *same* firmware code and walks it with the same `bc_advance()` logic through the real streaming receiver (`make bcrepro`, now part of the `make robust` gate); it decodes 6/6 frames at rung 4 and 2/2 at rung 0, under the wire's requantization, so the build-and-walk path was exonerated in seconds rather than in seven-minute board cycles.

That left the air, and the beacon was extended to say so: an eight-deep event ring recording every receiver event with its mode, type, packet type, capture drops and position, and the loudest mean-square the receiver heard while not transmitting. At the lost group the receiver had heard a full-strength carrier (1.15×10^8, against $\sim 2 \times 10^4$ for a quiet wire) with *zero* capture-FIFO drops, and had produced no event at all — a missed acquisition, which takes every frame behind it with it. That is the shape of every non-ARQ loss, and it is the failure rate ARQ has been hiding all along by retransmitting (Section 10.7 measured the same thing from the other side, at ∼8% of acquisitions before the lag-N unwrap fix). Broadcast cannot retransmit, which is precisely why the EOS event reports what arrived.

A file over the broadcast

`bcastfile [-r <rung>] <path>` carries a file the same way — raw bytes, the opaque payload type, no delivery guarantee — and the receivers on every path (both consoles and the socket demo) store it as `rx_broadcast.bin`. The constraint worth the paragraph is memory: the socket demo holds a 64 kB source buffer because a PC can, while the board's broadcast source is 8 kB of DTCM, so the file *streams* from the host. Two bits of the command's payload-type byte do the framing — *more follows* and *continuation* — with both clear meaning the old one-shot broadcast, so nothing existing changed; the host paces its chunks against a `bc_free` field the status frame now reports.

Chunking put two rules into the transmitter that the one-shot case never needed. The EOS flag is emitted only once the host has declared the stream complete — an EOS computed from "buffer drained" would fire at every pacing stall. And while chunks are still arriving, only *full* groups are keyed: a starved tail group would go out without EOS, and the receiver's walk would never see the end of a stream whose true last frame was still in transit over USB.

The first live run earned its regression comment. The C console's chunk pump marked completion by setting its offset to −1 — and kept looping, because −1 is less than the

length and the loop had no `break`: it re-sent overlapping chunks twenty-four times, and the receiving board stored 19 380 bytes from a 1 024-byte source. (The Python console's pump, written the same hour, had the `break`.) With the loop fixed and the board taught to ignore a continuation when no broadcast is in flight, the retest delivered the 1 kB file byte-exact in 19 seconds at rung 12 — 44 frames in eleven groups, zero lost, one completion message.

The negotiation belongs where the payload waits. The socket demo has always *held* a broadcast on an idle link and driven the ladder up first (`bc_poll_link`: control-class probes every 25 s, release when the link can carry a broadcast, give up after 180 s). Porting broadcast to the firmware left that machinery host-side, on the reasoning that the host sees the status stream and can probe with a message first — and the first real operator promptly typed `bcastfile` into two fresh consoles and watched nothing happen, because the idle-link default is rung 0 and the first 42-second EXTREME frame shows no evidence of itself. Two patches told the story in order: first the board's announce line gained the *consequence* (`~32 min total - idle link defaults to EXTREME: exchange a message first, or use -r`), and then the honest fix landed — `bc_poll_link` runs on the board now, semantics line for line. Measured with the operator's exact flow, no priming, no flags:

```
12:11:38 board:  broadcast:  held - link is idle, probing to bring it
up (release at NORMAL, give up after 180 s)
12:12:14 board:  broadcast:  1024 B at rung 12 (NORMAL), 4 frame(s)
per group, 1.6 s each, ~18 s total
12:12:33 [B] stored 1024 bytes as rx_broadcast.bin
```

Thirty-six seconds from command to release, zero operator steps, the LINK probe visible on the peer's console as an ordinary message. An explicit `-r` bypasses the hold: the operator said where.

And the console needs no buffer at all. The first `bcastfile` read the whole file into a host buffer whose 64 kB cap refused the first real file (a 73 kB PNG). The cap was a leftover of one-shot thinking: the pump is chunked and paced against `bc_free` anyway, so it now reads the file *sequentially*, a kilobyte per chunk as the board drains — no buffer, no cap, with a short read mid-stream (the file truncated underneath the console) ending the broadcast cleanly rather than leaving the board waiting for continuations forever. The board never held more than its 8 kB source either way; only the host had forgotten that. Figure 44 draws the complete file-broadcast flow as it now stands — the chunked feed, the hold with the capability exchange as its probe, and the loss accounting the campaign forced into the EOS statistics.

Blocks, receiver statistics, and a broadcast that adapts

The last extension makes the one-way street report back. When the sender *holds* a capability record whose new `CAP_BC_STATS` bit is set — the receiver's declaration that it will answer, exactly the negotiation the operator asked for — the stream is cut into ~45 s blocks. Each block ends with an *end-of-block marker*: a dataless single-frame group (SYNC flag, zero data bytes, no EOS), chosen over new frame flags because an old receiver appends zero bytes for it and walks past — the wire stays compatible with every earlier

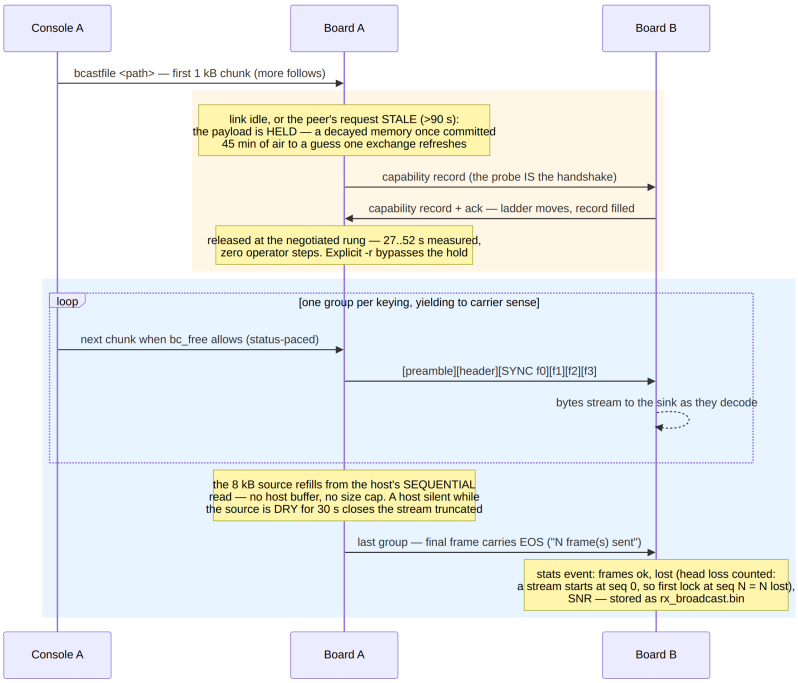


Figure 44: The chunk-fed file broadcast, end to end. The hold phase (top) probes with the capability record itself when the peer is a stranger — or whenever the remembered rung has gone stale, after a 10-minute-old memory committed 45 minutes of air to what one exchange refreshes. The stream phase repeats one group per keying, the host re-reading the file sequentially against the pacing field in the status stream.

firmware. The sender then pauses, and the receiver answers with a small unicast frame (the last free packet type): frames decoded, frames lost since the stream began, the SNR, and the rung it wants. The sender logs the quality and *re-rungs the remaining groups*, which this wire has always permitted — every group re-announces its geometry in its SYNC descriptor. Figure 45 traces the measured exchange.

The first two live runs each contributed a rule. The receiver's "desired rung" initially came from its rate controller — which is the *chat link's* decayed memory, and answered "rung 0" into a stream it was decoding at +22 dB; the request now comes from the stream's own measured SNR mapped through the ladder sensitivities, which is the question actually being asked. And the reply must *fit the window*: the first exchange keyed a 22-second EXTREME stats frame into a five-second pause, the sender resumed its groups mid-reply, and the collision cost the receiver the rest of the stream — 497 bytes stored of 8192. A receiver whose reply would need a rung below NORMAL now stays silent, and silence is itself information.

The verification run started deliberately slow: `bcastfile -r 8`, 8192 bytes. One block later the stats frame came back — `200 ok, 0 lost, asks rung 12 - re-rung` — the remaining groups went out at rung 12, and the file stored byte-identical with 350 frames and zero lost, a minute faster than the same file at flat rung 8. A broadcast to strangers, or to a peer that never declared the bit, remains exactly the classic silent stream.

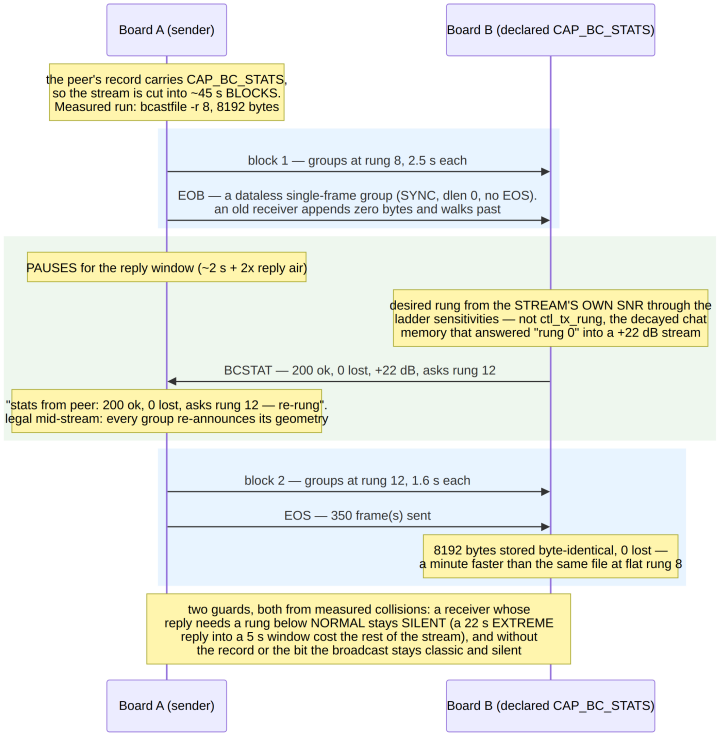


Figure 45: Block statistics and the mid-stream re-rung, with the measured run's numbers. Both guard rules at the bottom came from collisions the first live runs produced.

Adaptation under a fade

The wire between the boards cannot fade, so the fading test is a host twin in the project's usual style: `make bcfade` runs the firmware's block/stats/re-rung machinery — sender state machine, receiver walk, both control decisions — over an AWGN channel whose SNR follows a V-shaped profile ($+20 \to -5 \to +20$ dB over a 60-second floor). Its first run earned the harness its place: the adaptation *did not adapt*. The receiver's SNR estimate is measured only on frames strong enough to decode — survivorship bias — and read +22 dB inside the −5 dB fade while 41% of a block died; meanwhile the EOB markers and stats replies died with the fade, so the feedback vanished exactly when it was needed. Two control laws fix it, harness-proven and then ported to the firmware verbatim: a lossy block anchors its rung request on the receiver's *own last ask*, never on the SNR, with severe loss (>40%) going straight to the NORMAL floor; and a promised reply that never arrives steps the sender down two rungs and shortens the next block to 20 s — silence is itself feedback. On identical fades:

fade floor	arm	delivered (of 8192)	rung trajectory
−5 dB	fixed rung 12	4154	12 throughout
−5 dB	adaptive	**5481** (+32%)	$12 \to 10 \to 8 \to 4 \to 12$
−2 dB	fixed rung 12	4392	
−2 dB	adaptive	**6100** (+39%)	

Figure 46 draws the measured deep-fade run as the exchange the two ends actually had — including the two windows in which nothing was exchanged at all, which is where half the control law lives.

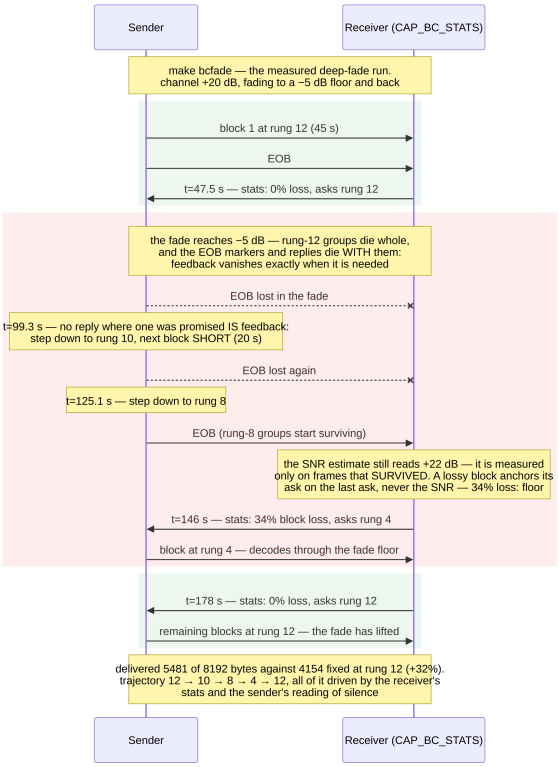


Figure 46: The deep-fade run of `make bcfade`, drawn from its transcript. The red span is the −5 dB floor: the lost EOB markers become step-downs (silence as feedback), the survivorship-biased SNR is overruled by the block's loss rate, and the rung-4 blocks carry the stream through the floor until the recovery ask lifts it back.

What the clean wire then taught

Verifying the ported laws on the boards flushed out five real firmware defects, one per run, each convicted by the boards' own counters — a sequence worth recording because every one masqueraded as something else first:

- The *sender was not listening on the mode it broadcasts in*: `follow_rung` knew nothing about broadcast, the active-mode mask read EXTREME-only in the middle of a NORMAL broadcast, and every reply the receiver keyed was physically undecodable.
- The reply window was anchored at EOB *build* time, spending its first ∼2 s on the EOB's own air; it arms at carrier drop now, sized ∼9.4 s for the real chain (the peer's end-of-frame commit, its keying, the reply's air, our commit) after timestamps showed the receiver keying at :35 against an expiry at :40.
- The reply transport is pinned to the NORMAL floor — BPSK 1/2 decodes at any SNR the stream itself survives, and the desired rung rides inside the frame — and the receiver logs `stats keyed`, which is what turned counter guesswork into comparable timelines.
- After *every* transmission of our own, the receivers are re-armed: the capture stream stitches pre-transmission samples to post, the detectors false-lock on the seam, and the re-arm skip blinds them for exactly the 1–4 s in which a stats reply arrives.

Twenty-two garbage-header events in one run; two of three replies lost. The chat link never noticed in a year of running, because its replies arrive seconds later.

- The station must not key into a stats window the broadcast opened: a leftover chat frame in the window defeated every *first* exchange while later ones worked — which is precisely why it masqueraded as a timing race until the key-up recorder said otherwise.

The final clean run: three stats windows, three replies decoded within 1–2 seconds each, the deliberate rung-8 start re-rung to 12 in the first window, and the file stored byte-identical. The fourth item is the one with reach beyond broadcast: post-transmission receiver blindness was latent in every half-duplex exchange this system runs, and only a reply fast enough to land inside it could make it visible.

12.14 The Capability Handshake, and 2 kB Messages

The last structural inefficiency the two-board stand exposed was not a defect but a shape. A 68 kB file went out as 290 console parts, each part one 256-byte station message, each message two burst fragments — so the link paid one acknowledgment per 260 bytes and reached about 50 B/s at rung 12, on a wire whose raw rate is ∼1 kbit/s. The cap was `ST_MSG_MAX`: a message is one pool slot, and a two-fragment transfer can never fill the eight-fragment window the streaming machinery was built for. And underneath that sat a protocol habit worth retiring: everything a station knew about its peer, it had learned *by failing at it*. Streaming capability was inferred from a window that came back one-eighth acked (Section 7.5), then re-probed every eight transfers in case the verdict was a forged fade; the fragment size was chosen from the sender's own limits; and `sendfile` opened with a bare `LINK` frame whose only job was to make the ladder move.

The handshake (Figure 47) makes the peer's abilities declared instead of discovered. A ten-byte record — version, capability bits (`stream ext ldpc burst bcast`), message size, window ceiling, pool slots, firmware version — rides a Data frame whose link-control flags are 7, the third combination impossible in the legacy protocol (the burst frames took the other two). Three legs: the initiator sends its record when it has bulk traffic and knows nothing about the peer; the peer stores it and answers with its own record plus an acknowledgment bit; the initiator's next frame carries an ack of that sequence number, which is the third leg — the peer now knows its record arrived. On the stand the whole exchange costs one second at the control rung:

```
23:05:23 [A] . caps sent:  stream ext ldpc burst bcast msg 2048 B,
window 8
23:05:24 [A] . caps received:  stream ext ldpc burst bcast msg 2048 B,
window 8
```

Three properties carried the design:

Silence is a fact about the peer, not the channel. An older firmware reads flags 7 as a no-data frame and never answers, so the probe's timeout is *forgiven* by the rate controller — charging it would walk `consecutive_losses` toward the hard rung-0 fallback for talking to an old radio — and after two unanswered tries the peer is marked legacy and today's defaults apply, re-asked after five minutes. This is measured in the test suite: the legacy-peer case completes its transfer on the defaults with the ladder untouched.

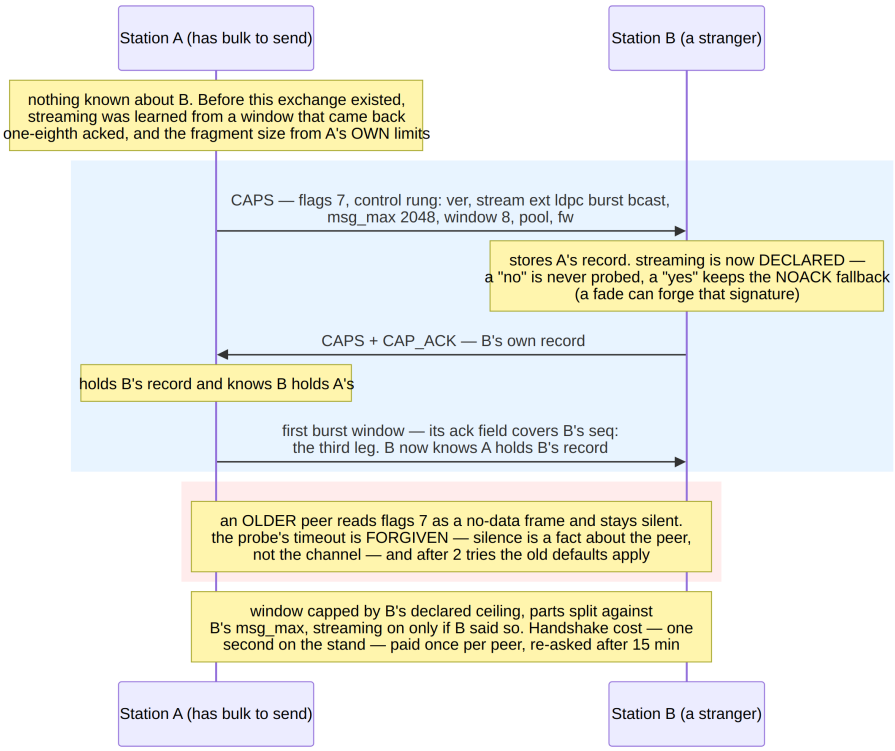


Figure 47: The three-leg capability handshake. The record rides the third impossible flag combination, so an older peer reads it as a no-data frame and simply does not answer — which is itself the answer, and is forgiven rather than charged to the rate controller.

A declaration outranks every inference. A peer that says it cannot stream is never streamed to — the retry counter is set to a value no clean exchange can reset, where the old NOACK verdict was deliberately re-probed. A peer that says it *can* keeps the NOACK fallback anyway, because a deep fade can still forge the one-fragment-acked signature and the channel remains entitled to its opinion. And the declared streaming bit comes from the operator's `burst_stream` knob, *not* from the PHY's `receive_burst` hook: the streaming receivers (demoapp and the firmware) leave that hook NULL by design and walk streams through `rxs_continue_burst`, and the first version — which masked the bit on the hook — sent the smoke test from 5 frames to 29, every window transmitted per-frame against a peer that streams perfectly well.

The record's numbers are the receiver's, not the sender's. The window is capped at engage by the peer's declared ceiling, and the consoles split files against the smaller of the two message limits: the board reports its own `ST_MSG_MAX` in the USB INFO reply and the peer's record in every status frame.

With the peer's limit known, the boards' limit could grow to meet it: `ST_MSG_MAX` is 2048 on the radio firmware (the station moved wholesale to DTCM for the 21 kB the pool grew — Section 12.4's placement rules made that a two-line change), and one USB frame now carries one whole message. The same 68 kB file became 34 parts of eleven fragments, each part one streamed window:

	before	after
shape	290 parts × 2 fragments	34 parts × 8-fragment windows
acknowledgments	one per 260 B	one per ∼1.6 kB
transfer time	∼25 min	**13.0 min, byte-exact**

The first full-size run also handed over one more calibration. Thirteen of 34 windows timed out — every one a forgiven first miss, so nothing broke — and the diagnostic stream showed why: after a 13.4-second streamed window the receiver answers only after its end-of-stream commit, which arrives as one ∼2.5 s `rxs_push` (Section 12.8 measured the stall) plus carrier release. The reply timer's RFC-6298 smoothing had learned the *common* overhead — 70 ms, from the short exchanges — and a bimodal distribution is exactly what exponential smoothing cannot hold. The stall is physics, not latency to learn, so it gets a fixed allowance: a reply awaited after a streamed window adds `RTO_STREAM_COMMIT_S` (5 s) to the budget, and streamed exchanges no longer feed the estimator at all. The re-run: 4 timeouts instead of 13, zero retransmissions, and the same 780-second transfer time — the allowance only lengthens waits for acks that are late anyway.

Two more levers, and which one mattered

The 13-minute transfer still ran at 66% of the rung-12 channel rate, and the per-part rhythm said where the rest was: each 2 kB part went out as *three* acknowledgment cycles — a window of eight, a window of two, and a lone tail fragment. Two levers were measured separately, on the same file and the same wire:

	time	rate	shape
baseline: 2 kB parts, window 8	780 s	87 B/s	3 acks per part
window-aligned 1600 B parts	669 s	102 B/s	1 ack per part
window 16, parts *not* re-aligned	695 s	98 B/s	tail penalty
aligned 3.2 kB parts, window 16	**596 s**	**114 B/s**	1 ack per part

The first lever costs nothing: one console part is one station message, so splitting parts at `win_max` × 200 minus the envelope head — the peer's declared window, from the handshake — makes every message an exact multiple of the fragment and every part exactly one acknowledgment. The third row is why this matters more than window size, and it is the row worth remembering: a *bigger* window measured *slower*, because the streamer refuses short tails inside windows (deliberately — a stream is uniform blocks), so a misaligned part paid a whole extra ack cycle for its 27-byte tail. Alignment first, capacity second.

The second lever buys the remainder with RAM nobody was using. `BURST_STREAM_MAX` rose to 16 (a 25.3 s window, still under the 30 s single-acknowledgment exposure cap) and messages to 3328 bytes — and the memory to pay for it came from SRAM4, the one region every earlier budget had left untouched. Its placement logic inverts the usual instinct: SRAM4 sits in the D3 domain behind two bus bridges, the *slowest* RAM on the part, which makes it exactly right for the station struct (∼58 kB with the larger pool) — touched at frame cadence from the main loop and never from the ISR. Opening it took a linker region, a zero loop in startup (no RCC gate exists for CPU access), and hoisting the 16-block stream buffer to file scope so the linker could park its 33 kB in D2 by name. After the move: DTCM 56 of 128 kB, AXI 496 of 512, D2 293 of 295, SRAM4 58 of 64 — and ITCM's 64 kB still the untouched reserve.

The final configuration moves the 68 kB file in 9.9 minutes at 87% of the raw channel rate, byte-exact, with three timeouts across the hundred-odd windows of the measurement campaign and zero retransmissions; what remains above the wire rate is preambles, turnaround and carrier-sense gaps. One operational note carried into the documentation: the *first* bulk transfer to a stranger still splits conservatively at window 8, because the handshake is triggered by that very transfer — its record arrives too late to size it. A small bulk item sent first primes the record when the difference matters.

The ceilings become operator knobs

The record's window field had been the compile-time buffer limit, and the protocol had carried a related embarrassment since its first revision: `config rung_ceiling` — the operator's cap on the rate ladder — was documented in the USB key list and implemented nowhere. The modem's configuration handler simply had no case for it, and nothing ever noticed, because nothing ever read the setting.

Both ceilings are now real, runtime, and declared. `config win_max` bounds the streamed window this station accepts *and* sends (byte 4 of the record); `config rung_ceiling` bounds the rung it transmits at or requests (byte 9, encoded as ceiling+1 so the zero an older firmware sends still reads as "unspecified"). Changing either while a peer is known pushes a refreshed record at once, rather than waiting for the next transfer to re-handshake.

Enforcement sits in the station, deliberately outside the rate controller: every transmit-rung decision passes through a clamp — the controller's choice, bounded by our own ceiling and by the fastest rung the peer declared — because clamping *inside* the controller would poison its per-rung offset learning with rungs it never actually chose. Requests are clamped the same way, and the window at engage is the minimum of the operator window, the buffer limit, our declared ceiling and the peer's.

The verification ran over USB, no rebuild involved: board B was given `win_max 4` and `rung_ceiling 9` from its console, and board A — whose controller wanted rung 12 on a clean wire — sent fourteen frames at rung 9 and none above, every window engage reading `4 of 16 (ceiling)`, with A's status line showing the declared limits: `window 4, rung ceiling 9 (handshake complete)`. The suite holds the same case synthetically: a peer declaring window 4 and ceiling 9 against a controller that wants the top rung, asserted on both the window and every frame's rung.

12.15 Demonstration Infrastructure

The C stack runs end-to-end in an interactive demonstration (`demoapp/`): a virtual channel daemon exposes two station devices and a configuration device over Unix sockets, moves 12 kHz `int16` audio in paced ticks, and applies propagation delay, AWGN sized against the delivered-signal RMS, one-pole Rayleigh fading, and half-duplex muting; an optional audio mode plays each station's receive signal on one stereo channel with the sound card as the pacing clock, making the protocol audible (the EXTREME bootstrap's 21-second warble adapting up to sub-second QAM16 chirps). Two console station applications (three streaming receivers each, one per mode, over the shared ring) provide messaging, multi-part file transfer over the burst protocol, channel statistics, the diagnostic event stream, and the oscillator fine-tune command. A scripted smoke test drives the whole stack — text plus a 5 000-byte file across the impaired channel, byte-compared on arrival — at 25× time scale; the protocol clocks itself from received samples, so behaviour

is identical at any scale. The stop-and-wait inefficiency, the burst protocol's window bugs, the carrier-sense threshold and the rung-0 fallback were all found by this infrastructure rather than by inspection.

Figure 48 traces a file transfer through it. Two decisions in that sequence are worth isolating.

The file is deflated *whole* before being split into parts, not per part: a 14162-byte configuration file becomes 5015 bytes on air (2.82×), and compressing each 3 kB part separately would throw most of that away. Compression lives in the application rather than in `cport/`, which keeps the C port dependency-free — an MCU build swaps zlib for miniz or heatshrink and the wire format is plain DEFLATE either way.

The transfer also *negotiates before it commits*. The transmit rung decays by one step per 90 s of peer silence (Section 7), so a transfer started after a quiet spell would otherwise open at a rung the link has never been tested at: measured, a 37.0 s frame at rung 4 immediately before the next exchange settled at rung 12 and 5.1 s. A sub-second control frame is answered, and an answered frame is what moves the ladder, so the probe goes first and the bulk queue follows behind it. The staleness test must ask whether the peer has *ever* reported rather than subtracting from the initial timestamp — the sentinel is -10^9, and doing the arithmetic anyway reported that the peer had last been heard from 1000000058 seconds ago. It is the same rule the link layer already follows for sequence numbers, where every value 0–7 is legitimate and "nothing yet" must be its own state.

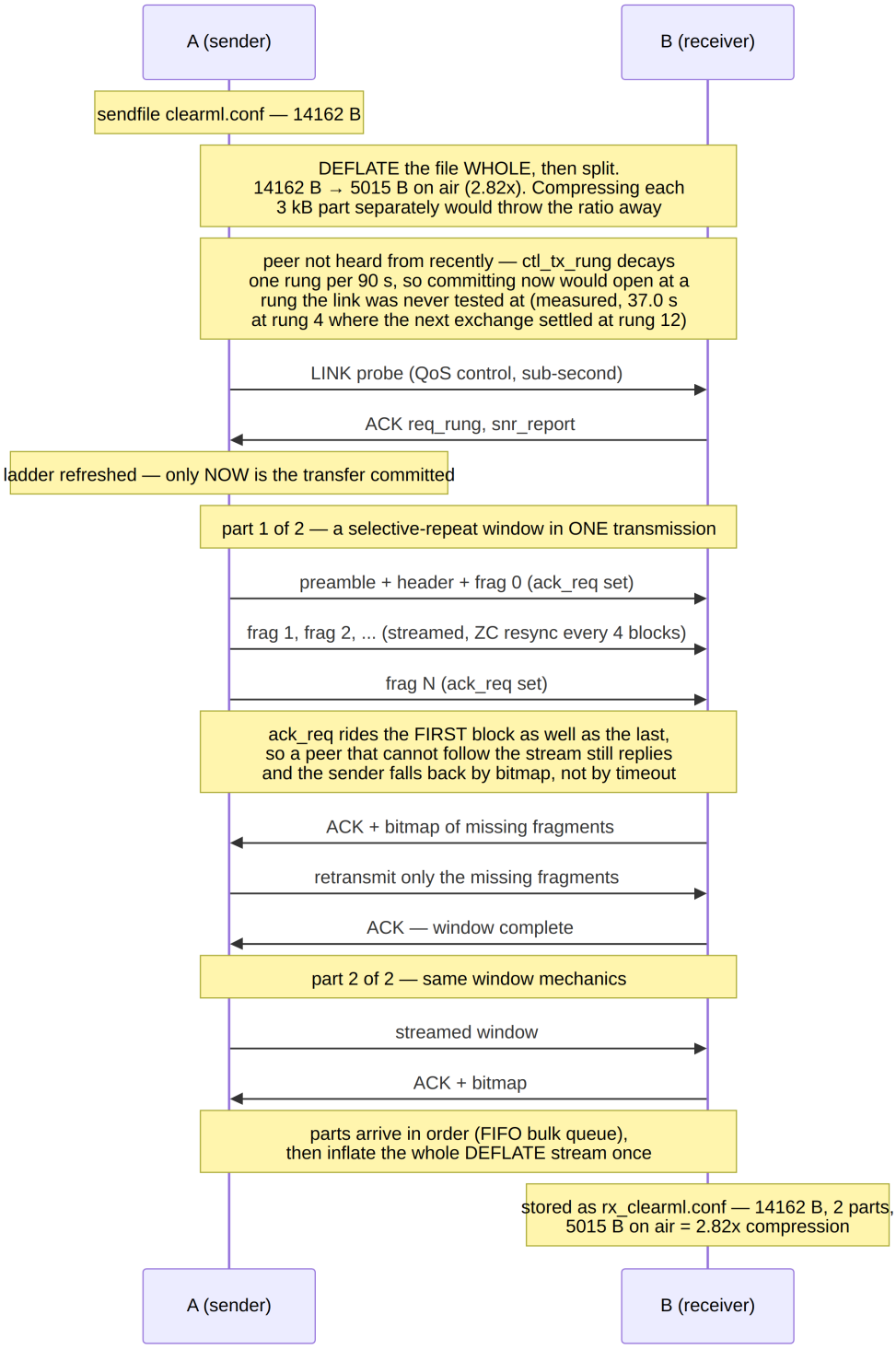


Figure 48: A file transfer: whole-file compression, a link probe before the transfer is committed, then selective-repeat windows carried one per transmission.

12.16 Writing the Interfaces Down

The board stopped being a program and became a *device* once three consumers appeared for it — the C console, the Python console and the host library — each implementing the same protocol from the same header. `cport/src/usb_proto.h` is normative and always was, but a header states layouts, not rules: that the IN endpoint must be drained

rather than reset (Section 12.6), that an empty CONFIG payload is a query, that broadcast chunks are paced against a field in the status frame, that a file part should be window aligned or pay an extra acknowledgment (Section 12.14). Every one of those was found by measurement in this chapter, and each lived only in the firmware and the commit log.

Four documents now carry them: the transport (framing, frame types, payload layouts, configuration keys), the modem's command/event model as a host application sees it — submit, pace, configure, stay alive, and the envelopes that ride inside — the channel drivers behind the single 12 kHz audio interface everything else speaks, and the two consoles' commands together with the behaviours that cannot be guessed from them. They are narrative, not normative: where a document and the code disagree, the code is right and the document is the bug.

Which makes them worth checking rather than trusting, so all twelve were audited against the source. The result is a fair picture of how documentation rots. Nothing had gone wrong in the parts of the system that stopped changing; the drift was entirely where the measurement campaign had moved most recently. The PHY document still described the broadcast command as one transfer of at most 1022 bytes, true until the host began feeding it in chunks (Section 12.13). The link document had the station in DTCM holding 2 kB messages — three revisions behind SRAM4, 3328 bytes and the 114 B/s those bought (Section 12.14). The fixed-point document described the integer SNR estimator's raw output but not the measured output map since bolted on top of it (Section 12.11), which is the half with a behavioural contract. The source-layout table predated `cport/` entirely. And the freshly written diagnostic-event count said nineteen where the enumeration has seventeen — a number that can only be got right by counting the code, which is the argument for auditing documentation rather than for writing it more carefully.

12.17 Speaking KISS, and Where That Belongs

Everything above assumes software written for this modem. The question of what it would take to run *existing* packet-radio software over it has one cheap answer — KISS, the framing every amateur TNC has spoken since 1987 — and one design decision that is more interesting than the protocol.

KISS is trivial: SLIP-style framing, one type byte, six configuration commands. Implementing it in the firmware would take an afternoon, and would be a mistake. The board already speaks a protocol that carries the rung, the SNR, the peer's declared capabilities, the die temperature, the broadcast pacing signal and a diagnostic stream; a KISS TNC has no way to express any of it, because it was designed for modems that have nothing to say. Adding KISS to the firmware would duplicate the transport, add a fourth wire format to keep in step across the C, Python and board twins, and present an adaptive modem as a dumb one. So the translation is a host program (`host/kiss_bridge.py`), it presents either a TCP port or a pty for `kissattach`, and `cport/` does not change at all.

The mapping is a KISS data frame to one station message, boundaries preserved, over the ARQ that already exists; a second mode sends each frame as one non-ARQ transmission, which is what AX.25's connectionless UI frames actually describe. What the bridge does *not* do is carry connected-mode AX.25: two retransmission engines with independent timers, one adapting the rung underneath the other, is a layering accident rather than a feature.

The constraint that shapes everything is air time, not frame size. The station's own

estimator gives, for a full AX.25 frame:

Table 27: Air time for one AX.25 frame, from `estimate_air_time`. The rung, not the byte count, decides whether a frame is sendable.

Payload	rung 0	rung 4	rung 8	rung 10	rung 12
32 B	48.3 s	3.2 s	1.3 s	1.1 s	1.0 s
128 B	147.0 s	9.7 s	2.9 s	2.2 s	1.7 s
200 B	221.0 s	14.6 s	4.1 s	3.0 s	2.2 s
256 B	278.6 s	18.4 s	5.1 s	3.6 s	2.6 s

A 256-byte frame is 2.6 s at the top of the ladder and four and a half minutes at the bottom — past every carrier-sense constant the station has, and past the 45 s fragment ceiling that already governs burst ARQ. The bridge therefore admits frames by *air time at the current rung* and refuses the rest with the reason, rather than keying a transmitter for four minutes; `paclen` 200 is the operating point, because a full frame is then ≈216 bytes, inside the 255-byte single-frame payload cap, and 3.0 s at rung 10.

One implementation detail earned its place in the tests. The first version imported the model's `estimate_air_time` directly — and would not start, because that import pulls in NumPy and SciPy while a bridge runs in a virtual environment that has `pyusb` and nothing else. The air-time model is now a generated table, and the test asserts against the real function that it is never *optimistic* (worst case 0.43 s pessimistic, across every rung and every length), so an admission decision cannot be wrongly permissive. It is the same rule the C port follows for compression: the dependency lives in the tool, not in the thing being reused.

Measured end to end on the two boards: a 48-byte AX.25 UI frame — shifted callsign addresses, control `0x03`, PID `0xF0`, an APRS position — entered a TCP KISS client, crossed the wire, and left the peer's bridge byte-identical 3.4 s later, 1.4 s of that being air at rung 12.

The same path through the operating system's own AX.25 stack took two further defects to open, and both are worth the space because neither was visible from the radio.

The first was self-inflicted. The air-time gate refuses what will not fit the current rung — and an idle link decays to rung 0, where it admits 28 bytes, less than an AX.25 header. So every frame was refused, nothing was transmitted, and the ladder whose recovery would have fixed it never moved: a gate that deadlocks what it protects. The repair is the one the firmware already uses for a broadcast of unknown rung (Section 12.13): hold the frame, probe with something small, release when the rung improves, give up loudly after a deadline. The negotiation belongs where the payload waits, on either side of the USB cable.

The second was an interoperability detail that no amount of measuring the link would have found. Frames entered the bridge and never reached the air, while the boards' own counters showed a frame crossing on *some* attempts — because Linux's KISS line discipline probes its TNC for checksum support: the first frame after an attach carries the SMACK flag, bit 7 of the type byte, the second carries FLEX, bit 5, each with a CRC appended, after which it settles into plain KISS for good. Those bits share the nibble that carries the port number. Reading that nibble as a port turned the first two frames of every attach into traffic for "port 8" and "port 2", which the bridge dropped as foreign; only the third frame — plain KISS — survived, which is exactly the intermittency the

board counters showed. Three rounds of on-target measurement confirmed the air path each time and could not have found this, because the loss was one layer above the radio. What found it was a single line of the bridge's own log, `frame for port 8 dropped`, once the operator pasted it. That is the argument for the other half of the repair: refusals, holds, drops and every frame in each direction are now printed unconditionally, because a tool that discards work silently makes its own diagnosis impossible.

With both fixed, the stack works as a stack: two bridges, two `kissattach`, `ax0` and `ax1`, and a `beacon` on one interface arrives on the other — including the first frame after an attach, the case that had never once worked. A second, 51-byte frame followed in under 15 s. Thirty-one checks cover the codec and every mapping decision without hardware, the two probe flavours among them.

12.18 Carrying IP, and Whose Timers Are Wrong

Running IP over the KISS path fails for a reason that has nothing to do with the radio: the kernel's AX.25 stack is not network-namespace aware, and its receive handler discards frames arriving on a device outside the initial namespace. With the far interface in a namespace, its device counted 82 received packets while the stack reported `0 ICMP messages received` — the frames crossed the link correctly and died one layer above it. That rules out containers as well, a container's isolation being a network namespace, and leaves a second machine.

A TUN device has no such restriction, so `host/ip_tun.py` carries IP packets directly as station messages and both ends run on one host. It works: pings answered with a 3.8 s round trip at rung 12, no loss, and a 4 kB TCP transfer arriving byte-identical. The interesting part is what the throughput did on the way there.

Table 28: The same 4 kB TCP transfer under four queueing policies. Every figure is measured on the two-board stand; the link itself was unchanged throughout.

Policy	Ping	TCP	Shape
read, drop what will not fit	—	8.8 B/s	12 application drops per kB, a 77 s tail
queue 8 packets	117 s	44.2 B/s	bufferbloat
queue by time (1–2 packets)	3.8 s	21.1 B/s	a 130 s stall mid-transfer
fq_codel tuned to the link	**3.8 s**	**43.7 B/s**	smooth, 948 B per 10 s

Three defects, and all three are the same defect.

The first was mine: the tunnel read every packet and discarded what would not fit. That destroys the interface queue and hands TCP a loss for every packet of its opening burst — Linux starts with ten — which TCP reads as congestion and answers with a collapsed window and a multi-second retransmit timeout. Not reading while the board is busy leaves the packets where the kernel can apply backpressure properly, and took the transfer from 8.8 to 44 B/s with no drops at all.

The second was the queue depth. Eight packets was chosen as "about a minute of air at rung 12" and then met rung 4, where every packet is four times slower: round trips of 117 s, all of it queueing. A queue has to be sized in *time*, and the useful depth here is tiny — this link's bandwidth-delay product is about 440 bytes, less than one packet — so anything beyond a couple of packets buys latency and no throughput.

The third was invisible, and is the one worth carrying away. Shortening the queue fixed the latency and cost half the throughput, with a 130-second hole in the middle of a transfer and *zero drops recorded at either end*. Zero drops was the clue: the losses were the qdisc's. The kernel's default is `fq_codel` with a 5 ms target and a 100 ms interval, sized for links where a packet takes microseconds; here a packet is 8.6 *seconds* of air, so CoDel sees a standing queue the instant anything is buffered and drops it. It had been doing so through every measurement, and no queue length could have fixed it. Tuned to the link — a target that must exceed one packet's serialisation time, 10.3 s at rung 12 against 8.6 s of air — CoDel does what it is for, dropping only when delay persists, and both properties arrive together.

The pattern is the same one Section 12.6.1 describes on the USB side, where every host timeout was shorter than the device's 2283 ms decode. A link whose frames take seconds invalidates timing constants three orders of magnitude away from it, in code nobody thinks of as part of the radio: a USB write deadline, a TCP retransmit timer, an active queue manager's target delay. Each was chosen sensibly for the hardware its author had in mind, and each is wrong here in the same direction.

12.19 Whose State Is It: a Review, and What It Found

The operator noticed the boards were busy with nothing attached to drive them. They were: one station was keying an acknowledgment-only frame every second — 117 frames in 108 s — which its peer decoded and, correctly, never answered. It had been doing so unattended.

The cause is worth stating precisely, because six more defects turned out to share its shape. `poll_tx` begins with an early-out: return unless something is owed, queued, or a capability exchange is pending. A flag called `caps_kick` bypasses that gate. But `caps_probe_wanted()` declines on its *first* line for a peer whose record is already held — so the block that would have sent the probe, and which is the only place that clears the kick, never ran. Execution fell through to the message path, found no fragment, and emitted a no-data frame. Forever.

One bug of that shape justifies looking for others, so the station and link layers were reviewed along four axes: state that is set and never cleared, timing and arithmetic, bounds and memory, and ARQ protocol logic. Table 29 is what came back, after each finding was re-derived against the code and, where possible, measured.

Table 29: Seven defects, and the observation that identifies each. None of them announced itself: the counters were healthy, or said the opposite.

Defect	Symptom	What it really was
Burst state outliving its message	bytes from before the message pool, encoded and transmitted	`msg_data()` returned `pool[-1]` for a released slot
Rung request decaying per *call*	ladder collapses to EXTREME after a fade	a getter that mutates, called twice per frame
Acks on CAPS / burst frames discarded	a retransmission per frame the peer sends, uncounted	three branches return before the ARQ retire
Broadcast hold not cleared	the second `bcast` announces and never keys	a flag whose only consumer a legitimate path skips
Burst falling through to stop-and-wait	duplicate transmission; station busy forever	two paths owning one message
Full store acknowledging a drop	sender believes a lost message arrived	`done` set before the delivery attempt
Window air time understated 23 %	a 37 s transmission under a 30 s cap	a model standing in for the builder

Three observations generalise beyond this codebase.

The recurring shape is ownership. Almost every entry is state with a single consumer that some other legitimate path walks around, or state owned by two things at once. The repairs are correspondingly structural rather than local: the burst state is now dropped as a unit by one function instead of by clearing a flag; an engaged burst owns its message and the stop-and-wait path will not touch it; the silence decay is charged against a marker so that asking twice cannot decay twice; and the window's air time is asked of `tx_burst_len` rather than modelled from a subtraction that was never quite right.

The failures were silent by construction. A no-data frame is never replied to, so the stuck transmitter provoked nothing. A dropped message was acknowledged, so the sender saw success. The doubled decay moved a number that no counter reports. This is why the entry point was an operator noticing a blinking LED rather than any instrument in the system — and why the repairs that matter most are the ones that make a discarded frame say so.

A sanitizer only sees the path a test walks. The out-of-bounds read has existed for as long as the burst path, and every suite runs under AddressSanitizer and UndefinedBehaviorSanitizer — which do catch it: with the guard removed again as a check, UBSan reports the index at every message size tried, the default 256 and the shipping 3328 alike. What hid it was that no test had ever released a message while a burst was still engaged, so nothing reached `msg_data()` with a freed slot. The instrument was sharp enough; the scenario was missing, and naming it took a review rather than a tool.

The geometry matters too, but less than it first appears, and in a narrower way: at the default size that underflow lands *inside* the neighbouring controller struct, so only UBSan objects and ASan sees nothing; at the shipping sizes the same expression reads about 3 kB outside the object and both fire. The suites therefore now run at the shipping geometries as well as the default (`make sanitize-ship`), because a gate should exercise the configuration that ships — but the lesson that generalises is the first one.

12.20 From Simulation to a Radio

The demonstration apps talk to a *driver* over a socket rather than to hardware, which turns out to be the right seam for real radio: an SDR driver (`demoapp/sdr_driver.py`) presenting the same devices puts the entire C stack on the air with nothing in `cport/` changed. It reproduces the SSB model of Section 9 sample by sample — audio → analytic signal (the receiver's own 63-tap Hilbert design) → interpolation to the SDR rate → I/Q, and back, where taking the real part of the decimated baseband *is* a product detector. Tuning the radio to the suppressed carrier makes the two stations' LO error appear as exactly the audio-band CFO the modem already tracks. The DSP costs 4.4 % of one core to transmit and 3.1 % to receive per audio-second at 2.4 Msps, so a laptop runs it comfortably in real time.

Figure 49 shows the arrangement that needs no radio at all: the driver cross-connects two stations through the entire SSB chain — Hilbert transform, interpolation, int8 quantisation, decimation and product detection — so every line of the signal path is exercised, and the console apps above it cannot tell the difference. That is what the regression test uses.

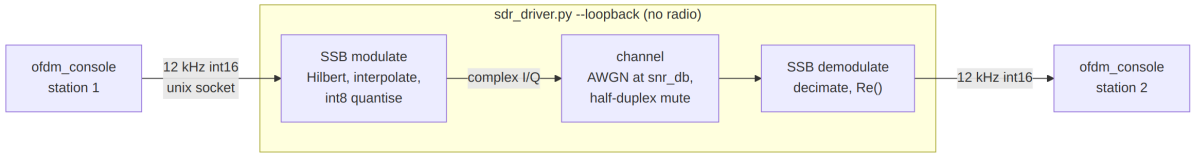


Figure 49: Hardware-free loopback: two stations cross-connected through the full SSB modulation and demodulation chain.

Bringing this up on a HackRF One produced four corrections that no amount of loopback testing would have found: the local oscillator must be *offset tuned* (with it on the carrier, the DC leakage lands inside the 3.5 kHz passband and the recovered audio was 98 % DC — the device has neither hardware DC correction nor AGC); the aggregate gain control is inert and the LNA/VGA/AMP stages must be driven individually (worth 32 dB of working level); `writeStream` only *queues* samples, so switching back to receive when it returns truncated the tail of every frame; and a killed process left the radio *keyed* until a receive stream was opened.

12.20.1 Instrumented Measurements

The two measurements that follow share the arrangement of Figure 50: a tinySA Ultra on USB, used in both directions. As an analyser it measures what the radio transmits; as a source — through its calibration output, which is a reference tone at a documented level — it measures what the receiver makes of a known input. Both are scripted, so the numbers below are reproducible rather than read off a screen.

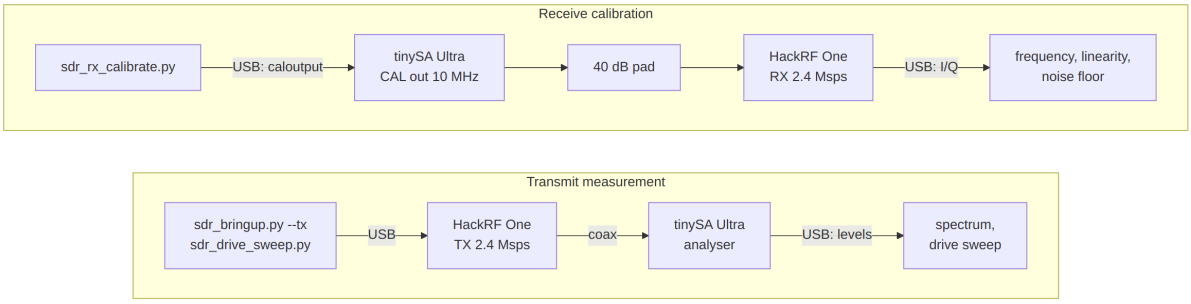


Figure 50: Instrumented bench, both directions: as an analyser the tinySA measures what the radio transmits; as a source its calibration output drives the receiver through a 40 dB pad.

12.20.2 The Transmitted Spectrum

Figure 51 is the waveform measured on a tinySA Ultra wired to the antenna port, captured by `demoapp/tinysa.py` (which sweeps with the transmitter off, keys it, and max-holds several sweeps). The occupied band matches the design: a −3 dB width of 2.42 kHz against the 2.1 kHz the numerology predicts, once the 3 kHz resolution bandwidth is allowed for, sitting exactly where the carrier and 300–2400 Hz audio band put it. That single measurement validates the absolute tuning, the offset-tuning arithmetic, the drive scaling into the 8-bit DAC and the single-sideband modulation at once — an error in any of them shows up as splatter or a mirror image. Visible 50 kHz below the signal is the LO leakage at −35 dBc, the price of the offset tuning that keeps DC out of the *receive* passband.

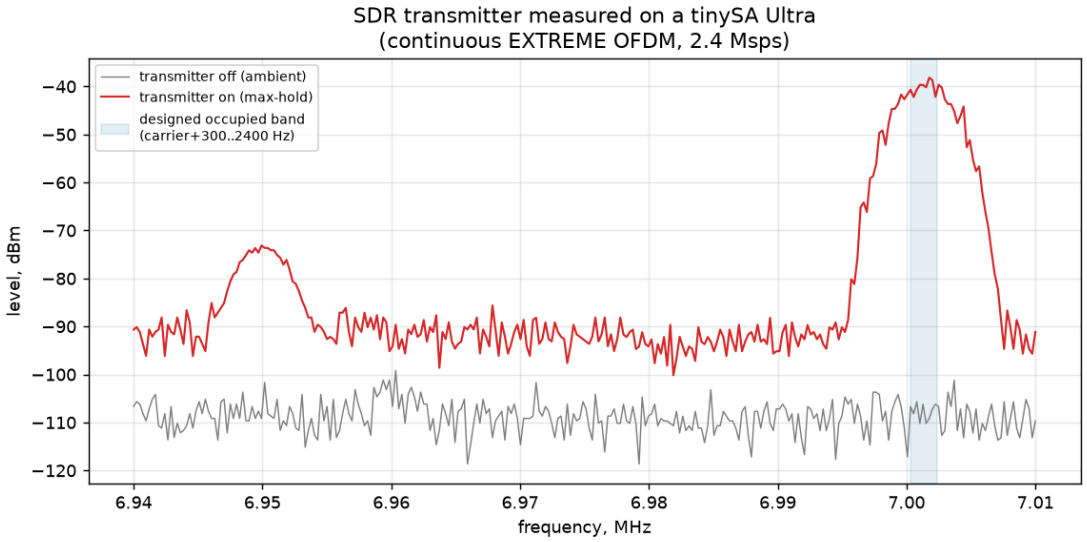


Figure 51: The transmitter measured on a tinySA Ultra: continuous EXTREME OFDM at 2.4 Msps, carrier 7.000 MHz, 3 kHz RBW. The shaded band is the designed occupancy (carrier + 300. . . 2400 Hz); the spur at 6.95 MHz is the offset-tuned local oscillator.

12.20.3 How Hard the Radio Can Be Driven

Figure 52 sweeps the transmit gain and measures, at each step, the fundamental, the shoulders 5–10 kHz outside the occupied band, and the harmonics. It should be read from

the middle outwards: below VGA 16 the reported shoulders are the *analyser's* noise floor rather than the transmitter's, because the fundamental is only −58 dBm. Above VGA 32 the degradation is real compression, provoked by the waveform's ≈8 dB PAPR — at full drive the shoulders reach −22 dBc and the second harmonic −38 dBc, both worse than the −43 dBc usually demanded of spurious emissions.

The asymmetry between the two impairments is the practical result: a low-pass filter removes the harmonics but does nothing about the shoulders, which are spectral regrowth a few kilohertz from the carrier and inside any filter's passband. The usable window with this waveform is therefore about VGA 16–32, i.e. −42 to −24 dBm, and real power has to come from a *linear* external amplifier rather than from winding this one up — the same linearity requirement the OFDM waveform imposes everywhere else in the chain.

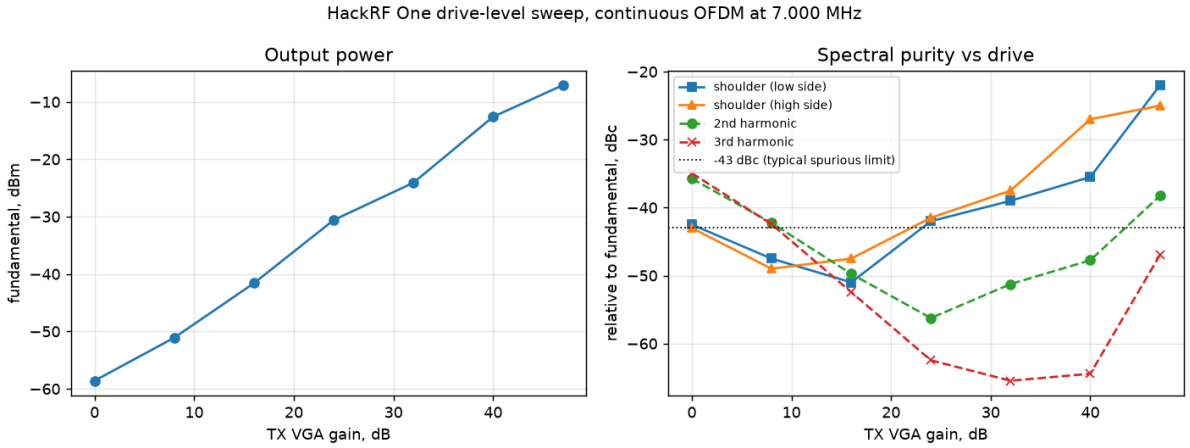


Figure 52: Drive-level sweep on the HackRF One (`demoapp/sdr_drive_sweep.py`): output power, and spectral purity relative to the fundamental, against transmit gain. Points below VGA 16 are limited by the analyser's noise floor, not by the transmitter.

12.20.4 Decoding Off the Air

A half-duplex radio cannot receive its own transmission, so the last question — does a receiver actually *decode* this over a real RF path? — needs a separate transmitter. Figure 53 shows the one used: a laptop's sound card plays a generated file into an AM modulator carried by an external generator at 7.000 MHz, and the result reaches the SDR through a 40 dB pad.

AM demands a different detector, for a reason worth stating because it is a property of the waveform rather than an inconvenience. The receiver is single-sideband: it tunes to a *suppressed* carrier and takes the real part of the baseband. Present it with a double-sideband signal and the lower sideband folds onto the upper one; worse, any frequency error between the generator and the SDR multiplies the audio by a slow cosine, splitting every subcarrier in two. The test therefore recovers the carrier that AM conveniently provides, derotates by it so the carrier sits at exactly 0 Hz and zero phase, and only then takes the real part. That is classical synchronous detection, and it is why the decoded CFO in Table 30 reads zero: the carrier lock has already absorbed the error the modem would otherwise estimate.

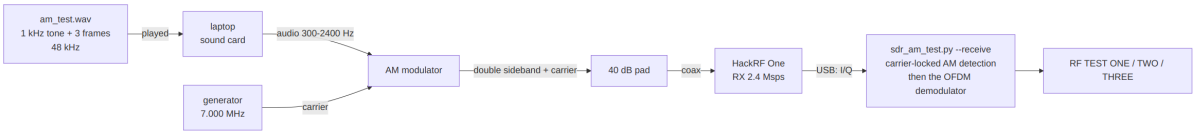


Figure 53: Off-air receive test: an external AM transmitter feeds the SDR through a 40 dB pad.

Table 30: All three transmitted frames recovered over a 7 MHz RF path (`results/am_rx_offair.wav`).

Frame	Payload	Length	SNR
NORMAL QPSK 1/2	`RF TEST ONE`	1.0 s	+13.6 dB
NORMAL BPSK 1/3	`RF TEST TWO`	1.8 s	+14.4 dB
ROBUST BPSK 1/3	`RF TEST THREE`	7.3 s	+8.5 dB

Figure 55 is what those frames look like as the demodulator received them. The analysis FFT is 128 bins at 12 kHz, which is exactly the modem's own subcarrier grid, so every row of the waterfall is one OFDM bin and the structure of Section 2 is directly legible: the Newman tone comb striping the preamble, then the header, then the data block filling 300–2400 Hz. The two NORMAL frames last about a second; the ROBUST frame occupies 7.35 s for the same 27 bytes, which is what 16× tiling costs and buys. The region boundaries are not fitted — they are computed from the decoded header's own modulation, code rate and length fields, and land on the measured burst to within 15 ms.

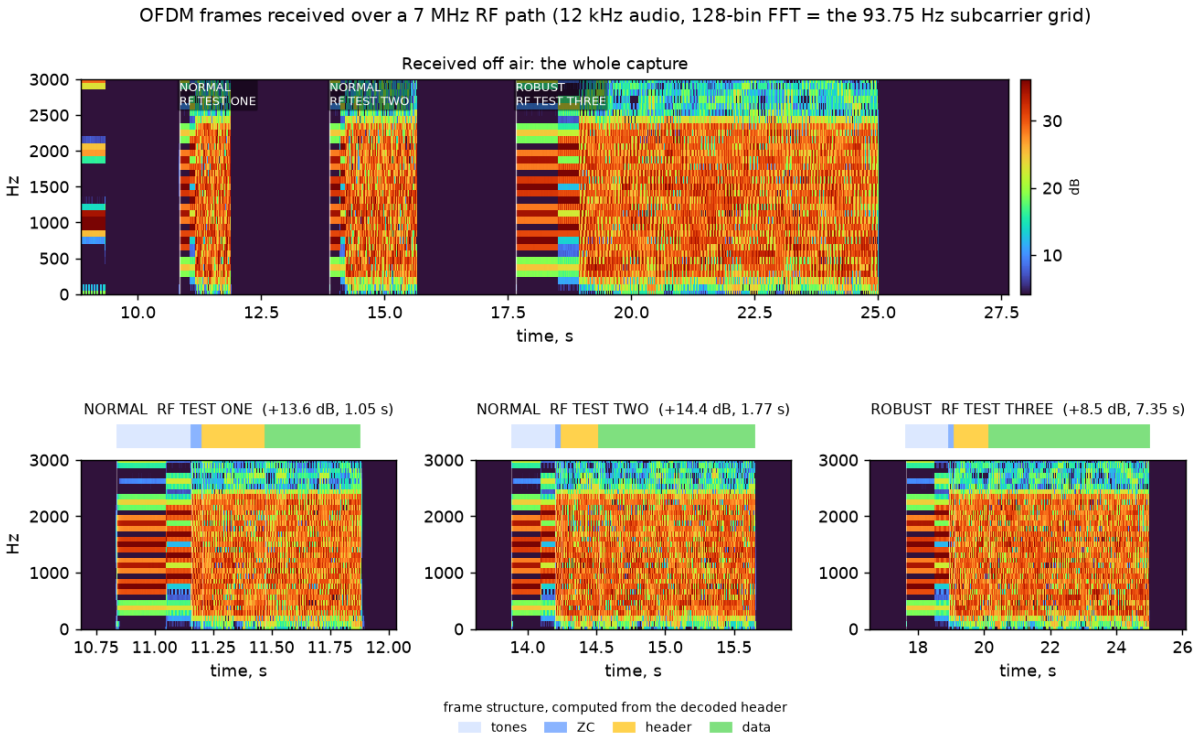


Figure 54: Frames received over the air (`demoapp/sdr_waterfall.py` on `results/am_rx_offair.wav`): the whole capture above, each frame close up below with its preamble, header and data spans marked.

Figure 55 shows those frames as the demodulator received them. The analysis FFT is 128 bins at 12 kHz, exactly the modem's own subcarrier grid, so each row is one OFDM bin and the frame structure of Section 2 is directly legible: the Newman tone comb lighting only a few bins, the Zadoff–Chu symbol closing the preamble, then the header and the data block filling 300–2400 Hz. The spans above each waterfall are computed rather than fitted — from the mode's tone-field geometry and the decoded header's own modulation, code rate and length — and they land on the measured burst to within 15 ms. The comb's end is measurable independently: bin occupancy sits at 0.17 through the tone fields and jumps to 0.9 at 0.321 s, against a predicted 0.320 s. It is worth noting that the ZC symbol occupies all 23 carriers, so on a waterfall it resembles data even though it belongs to the preamble — which is why it is drawn as its own span.

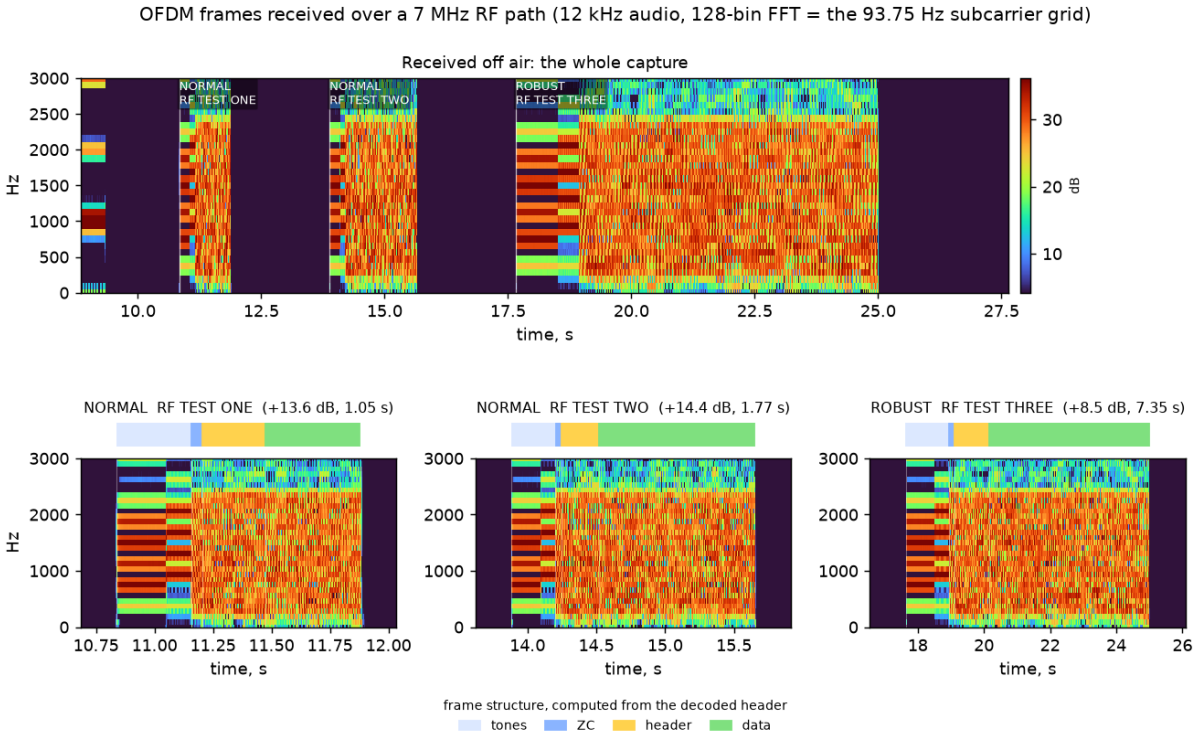


Figure 55: Frames received over the air (`demoapp/sdr_waterfall.py` on `results/am_rx_offair.wav`): the whole capture above, each frame below with its measured structure. The ROBUST frame carries the same 27 bytes as its NORMAL neighbours and occupies 7.35 s doing it — what 16× tiling costs, and buys.

Every frame transmitted was recovered, across two link modes and two modulations, with the mode identified from the preamble root alone — the mechanism the whole adaptive ladder depends on, here exercised over real RF rather than in simulation. ROBUST reports the lowest SNR by design: its 16× tiling spreads the same energy over a longer symbol, so the per-symbol estimate falls while the decoded margin grows.

Two honest qualifications. The transmitter is an AM modulator rather than the SDR of the preceding sections, so this validates the receive chain over RF and not a complete transmit-to-receive loop, which still wants a second radio. And a first, coarser scan of the same recording found only two of the three frames, because it advanced four seconds after each hit while the demodulator returns one frame per window — the miss was in the measurement harness, not the link, which is exactly the kind of result worth re-checking

before blaming the hardware.

12.21 Voice at 250 Bits per Second

Every payload so far has been data. Voice was never plausible on this link: the classic amateur speech codecs start around 700 bit/s (Codec2 700C, what FreeDV uses on HF), and rung 12 carries a measured 102 B/s = 816 bit/s of *goodput*, leaving no room for framing, acknowledgements or a fade. Neural codecs have since moved the floor. LSCodec-25 Hz [9] (MIT licensed) emits 25 tokens per second, each a 10-bit index into a 1024-entry codebook: **250 bit/s**, or 31.25 B/s. That is a quarter of what the top rung carries.

Most of what follows is evaluation: the measurements decide what a voice mode would cost and which of its problems are fixable. The last part is not. §12.21.12 puts the codec on the air between the two boards — push a button, speak, and the far station writes a `.wav` — and that exercise cost exactly one firmware change, for a reason no host-side measurement could have found.

12.21.1 The link is not the constraint

At 31.25 B/s one station message (`ST_MSG_MAX` 3328) holds **106 seconds of speech**, and the streaming burst machinery already carries it: a 200-byte fragment is exactly 160 tokens, or 6.4 s. Table 31 gives the delivery ratio against the ladder.

Table 31: Speech delivered per second of air, from the streamed goodput of each rung. Real time is the 1.00× line; the modem's headline sensitivity, EXTREME at −17.9 dB, is unusable for voice by three orders of magnitude.

Rung	Sens. (dB)	Goodput	106 s memo	Speech/s of air
0 (EXTREME)	−17.9	1.0 B/s	3465 s	0.03×
4 (NORMAL)	−7.6	14.5 B/s	230 s	0.46×
7	−5.3	43.5 B/s	77 s	1.39×
10	+0.7	86.9 B/s	38 s	2.78×
12	+4.7	**130 B/s**	**26 s**	**4.17×**

Voice runs faster than real time from rung 7 upward and is store-and-forward below it. The cost is that the link's whole sensitivity advantage is forfeited: at the EXTREME bootstrap rung a 106-second memo needs an hour of air. A voice mode is a good-conditions mode.

Compute is likewise not the constraint, provided it stays on the host. Measured on the demonstration laptop (Intel i5-8300H, CPU only), the 17.5 M-parameter encoder runs at 14.3× real time on *one* core and 31× on four; the 31.3 M-parameter vocoder decodes at roughly 3×. Encoding a 106-second memo takes 3.4–7.4 s against 26 s of air, so it overlaps keying. Memory is the awkward figure: a one-shot encode costs about 14 MB per second of audio, so a full-length memo peaks near 2 GB. None of this belongs on the STM32 — 48 M parameters of PyTorch is four orders of magnitude beyond the part — which places a voice mode exactly where KISS (§12.17) and the IP tunnel (§12.18) already are: a host program above an unchanged modem.

12.21.2 Two findings that are not about the bitrate

The codec is domain-sensitive, and fails abruptly. Reconstruction quality was measured as ASR word error rate against ground-truth transcripts, using a CTC recogniser (`wav2vec2-large-960h-lv60-self`) rather than an attention-decoder model, because the latter hallucinates on damaged audio and scores it as total loss. The result depends on the corpus far more than on anything in the codec.

Table 32: The same weights, prompt scheme and recogniser on two corpora, 16 and 15 utterances. LibriTTS is what LSCodec was trained on; LibriSpeech is ordinary read speech from the same books, recorded and processed differently.

	LibriTTS *(in domain)*	**LibriSpeech** *(out of domain)*	**Paper** *(reported)*
WER, original audio	8.28 %	1.01 %	1.20 %
WER, after 250 bit/s	10.83 %	22.64 %	5.46 %
Added by codec	**+2.6 pts**	**+21.6 pts**	+4.3 pts
Speaker similarity	0.875	0.864	0.945
Utterances destroyed	**0 of 15**	**3 of 16**	—

On its own corpus the codec costs 2.6 points of word error and loses nothing outright, reproducing the paper's 4.3. Move one corpus away and it costs 21.6 points and destroys three utterances in sixteen — not degraded, destroyed: normal duration, normal energy, and a transcript reading `Eatly Waty Ing Ening` where the sentence was "it is obviously unnecessary for us to point out…". The same failures appear under a second recogniser of a different architecture, so this is the audio and not a transcription artefact.

That failure mode is the important one for a radio link, because it is **invisible to every layer below the ear**. The bitstream is valid, the CRC passes, the ARQ is satisfied, the byte count is right, and the far operator hears confident nonsense. Shack audio — a real microphone, a real room, SSB processing — sits further from LibriTTS than LibriSpeech does, so the right planning figure is the middle column, not the paper's.

Four candidate explanations were tested and eliminated before the corpus one was accepted: a substituted prompt encoder (the official WavLM checkpoint's distribution link returns `AuthenticationFailed`; using the authentic weights improved speaker similarity $0.793 \rightarrow 0.845$ but moved word error not at all), a weak recogniser, a poor prompt choice (giving each utterance *itself* as prompt changed nothing), and a feature-layer indexing error. The pipeline reproduces the paper on the paper's own corpus, which is what licenses the comparison.

The prompt, not the bitstream, carries the speaker. LSCodec is speaker-*decoupled* by design — that is how it reaches 250 bit/s. The tokens carry what was said; a reference prompt supplies who says it. Decoding one byte-identical token stream against two different prompts gives two different people:

Table 33: Speaker-embedding cosine similarity (Resemblyzer). The token stream is identical across each speaker's rows; only the prompt differs.

Reconstruction	**vs A**	**vs B**	**Outcome**
A, matched prompt	0.845	0.429	still A
A, crossed prompt	0.428	0.673	**became B**
B, matched prompt	0.478	0.778	still B
B, crossed prompt	0.760	0.570	**became A**
two unrelated real speakers	0.431	—	baseline

A crossed reconstruction scores 0.428 against its own speaker, landing on the 0.431 baseline between two unrelated people. A wrong prompt does not colour the voice, it substitutes a stranger's. For amateur service that is a station-identification question before it is an audio-quality one, and it is architectural: no amount of retraining makes a speaker-decoupled token carry timbre.

12.21.3 What enrolment costs

The prompt is not a fixed-size speaker vector but a *time series* of WavLM-Large layer-6 features, $T \times 1024$ float32 at $\sim$50 Hz — 200 kB per second of prompt audio, some 6400$\times$ the speech bitrate. Sending features over the air is out of the question. Sending prompt *audio* is not, because the quality saturates almost immediately.

Table 34: Enrolment prompt length against reconstruction speaker similarity, and what each length would cost to transfer. Air time at the measured rung-12 goodput of 102 B/s.

Prompt	**SECS**	**fp16 features**	**Opus 16 kbit/s**	**Air**
0.5 s	0.751	48 kB	1.0 kB	10 s
1 s	0.826	98 kB	2.0 kB	20 s
2 s	**0.855**	198 kB	**4.0 kB**	**40 s**
4 s	0.858	398 kB	8.0 kB	78 s
16 s	0.869	1592 kB	32 kB	314 s

Two seconds buys 98 % of the benefit; eight times more data adds 0.014. So enrolment is a one-off exchange of about 4 kB — roughly 40 s of air, or two station messages — after which every transmission from that station is pure 250 bit/s payload. Two constraints follow. The prompt cannot be sent *through* LSCodec, which strips exactly the information it needs to carry, so a voice mode needs a second, conventional codec for enrolment alone. And each station must hold WavLM-Large (1.2 GB) locally to turn received prompt audio into features; the cached result is about 198 kB per correspondent.

12.21.4 How the prompt reaches the vocoder

The prompt is not a speaker vector but a *time series*: WavLM-Large [5] layer-6 features, $T' \times 1024$ at $\sim$50 Hz, with $T' = \lfloor (n - 400)/320 \rfloor + 1$ for n input samples — 99 frames for the 2 s enrolment above. Figure 56 shows where it enters.

Two details cost time to establish and are recorded here because neither is discoverable from the configuration. The vocoder for LSCodec-25 Hz was trained at 50 Hz, so the token

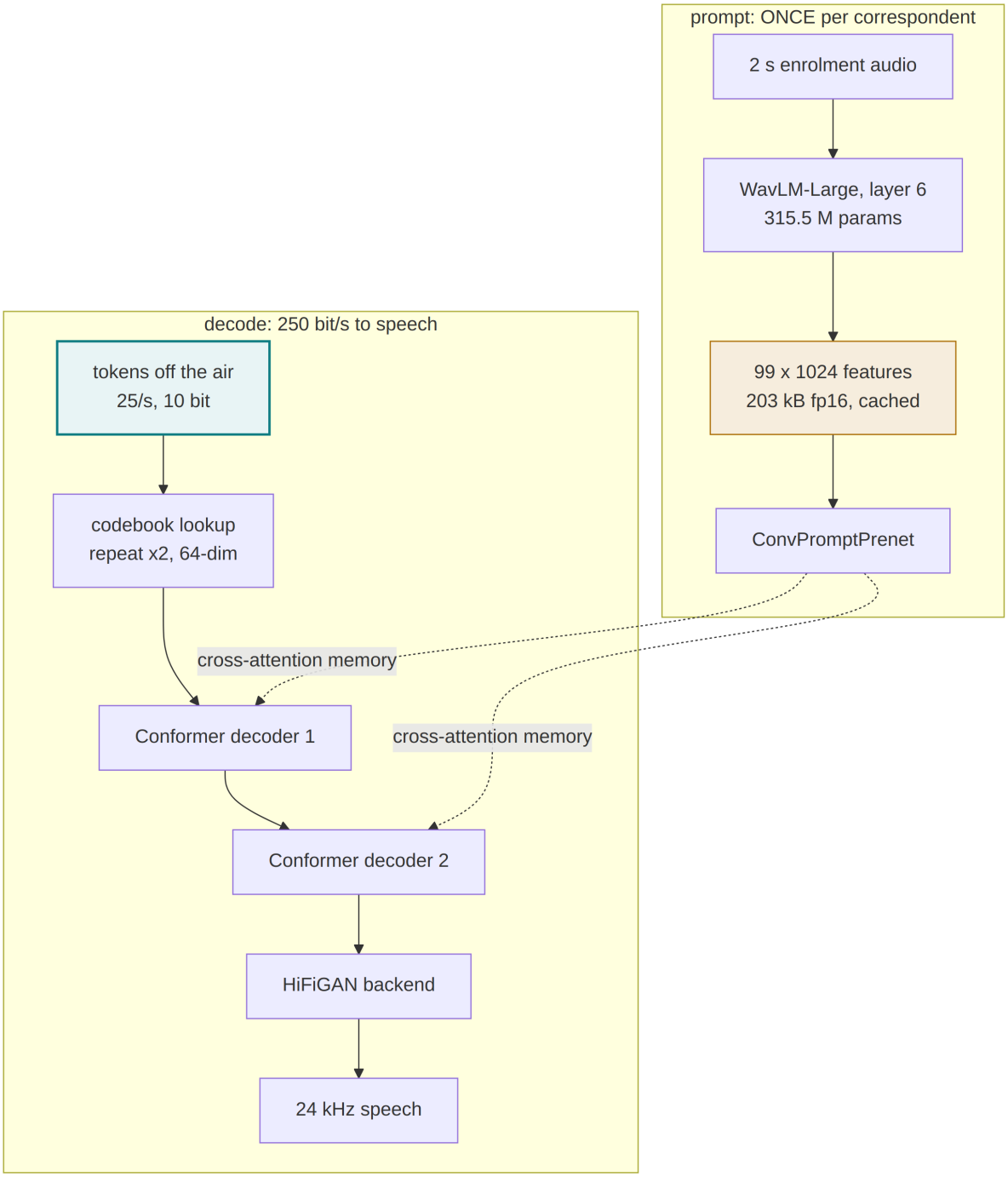


Figure 56: The decode path and the prompt path. The prompt is **cross-attention memory** for both Conformer stages, not a concatenated or pooled input — which is why its length is independent of the content's, and why 99 prompt frames can condition a 640-frame message.

sequence must be `repeat_interleaved` by two before the codebook lookup; the only trace of this is the `repeat_input_tokens` flag, and omitting it yields speech at half speed. And the frontend's own docstring describes the prompt as a "mel-spectrogram prompt" — inherited text from CTX-vec2wav. It is not mel: it is 1024-dimensional WavLM, and feeding mel produces confident nonsense with no error raised.

12.21.5 Can the prompt be made smaller?

At 200 kB per second of prompt audio, the enrolment is ∼6400 times the speech bitrate, which is what makes it the awkward part of the design. Three families of compression were measured against a single metric: mean absolute log-mel distance between the decode under a degraded prompt and the decode under the exact one, on the same token stream. The scale is anchored at both ends — 0 dB is the correct prompt, and **8.88 dB is a different real speaker's prompt**, i.e. the point at which the speaker is simply wrong.

Pruning is ruled out by the distribution before any decode: only 3.3 % of values fall below 0.1 in magnitude, and *zero* of the 1024 channels have low variance. There is nothing to prune. The distribution is also heavily outlier-dominated — kurtosis 52.6, range $-35.6 \ldots +57.4$ while 98 % of values lie within ± 8, with per-channel offsets whose median magnitude is 0.74 and maximum 47 — which decides how quantisation must be applied.

Table 35: Prompt compression, all on the same 400-byte token stream. Per-channel affine quantisation is essential: at 4 bits it costs 0.35 dB, while a single global scale costs 2.36 dB for the same storage.

Prompt representation	Bytes	mel dB	Verdict
fp32 (reference)	406 kB	0.00	—
fp16 (as cached)	203 kB	0.00	free
int8, per-channel	**105 kB**	**0.03**	**free; use this**
int6, per-channel	80 kB	0.09	free
int4, per-channel	55 kB	0.35	small
int2, per-channel	29 kB	1.50	audible
int8, per-tensor	101 kB	0.16	5× worse, same size
int4, per-tensor	51 kB	2.36	ruined by outliers
low-rank $k = 20$	47 kB	0.89	loses to int4
every 2nd frame	102 kB	1.31	loses to int4

The interesting failure is vector quantisation, because it is the only scheme that could plausibly beat sending audio. A *shared* codebook — trained once and shipped with the software, so only indices cross the air — was fitted on 280 s of speech from other speakers, with the test speaker held out.

Table 36: Vector-quantising the prompt against a shared codebook. Every variant is at or past the point where the speaker is simply wrong (8.88 dB), including the ones that would beat 2 s of Opus audio on size.

Scheme	Bytes	vs 4 kB Opus	mel dB
int4 per-channel (best scalar)	54.8 kB	13× worse	0.35
PQ, 32 blocks × 256	3168 B	about equal	7.87
PQ, 8 blocks × 256	792 B	5× better	9.88
full-frame VQ, $K = 1024$	123 B	33× better	10.34
a different real speaker	—	—	*8.88*

This is a rate limit, not a codebook-quality limit, and the distinction matters because only the latter would be fixed by more training data. The k-means converged (identical at 12 and 60 iterations) with zero dead centroids out of 8192. In feature space, PQ at 0.25 bits per dimension reaches a relative L_2 error of 0.593, against 0.072 for int4 at 4 bits per dimension — and 0.764 for simply substituting the per-channel mean. Three-quarters of a bit short of that baseline is not a codebook that needs more speakers. The features are also not redundant enough to escape through rank: 90 % of the variance still needs 40 of 99 components, and adjacent frames correlate only 0.79.

The reason is worth stating plainly, because it generalises past this codec. **WavLM features are an expansion of the audio, not a compression of it**: 2 s of speech is 32 000 samples and 101 376 feature values, 3.2× more data. Opus reaches 4 kB by coding the audio, which is a far more redundant signal. Coding the expanded representation instead cannot win, and the conclusion is that **the enrolment goes over the air as audio and every station keeps WavLM locally**. Quantisation is a storage decision — int8 per-channel, 105 kB per correspondent — not an architectural one.

A secondary observation, unexpected and useful for anyone tempted by this: the vocoder tolerates a *valid but wrong* prompt better than a *correct but corrupted* one. Quantised prompts land off the feature manifold and decode worse than a constant.

12.21.6 Borrowing a purpose-built speaker encoder

If the prompt cannot be compressed, the alternative is to replace it. Several codecs represent the speaker as a small *time-invariant* code rather than a sequence: TiCodec [16] splits speech into time-dependent and time-invariant parts and quantises the latter into eight discrete tokens per utterance; FreeCodec [20] takes an ECAPA-TDNN speaker embedding and quantises it with group VQ, eight groups against a 1024-entry codebook — **80 bits, ten bytes, for the whole utterance**; FACodec, the codec inside NaturalSpeech 3 [14], factorises speech into content, prosody, acoustic detail and timbre, extracting timbre with a Transformer encoder. A recent review [10] maps the space.

Ten bytes instead of four kilobytes would remove the enrolment exchange *and* the 1.2 GB WavLM dependency, so it is worth knowing how far apart the representations are. FACodec's timbre extractor was run against LSCodec's vocoder directly: 69 utterances from 23 LibriTTS test-clean speakers, 2 s each, both representations computed, and a ridge adapter fitted from the 256-dimensional timbre vector to the 1024-dimensional prompt, evaluated on speakers it never saw.

Table 37: FACodec timbre driving LSCodec's vocoder. Two losses are separable, and the first one bounds the whole approach.

Prompt fed to the vocoder	**mel dB**
true prompt, full sequence (reference)	0.00
true prompt's own mean, broadcast — the CEILING	**3.64**
FACodec timbre → ridge → broadcast	7.18
a different real speaker, full sequence	8.88
corpus mean, timbre ignored (null)	10.16

The adapter carries real information — held-out cosine +0.36 and R^2 +0.12 on the speaker-discriminative component, and its decode beats both the null and a different real speaker — but it lands at 7.18 dB against a ceiling of 3.64. That ceiling is the finding. It is what *any* time-invariant code scores with this vocoder, even given the true prompt's own mean, because the two Conformer stages cross-attend over 99 frames and genuinely use their temporal detail. A constant memory throws that away before the speaker question is even reached.

So the answer is not a better adapter, and not more speakers: the vocoder's prompt path would have to be retrained to consume a global code. That is precisely what TiCodec and FreeCodec do from scratch, and why eight tokens suffice for them. The adapter figure is a floor — 69 samples fitting a 257×1024 map is data-starved, and a nonlinear head with thousands of speakers would do better — but the ceiling comes from the architecture and no amount of data moves it.

12.21.7 A proposed voice mode

Figure 57 is what the measurements above support. Every component is host-side; `cport/` does not change, and neither does the wire format.

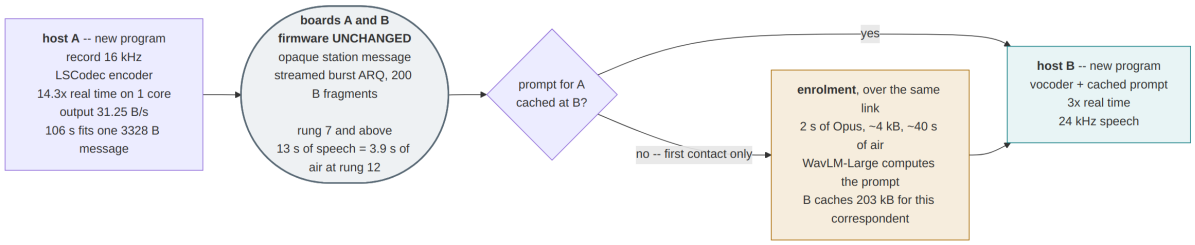


Figure 57: The proposed voice mode. The tokens are an opaque payload of exactly the size the streaming burst machinery already carries, and enrolment is a one-off exchange on the same link.

The design commits to four things the measurements decided. Enrolment is *audio*, not features, and happens once per correspondent (§12.21.5). The cached prompt is int8 per-channel, 105 kB. The mode is available from rung 7 upward and is store-and-forward below it, which forfeits the link's sensitivity advantage and is not negotiable (§12.21). And the speaker's identity travels in the prompt, never in the payload, so a station that has not enrolled a peer cannot render that peer's voice at all — which is a feature for the identification question and a constraint on the protocol.

Two paths lead off this design, and they differ in what they cost the wire. Fine-tuning the **vocoder** on in-domain or Russian speech changes no bitstream: a peer with

stock weights still decodes, a peer with tuned weights decodes better, and there is no capability bit. Adopting a **global speaker token** in place of the prompt would collapse enrolment to about ten bytes, but requires retraining the prompt path and therefore matching models at both ends — and, per LSCodec's own comparison, would trade away some of the speaker similarity that made this codec attractive: LSCodec reaches 0.853 SECS at 0.25 kbps where TiCodec reaches 0.714 at 0.75 kbps, precisely because it spends no bits on identity.

12.21.8 A non-English trial

The corpus finding predicts trouble for any operator not speaking English audiobook prose, so a real recording was tried: a Russian sentence (*Zhili byli ded da baba, i byla u nikh kurochka ryaba*), phone-grade, 4.16 s. It encoded without complaint — 101 tokens, 127 bytes, 244 bit/s, 0.97 s of air at rung 12, 84 of 1024 codebook entries used.

With the speaker's own prompt, 7 of 10 words survived the round trip and character error rate went from 10.6 % to 21.3 %; a native listener judged the result intelligible and recognisable, which is the verdict that matters and which the error rate understates. The three words lost were the content words — *zhili*, *kurochka*, *ryaba* — while the function words came through: at 250 bit/s an English-trained codebook has nothing to spend on unfamiliar phonetics. Substituting an English speaker's prompt degraded it further (34.0 % CER) and dropped the recogniser's confidence that the audio was Russian at all from 96 % to 37 %. The prompt is carrying language as well as timbre, so for a non-English operator a wrong prompt costs the voice *and* the accent.

12.21.9 Streaming, and the one-shot trap

Everything above encodes a whole utterance at once, which costs memory that grows with its length — 124 s of speech peaked at **7.7 GB** of resident set — and rules out both continuous operation and pipelining the encode against the transmission. Chunking the input looked like it would cost quality. It does not; it *improves* it, and the reason is worth recording because it inverted the measurement three times before it was understood.

Chunked and un-chunked encodings of the same audio disagree on 25 % of their tokens even with generous overlap, and no amount of extra context closes the gap. The cause is not the receptive field. Every normalisation in the encoder is `Fp32GroupNorm(1, dim)` — group norm with a *single* group, which normalises over channels **and time** jointly. Each layer therefore subtracts a mean and divides by a deviation computed over the whole input, so the effective receptive field is the entire utterance, not the 43 aggregator frames (1.72 s) the convolutions imply. A chunk has different statistics from the utterance containing it, and every token inside it shifts, not just those near a seam: at 40+ frames from any boundary, agreement was 27.5 % — as wrong as at the boundary itself.

The consequence is that the encoder implicitly assumes utterance-length input, and *it was trained on utterances of a few seconds.* Encoding half a minute in one pass is as out-of-distribution as any other mismatch, which is what the numbers say once they are taken against the original audio rather than against a one-shot baseline.

Table 38: Log-mel distance to the ORIGINAL audio, not to another decode. Beyond about 15 s the one-shot encode degrades and chunking does not — at 30 s it is 4.5 dB better. Earlier figures in this project measured chunking against a one-shot baseline that was itself drifting.

Utterance	One-shot	6 s chunks	12 s chunks
13 s	6.14 dB	**6.04 dB**	6.16 dB
30 s	14.16 dB	**9.63 dB**	9.61 dB
60 s	13.42 dB	**11.24 dB**	13.11 dB

A tempting alternative — freezing the group-norm statistics from a warm-up window so chunks become independent — makes streaming *exact* (100.0 % token agreement against its own un-chunked result, against 75 % with live statistics) and costs **4.32 dB**, because those statistics are what make the encoder invariant to recording level: the input deviation of the first layer varies ±22 % across utterances, and layers 9–11 by 39–71 %. It is rejected, and it makes a wider point — **token agreement is a misleading metric**. Live-statistic chunking disagrees on a quarter of its tokens and costs 2 dB; frozen statistics agree perfectly with themselves and cost twice that. The tokens a chunked encoder gets "wrong" are ones the vocoder is insensitive to.

12.21.10 Latency: the chunk is not the knob

A chunk is never encoded alone. The encoder is given a window that extends past the chunk on both sides, and the tokens belonging to that extra audio are then thrown away — neighbouring chunks emit them. Figure 58 shows the arrangement. The two sides are not symmetric in cost, and that asymmetry is the whole result:

- **Left context** is audio that has *already been spoken*. It is sitting in the buffer when the chunk is encoded, so including it delays nothing. It is free, and should be generous.
- **Right context** is audio that has *not been spoken yet*. To include it the encoder must wait for it, so every millisecond of it is a millisecond of latency — and it is the only part of the window that is.

Algorithmic latency is therefore the chunk length plus the right context alone, not the window width. Treating the two separately turns a 7.3 s pipeline into a 1.3 s one for two tenths of a decibel.

Table 39: Chunk and context against latency, measured on two minutes of Russian narration. Chunk length is nearly irrelevant; the last 0.32 s of lookahead is a cliff.

Chunk	Left ctx	Right ctx	Latency	mel dB
6 s	1.28 s	1.28 s	7.28 s	7.20
2 s	1.28 s	1.28 s	3.28 s	7.29
2 s	1.28 s	0.88 s	2.88 s	7.33
2 s	1.28 s	0.32 s	2.32 s	7.33
1 s	**1.28 s**	**0.32 s**	**1.32 s**	**7.40**
2 s	1.28 s	**0.00 s**	2.00 s	**16.76**

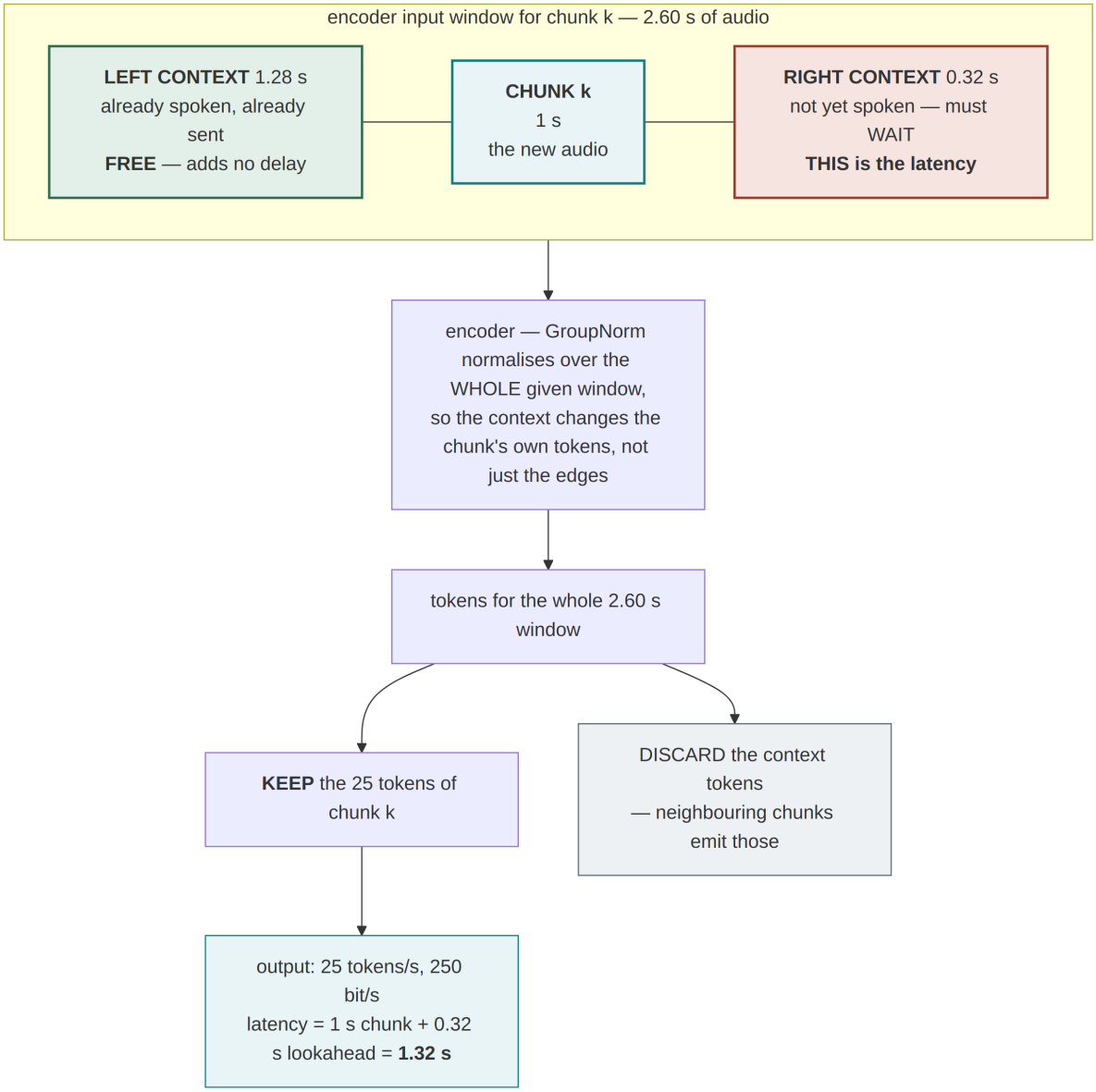


Figure 58: One chunk's encoder window. Only the right context costs latency; the left context is audio already in hand. Because the group norm covers the whole window (§12.21.9), the context changes the chunk's own tokens rather than merely repairing its edges.

Going from 6 s chunks to 1 s costs 0.20 dB. Removing the final 0.32 s of lookahead costs **9.4 dB** and the decode collapses. Quality never depended on chunk length — only on having *some* right context — so the latency was reducible almost for free, and a listener could not distinguish the 1 s and 6 s outputs at all. The settled configuration is **1 s chunks, 1.28 s of left context and 0.32 s of lookahead**, for 1.32 s of algorithmic latency, flat memory, and no change to the model, the bitstream or the protocol.

Chunking costs compute in overlap: each 1 s chunk re-encodes 1.6 s of context around it, and encoding falls from 33× real time at 6 s chunks to 17× at 1 s. Decoding is unaffected at 5× and is now the slower half — the figure a receiver must keep ahead of.

12.21.11 Five minutes of Russian, and an English accent

Streamed at the settled configuration, five minutes of continuous narration gives the end-to-end picture for both languages. The method matters here, because the figures are not comparable with the decode-against-decode distances used earlier in this section.

Both sources are public-domain LibriVox recordings of a single speaker: *The Heart of a Mystery* (chapter 1) for English, and Zinaida Gippius's *Chortova kukla* (chapter 1) for Russian. Each was converted to 16 kHz mono and cut to exactly 300 s starting 25–30 s in, past the LibriVox announcement, so the material is continuous narration rather than the concatenated utterances used for the receptive-field work. The prompt is the first 2 s of the same recording — the speaker enrolling with the opening of their own transmission — and decoding runs in blocks of 150 tokens against that one cached prompt.

The distance is the mean absolute difference of 80-band log-mel spectrograms (1024-point FFT, 256 hop), with the 24 kHz decoder output resampled to 16 kHz so both sides are measured at the same rate, over the frames the two share. **It is taken against the ORIGINAL AUDIO**, so unlike the earlier tables it includes the codec's own reconstruction loss and its absolute value is not comparable with them; only the comparison *within* Table 40 is meaningful. Language identification is Whisper `small`'s `detect_language` applied to eight equal segments of each file, with the returned probabilities averaged.

Table 40: Five minutes of long-form narration through the streaming configuration. The byte count is identical because the token rate is fixed at 25 Hz: language changes what the tokens say, never how many there are.

	Bytes	Rate	Encode	mel vs original
English (in domain)	9373	249.9 bit/s	33×	5.84 dB
Russian	9373	249.9 bit/s	28×	7.42 dB

Russian costs +1.58 dB, far less than the domain penalty on English read speech in Table 32, and a native listener judged the reconstruction satisfactory. **Five minutes of speech is 9373 bytes: three station messages, and 1.5 minutes of air at rung 12.**

The listener also reported something the distance metric does not capture — the decoded speaker has an English accent. It is measurable, and it locates a defect precisely:

Table 41: Language identification, averaged over eight segments of each five-minute file. English is unchanged; Russian loses 38 points and gains English.

	Original	**Decoded at 250 bit/s**
Russian narration	ru 99 %	ru 61 %, en 12 %, uk 6 %
English narration	en 100 %	en 100 %

An earlier result had attributed language leakage to the prompt: giving a Russian utterance an English speaker's prompt dropped its identified language from 96 % to 37 %. That is real, but it is not the whole mechanism, because here *the prompt is Russian* — the first two seconds of the same narration — and the accent persists. There are two independent sources, and the second is the vocoder itself, whose Conformer stages and HiFiGAN backend were trained entirely on English audiobooks and have learned how English phonemes are realised.

That is the useful part. The accent lives in the one component that can be fine-tuned **without touching the bitstream** (§12.21.7): a peer running stock weights still decodes, a peer with Russian-tuned weights decodes better, and no capability bit or protocol revision is involved. Language-identification confidence gives that fine-tune a ready objective — 61 % → 99 % — with the unchanged English column as its regression check.

12.21.12 Putting it on the air

The measurements above are all host-side. What follows is the codec actually running between the two boards: a browser front end (`host/webvoice/`) that drives one board as transmitter and the other as receiver, holds a push-to-talk button, and writes what the far station decodes to a `.wav`. It works, and how it got there is more instructive than the fact that it does: two host defects, one firmware policy that was right for files and wrong for speech, and a fourth problem that turned out not to be a defect at all — after two rounds of measurement had confirmed that it was.

Transport is **broadcast** (`PKT_TYP_BCAST`, ptype 15), not the ARQ path. For speech a late repeat is worth less than a gap: the same 20 s utterance costs 2.0–5.2 s per message through the acknowledged path against 0.9 s broadcast, and a retransmission that arrives after the listener has moved on is pure harm. Nothing is acknowledged and nothing is repeated.

Bit alignment, and why the first group always worked. The token packer emits ten bits each and zero-pads to a byte boundary, so 50 tokens is 500 bits in 63 bytes — four pad bits. Independently packed groups are then concatenated by the receiver, which has no framing inside the stream and unpacks the whole thing as one bit sequence. Every group after the first is therefore shifted by four bits, then eight, then twelve. The symptom is diagnostic in hindsight: the first second of every transmission decoded perfectly and everything after it was noise, which reads like a channel that collapses and is arithmetic. The fix is to emit only multiples of **four** tokens — 40 bits is exactly 5 bytes — and carry the remainder into the next group. Verified 300/300 tokens bit-exact end to end.

The latency was the group rule, not the codec. §12.21.10 settles the codec at 1.32 s. The radio then added four more, and none of it was DSP. The firmware keys

only *full* groups while chunks are still arriving, which is correct for a file — a starved tail group would go out without EOS, and the true last frame must carry it — but a group is $\texttt{BC\_GROUP} \times (\texttt{BC\_FRAME} - 2) = 96$ bytes, and at 250 bit/s that is 3.1 s of talking that must accumulate before the first carrier goes up.

This is the one place `cport/` did change. A real-time flag (ptype bit 5, `BC_RT`) keys what is queued instead of waiting. It has three parts and the middle one is the trap: the group must shrink to a *power of two* matching the frames actually keyed, because the SYNC descriptor carries $\log_2(\text{group})$ and a receiver told to expect four blocks when three were sent walks into a block that never comes and goes deaf for a block time — the same failure that once made streamed bursts loop. The flag is opt-in and off for every other broadcast, since a smaller group is a preamble per fewer frames and trades air time for latency.

A second, duller second and a half was the machine learning framework: the first encode cost 826 ms against 47 ms for every one after, all of it lazy allocation and kernel selection landing on the first transmission. Warming the graphs at startup moves it where nobody is listening. Together: first audio at the far station about **3.6 s** after the talker starts, against roughly 6 s before.

The receiver has to decode as it listens. An algorithmic latency of 1.32 s is a property of the codec, not of a system, and the first working receiver threw all of it away: it accumulated the whole transmission and vocoded once at end-of-stream, so the listener heard the first word only after the last one had been spoken. On a 200 s transmission that is ∼202 s of latency for word one — a store-and-forward voicemail wearing a real-time codec's clothes.

Decoding a chunk at a time as the tokens land is not free, because the vocoder normalises over time exactly as the encoder does (§12.21.9): a chunk decoded in isolation drifts from what the one-shot decode would have produced. Left context repairs it, and — this is the part that makes streaming decode viable at all — left context costs *no latency*, because those tokens have already arrived and already been played. Lookahead would cost latency, and the decoder uses none.

Table 42: Chunked vocoding against the one-shot decode of the same received bytes, 1 s decode steps. Log-spectral L_1; a waveform metric cannot measure this at all (below).

Left context	**log-spec L_1**	
0.0 s	0.3169	chunks decoded in isolation
1.0 s	0.2341	
2.0 s	**0.2240**	the knee, and what ships
4.0 s	0.2174	twice the context for 3 %

Measured end to end on the two-board stand, the assembled output measures 0.2044 against a one-shot decode of the identical received bytes, at identical length, slightly better than the offline sweep predicted; a 200 s transmission delivers 325 frames, zero lost, byte-exact, with end-to-end lag flat at 0.000 s per second of speech. The first-audio figures moved twice more after this paragraph was first written and are reported with the profiles below, where they belong.

One methodological note, because it cost a measurement. The first version of this sweep used waveform RMS error and reported ≈ 1.0 at every context length — which

reads as "chunking destroys the audio" and is meaningless. A vocoder is *generative*: two decodes of the same tokens differ in phase and agree perceptually, so a waveform metric is uncorrelated with quality by construction. The encoder study in §12.21.9 already used a spectral metric for this reason; the right move was to read it first.

Profiles: the decode latency becomes a knob, and grows a cost axis. The left context above is fixed at 2 s and free; two other quantities are not, and together they define a decode *profile*: the **chunk** (how much speech each step emits — the listener waits for the first one to fill) and the **lookahead** (tokens beyond the chunk fed to the vocoder and discarded — each second of it is a second of added latency). Both improve quality. What forced the profile table through three revisions is that they also cost *compute*, and a step must finish inside its own chunk's duration or the decoder falls behind speech without bound.

Table 43: Chunk and lookahead against quality and cost. L_1 is log-spectral distance to the one-shot decode of the same received bytes; duty is measured step time over the chunk duration, on both devices the host supports.

Chunk	Ahead	Latency	L_1	GPU	CPU	
1 s	0 s	1 s	0.2240	15 %	77 %	`live`
1 s	1 s	2 s	0.1236	19 %	**100 %**	`balanced` (GPU)
2 s	1 s	3 s	0.1113	12 %	63 %	`balanced` (CPU)
2 s	2 s	4 s	0.1108	15 %	83 %	dominated
3 s	2 s	5 s	0.0941	11 %	60 %	`quality`

Three lessons are in this table, each paid for. First, the 1 s/1 s shape was shipped as `balanced` after measuring only its quality; at exactly 100 % CPU duty it could not keep up, and the processor it saturated starved the browser's main-thread capture into delivering silence — the cost column exists because the quality column alone chose an infeasible configuration. Second, on the CPU a *bigger chunk is both better and cheaper*: the fixed left context is paid once per step, so amortising it over more output lowers the duty and removes seams together — the best row is also the cheapest, and varying lookahead alone was the wrong knob. Third, the entire constraint was an artefact: nothing had ever moved the models off the CPU. On the GPU a decode step falls from 830 ms to 148 ms, every shape in the table is idle-cheap, and the choice collapses to latency against quality — which is what a profile should have been all along. The operator picks `live`, `balanced` or `quality` per transmission; the page's option list is generated from the server's own table so the interface cannot drift from what was measured.

Measured on the air, 40 s per profile, all byte-exact with zero loss: first audio **3.6 s** (`live`), **4.8 s** (`balanced`), **8.5 s** (`quality`) after the talker starts, with end-to-end lag drift of −0.005, −0.014 and −0.005 s per second of speech — every profile holding steady where the mis-costed one drifted without bound.

The codec gets a name on the wire. While one codec was the only voice on the air it rode on the OPAQUE payload type and a receiver could only guess from the byte rate. Two small protocol changes make a second codec possible before it exists. The broadcast SYNC descriptor's payload-type registry — which had reserved Codec2 entries from the start — gains `BC_PT_LSCODEC_25`; the ptype is a label rather than a demultiplexer, so a

receiver that does not know the codec still stores the bytes exactly as before. And the capability record (§12.14) gains a codec bitmap: which voice codecs a station can *decode*, so a sender picks one the far end can play. The record grew from 10 to 11 bytes and the version deliberately did not move — an older record parses with the new field reading "never said", an older peer takes the prefix it knows, and both directions keep working, which is the one property a capability mechanism cannot trade away. The codec itself lives in the HOST, not the board — the board moves bytes — so the host declares its codecs over USB (a config key) and reads the peer's declaration back from the status frame, both under the same grow-only contract the temperature and rung fields established. Verified on the stand: the handshake completes with the codec bit crossing the air and arriving at the far host.

Group loss on a clean wire, and the wrong answer first. The transmissions kept reporting losses — 7.7 %, 11.8 %, 14.4 % of bytes — on a cross-wired pair at rung 12 carrying 250 bit/s. On a channel with tens of decibels of margin and a duty cycle this low that looks impossible, and chasing it produced two wrong answers before the right one. All three are reported, because how each wrong one survived its own round of measurement is the more useful part — and because the right one ended in the firmware, not in an explanation.

A lost unit is always the same shape: **one group, lost whole** — 30 or 35 bytes depending on how the group was sized, about a second of speech. Three independent measurements agree. The recording comes back short by exactly that much; the byte streams differ by exactly that much; and the sent/received dump names the offset:

```
sent 660 B, received 630 B, missing 30 B
streams agree for the first 310 B
⇒ 30 B missing at offset 310
   resumes at sent-offset 340; tail after that matches
```

The receiver never acquires that group's preamble. That is why nothing anywhere logged it: no walk is entered, so no failed-block event fires, and the loss is discovered only from the sequence gap in the *next* SYNC. A silent failure of this shape is what the sequence field exists for.

Three candidate causes were eliminated by counter rather than by argument, which is the only way to make progress on a fault that logs nothing. The receiving board's capture-FIFO overrun counter stayed at zero across every losing run — it was not dropping samples. Its `bc_rx_lost` beacon counter, which only the failed-block path increments, also stayed at zero — the frames were never *heard*, not mis-decoded. And the mid-stream broadcast statistics exchange, which does make the receiver key a frame, showed no correlation.

Two genuine host defects were found on the way and are worth having regardless of what caused the loss. The first is a race: the web server is threaded and the browser posted audio fire-and-forget, so the encoder and the broadcast sender ran on several threads at once — **27 collisions in a single 20 s transmission**, counted. The damaging one is the broadcast-open flag, read *before* the USB write and assigned *after* it, so two threads both see it clear and both send a non-continuation command; the firmware treats the second as a new broadcast, closing the one on the air and resetting the sequence. The second is the warm-up, which was declared complete the moment the far station *decoded* the message — while the acknowledgement it owed was still to be keyed, and the link is half duplex. Both are fixed. Neither was the cause.

How the wrong answer survived. The warm-up defect predicts losses at the *head* of a stream, and the first evidence agreed: 5 losses in 23 transmissions without the fix against 0 in 16 with it, Fisher $p = 0.066$, and three of four recoverable loss positions near the start. That is a plausible-looking result, and it was wrong twice over. The loss positions came from correlating audio envelopes, which is too blunt to locate a one-second gap in a re-synthesised waveform; and the two arms were measured an hour apart, so anything that drifted was attributed to the fix.

Repeating it properly settles it. Twelve *interleaved* pairs — the arms alternating within each pair, so drift cannot favour either — give 1 loss in 12 with the fix against 2 in 12 without, $p = 1.000$. There is no effect. The earlier sixteen clean transmissions were chance, and the interleaved design is what exposes that. By then the byte dumps were in place, and they close the argument outright: the gaps sit at 0.96 s, 6.88 s, 9.92 s, 10.88 s and 17.92 s into their streams — *uniformly spread*, with one transmission losing two. A warm-up that finished twenty seconds earlier cannot miss a group at 10.88 s.

The second wrong answer, and how it fell. The natural conclusion at this point — and the one a draft of this section briefly drew — was that 0.76 % is simply the acquisition statistics of non-acknowledged broadcast: §12.13 records 38 of 39 groups across an earlier campaign at rung 4, a 2.6 % loss rate, and the voice path was running three times better than that. Acquisition is a threshold decision, a high signal-to-noise ratio makes a miss unlikely rather than impossible, and nothing in a broadcast retries. Plausible, self-consistent, and wrong — the instinct that a clean channel should lose nothing was right all the way down.

What broke this one was refusing to accept "statistics" without a reproduction. A soak bench (`make bcsoak`) drives the firmware's own group build-and-walk through the streaming receiver at the voice shape — two-frame groups, random payload, random gaps, random sample-phase lead, thousands of groups — and the loss reproduced *on the host*, with no noise, no boards and no air: 28 of 720 frames, deterministic per seed, every missed group starting in the same ∼8-sample window of alignment mod 256. A rate that depends on sample phase is not statistics.

The actual defect. Traced on a failing seed: a tone field *entering* the detection window can produce one isolated above-threshold evaluation — metric $\sim 5 \times 10^7$ against $\sim 10^{11}$ for the aligned peak, at a coarse-frequency shift of -5 bins — whose above-threshold region dies immediately. The detector's region-argmax rule committed it, anchoring two blocks before the real preamble. The header then fails, and the recovery rearm does exactly what its comment has promised all along — "the true preamble, still in the ring, can be re-acquired" — except that nothing ever looked: the search only evaluated the *newest* block, so every window between the failed anchor and the present was skipped, and the failed ZC and header attempts had meanwhile consumed the true peak's samples. Three garbage attempts walk straight over the group. The comment described a recovery the machinery never performed.

Two fixes were built, and the difference between them is the useful part. The first made the machinery match the comment: rewind the search after a failure and recompute block summaries from the raw ring. It passed all 170 tests, the robustness gate and an 1800-group soak — and melted both boards within minutes of flashing: at EXTREME, failed acquisitions on plain noise are routine, each one re-scanned its whole excursion, and the capture FIFO never recovered (4.8 million dropped samples on the receiving board, which

decoded nothing at all). No host bench models EXTREME-scale CPU under a failure loop, which is exactly why it survived validation. The second fix is three lines and does strictly *less* work than the original code: the fast region-end commit now requires the metric to be *strong* ($\geq 64\times$ the threshold; real peaks measure 10^4–$10^6\times$), and a weak argmax takes the argmax-stability path the code already had — during which the true peak's evaluations arrive and replace it. The garbage is never committed at all. Prefer not committing garbage over recovering from having committed it.

Validation, in the order it should have happened the first time: the soak falls to 0 lost in 1800 groups; a gate-on/gate-off A/B at the EXTREME sensitivity knee decodes *identically* at every signal-to-noise point with zero false alarms in both arms; and on the reflashed boards, **24 of 24 live voice transmissions (∼900 frames) crossed with zero loss** — with the warm-up settle deliberately disabled in half of them, which is what being the actual cause looks like. Against the pre-fix 21.7 % per-transmission rate, 24 clean runs is $p = 0.003$. Whether the earlier rung-4 campaign's 2.6 % shares this cause cannot be settled retroactively, but the soak now guards the code path either way.

The methodological lesson is the expensive one and worth stating plainly. A fault that logs nothing invites the investigator to supply the missing narrative, and this one collected *three* narratives — a timing fix confirmed by two rounds of measurement, then "by design" backed by a plausible reference rate — before the real defect surfaced. Each fell to the same move: make the evidence harder to flatter. Interleave the arms so drift cannot masquerade as an effect; keep the raw artefact (the byte streams) rather than a statistic derived from it; and refuse "statistics" as an explanation until the statistic survives a reproduction attempt. All three were cheap, and none was done first. The one that found the bug — soak the exact code path at scale on the host — also produced the second regression, because host validation cannot see embedded CPU: the firmware that shipped was proven on the bench *and* the stand, and both proofs were necessary.

Why a lost group is survivable at all. The byte-alignment fix has a consequence worth stating separately, because it is the property that makes non-acknowledged voice viable. Once every group is a whole number of bytes carrying a whole number of tokens, a group that never arrives is a *gap* — the decoder resumes in step on the next one. Before the fix, the same lost group desynchronised everything after it. This is why a transmission measured at 7.7 % loss was still clearly intelligible: the listener hears a missing syllable, not a collapse. It is also the argument for broadcast over ARQ at this bitrate, made concrete: the failure mode of dropping a group is bounded, local, and about a second long.

This property remains load-bearing even now that the 0.76 % turned out to be a fixable defect: over a real fading channel, groups WILL be lost, no acknowledgement will recover them, and byte-aligned groups are what convert each loss into a missing syllable rather than a desynchronised remainder. The clean-wire campaign proved the tolerance works; the fix means it is no longer being spent on the receiver's own commit rule.

12.21.13 Where this leaves a voice mode

The bitrate fits, the compute fits, and the transport very nearly needs no change: the tokens are opaque bytes of exactly the size the streaming burst machinery already moves, and enrolment is two messages. A voice mode is a host program, like KISS and the IP tunnel, and §12.21.12 is that program working between two boards. It is also not

store-and-forward: §12.21.10 settles the codec at 1.32 s of algorithmic latency with flat memory, an order of magnitude below the link's own turnaround — provided the receiver decodes as it listens rather than at end-of-stream (§12.21.12), which is a property of the implementation and not of the codec, and which the first working receiver got wrong.

The single firmware change is worth recording, because the earlier draft of this section predicted none and was wrong for an instructive reason. Every host-side measurement was of the *codec*, and the codec was never the problem: the radio's own rule that a broadcast keys only full groups is correct for a file and is, for speech, three seconds of latency that no amount of chunk tuning can reach. A knob that a caller can decline (`BC_RT`) was the right shape, because the rule it waives is still right for everything else. The general lesson is the one this chapter keeps arriving at: a policy that is correct for the traffic it was written for becomes a defect when a new class of traffic arrives, and the fix is to make it a choice rather than to change it.

Two defects are characterised rather than fixed. The English accent of §12.21.11 is the cheaper one, and sits in the fine-tunable half. The other is unresolved: the failure *rate* on real shack audio. Three destroyed utterances in sixteen was measured on read speech one corpus away from training; the single Russian trial suggests the degradation is graceful there, but one utterance is an anecdote and not a rate. That number, and not the bitrate, decides the question — because a transmission that arrives as fluent nonsense with every protocol counter healthy is worse than one that fails.

Fine-tuning is the obvious remedy and splits usefully. The vocoder (31.3 M parameters) can be fine-tuned without touching the codebook, which means **the bitstream does not change**: a peer running stock weights still decodes the transmission, a peer with the tuned weights decodes it better, and no capability bit or protocol revision is involved. Retraining the encoder and quantiser would change what the tokens mean and *is* a protocol change, requiring both ends to match. Neither fixes the identity property, which is the architecture working as designed.

13 Discussion

13.1 What Binds Performance Now

The front end, not the code. The clearest lesson of the coding work is that FEC choice stopped mattering before code quality did: LDPC's 0.7 dB AWGN advantage compresses to parity end-to-end because the demodulator's LLR quality is the binding constraint. The monotone recalibration recovered 1.5–2 dB, but a standards-grade LDPC matrix (802.11n/NR-style [11]) *plus* fully calibrated LLRs would be needed to realise the remaining ∼2 dB. Any further sensitivity work should start at the LLR front end, not the decoder.

Synchronization is the sensitivity frontier. Both the original reproduction gap and the fade-loss analysis point the same way: at the edge, frames are lost to detection and timing, not to decoding. HARQ helps only in the noise-limited regime for exactly this reason — a sync-level loss leaves no LLRs to combine. The two-board stand made the same point at the *other* end of the SNR range: broadcast, which cannot retransmit, lost one group in 39 on a wire reporting +24.7 dB — a missed acquisition with 25 dB of margin, invisible to every acknowledged mode because ARQ had been quietly repairing it (Section 12.13).

Physics caps EXTREME. The 64× mode's 0.69 s coherent integration assumes a channel coherent over that window ($\lesssim$0.1 Hz Doppler spread). This is a physical ceiling, not an implementation one, and it is why the mode ladder — rather than ever-longer integration — is the right architecture.

Feedback latency at the floor. Once a direction falls to EXTREME, the adaptation loop period is bounded by 25–45 s frame air times. The controller is designed around this (fade-aware filtering, loss fallback, silence decay), but no tuning can make a 7.8 bit/s control loop fast.

13.2 Limitations

The channel model, while faithful to the article (and extended with Rayleigh fading and a physical LO model), is still a simulation; the measured sensitivities should be planned around with ∼3 dB of fading margin (−14 . . . −15 dB for EXTREME). The 16-QAM rungs run without LLR recalibration (the map is BPSK-trained); a modulation-specific map is straightforward but unmeasured. The LDPC matrix is a from-scratch IRA construction, not a standards matrix. The project also has no over-the-air validation yet — the WAV recordings are transceiver-ready precisely to enable it. The voice evaluation of §12.21 is bounded the same way: the codec's catastrophic-failure rate was measured on read speech, not on shack audio, and one non-English trial is an anecdote rather than a rate.

13.3 Future Work

In rough order of expected value: over-the-air testing with real SSB rigs using the recorded WAVs; a standards-grade LDPC matrix plus QAM-specific LLR recalibration; porting the fixed receiver's slew-limited tracker back to the float model (it is measurably better); pilot reduction or 2-D channel estimation to reclaim carrier capacity; per-carrier bit-loading on the measured channel estimate; frame aggregation in the ARQ to amortise preamble overhead at high rungs; a host-side voice mode over the 250 bit/s codec evaluated in §12.21 (design in §12.21.7), whose remaining question is a failure rate rather than a bitrate, and whose enrolment cost would collapse from kilobytes to tens of bytes if the vocoder's prompt path were retrained to take a global speaker code (§12.21.6); a 46.875 Hz-spacing variant for very stable channels; and shrinking EXTREME's frequency-search range after AFC netting converges (compute reduction on embedded targets).

14 Conclusion

The project set out to reproduce a published amateur-radio OFDM PHY layer and ended with a complete adaptive communication system. The reproduction itself is closed: 44/44 worked examples bit-exact, the BER/PER figures reproduced, and the sensitivity gap traced to three specific synchronization defects and eliminated to within 0.3 dB of a genie-synchronized bound. On that foundation, the waveform was extended an order of magnitude in each direction — down to −17.9 dB and 7.8 bit/s (78 % of Shannon capacity at −20 dB) and up to +4.7 dB and 1059 bit/s with 16-QAM — and wrapped in a link layer that measures everything it can observe and feeds all of it back: receiver-driven rate adaptation over a 13-rung measured ladder, CRC-gated HARQ from exported LLRs,

learned per-rung sensitivity offsets, and closed-loop AFC netting of the stations' carrier frequencies.

Two results deserve emphasis beyond the feature list. First, the measured LLR recalibration — 1.5–2 dB from correcting the *shape* of the demodulator's confidence, at zero over-the-air cost — illustrates how much sensitivity can hide between a correct demodulator and a correct decoder. Second, the fixed-point receiver matching and then beating the floating-point chain at the sensitivity edge shows that an RTL-oriented implementation need not be a degradation; the discipline it forces (bounded scales, division-free estimation, a tracker instead of a polynomial fit) produced the better algorithm.

Everything claimed here is regenerable from the repository: the regression suites are self-verifying, every sensitivity figure has its sweep script and JSON dump, and the final demonstration is a WAV file any copy of the receiver decodes blind. The natural next step is the one simulation cannot take: the same frames over a real HF path.

References

[1] VARA HF — signal identification wiki. https://www.sigidwiki.com/wiki/VARA_HF. accessed 2026-08-17.

[2] bashkirtsevich. Development of a digital amateur-radio protocol based on OFDM: The PHY layer (in Russian). Habr, 2024. https://habr.com/ru/articles/1070804/.

[3] David Chase. Code combining — a maximum-likelihood decoding approach for combining an arbitrary number of noisy packets. *IEEE Transactions on Communications*, 33(5):385–393, 1985.

[4] Jinghu Chen, Ajay Dholakia, Evangelos Eleftheriou, Marc P. C. Fossorier, and Xiao-Yu Hu. Reduced-complexity decoding of LDPC codes. *IEEE Transactions on Communications*, 53(8):1288–1299, 2005.

[5] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, et al. WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.

[6] David C. Chu. Polyphase codes with good periodic correlation properties. *IEEE Transactions on Information Theory*, 18(4):531–532, 1972.

[7] Steven Franke, Bill Somerville, and Joe Taylor. The FT4 and FT8 communication protocols. *QEX*, pages 7–17, July/August 2020.

[8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proc. 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[9] Yiwei Guo, Zhihan Li, Chenpeng Du, Hankun Wang, Xie Chen, and Kai Yu. LSCodec: Low-bitrate and speaker-decoupled discrete speech codec. In *Proc. Interspeech*, 2025. https://arxiv.org/abs/2410.15764; inference code at https://github.com/X-LANCE/LSCodec-Inference.

[10] Yiwei Guo, Zhihan Li, Hankun Wang, Bohan Li, Chongtian Shao, Hanglei Zhang, et al. Recent advances in discrete speech tokens: A review, 2025. https://arxiv.org/abs/2502.06490.

[11] IEEE. *IEEE Std 802.11-2020: Wireless LAN MAC and PHY Specifications*. IEEE, 2021.

[12] ITU-R. Recommendation P.372-16: Radio noise. Technical report, International Telecommunication Union, 2022.

[13] Hui Jin, Aamod Khandekar, and Robert McEliece. Irregular repeat–accumulate codes. In *Proceedings of the 2nd International Symposium on Turbo Codes and Related Topics*, pages 1–8, 2000.

[14] Zeqian Ju, Yuancheng Wang, Kai Shen, Xu Tan, Detai Xin, Dongchao Yang, et al. NaturalSpeech 3: Zero-shot speech synthesis with factorized codec and diffusion models. In *Proc. ICML*, 2024. FACodec; https://arxiv.org/abs/2403.03100.

[15] Donald J. Newman. An L^1 extremal problem for polynomials. *Proceedings of the American Mathematical Society*, 16(6):1287–1290, 1965.

[16] Yong Ren, Tao Wang, Jiangyan Yi, Le Xu, Jianhua Tao, Chuyuan Zhang, et al. Fewer-token neural speech codec with time-invariant codes. In *Proc. IEEE ICASSP*, 2024. https://arxiv.org/abs/2310.00014.

[17] Timothy M. Schmidl and Donald C. Cox. Robust frequency and timing synchronization for OFDM. *IEEE Transactions on Communications*, 45(12):1613–1621, 1997.

[18] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948.

[19] Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967.

[20] Youqiang Zheng, Weiping Tu, Yueteng Kang, Jie Chen, Yike Zhang, Li Xiao, et al. FreeCodec: A disentangled neural speech codec with fewer tokens, 2024. https://arxiv.org/abs/2412.01053.

Appendices

The four appendices below collect verbatim tool output that would interrupt the main narrative. Appendix A reproduces the output of the bit-exact article-reproduction suite (`verify_article.py`) referenced in Section 11; Appendix B reproduces the output of the fixed-point validation suite (`fixed_point.py`) referenced in Section 10; Appendix C reproduces the blind-decoded transcript of the recorded two-station demonstration session (`demo_wav.py`) referenced in Section 11; and Appendix D reproduces the transcript of the same demonstration running entirely on the fixed-point pipeline at +5 dB, where the ladder climbs into 16-QAM.

A Article-Reproduction Suite Output

Verbatim output of `./venv/bin/python experiments/verify_article.py`: the 44 bit-exact checks against the article's worked examples — numerology constants, Base38 callsign and QTH-locator encodings, CRC-8/CRC-16 values, convolutional-code outputs at all four rates, interleaver/scrambler sequences, and Zadoff–Chu sequence properties.

```
[PASS] subcarrier spacing = 93.75 Hz
[PASS] channel bins = 3..25 (23 carriers)
[PASS] center bin = 14 (1312.5 Hz)
[PASS] pilot bins = [3, 6, 10, 14, 17, 21, 25]
[PASS] data carriers = 16
[PASS] cyclic prefix = 32 samples (25%)
[PASS] tiled symbol = CP + 4*128 samples
[PASS] newman tone bins = [8, 12, 16, 20]
[PASS] newman shifted bins = [10, 14, 18, 22]
[PASS] callsign_to_int('R9FEU') = 4671407026402144
[PASS] int_to_callsign round-trip
[PASS] qth6_to_int('LO88CA') = 12261936
[PASS] int_to_qth6 round-trip
[PASS] crc8_lte example = 0b10101101
[PASS] crc16 example input is 236 bits
[PASS] crc16_ccitt example = 0b1001001100011111
[PASS] crc16 example payload decodes to article text
[PASS] crc16 example equals Data(reserved=12375, ...).encode() body
[PASS] Header(ver=1, BEACON, BPSK, R13, len=90) bits
[PASS] Header round-trip
[PASS] Beacon packet size = 90 bits
[PASS] Beacon callsign field bits
[PASS] Beacon QTH field bits
[PASS] Beacon round-trip
[PASS] Data packet size = 20+216+16 bits
[PASS] Data round-trip
[PASS] CC impulse response (rate 1/3)
[PASS] CC codec constants
[PASS] header CC rate 1/3 (93 bits)
[PASS] header CC rate 1/2 (62 bits)
[PASS] header CC rate 2/3 (47 bits)
[PASS] header CC rate 3/4 (42 bits)
[PASS] Viterbi round-trip CCLTEBPSK
[PASS] Viterbi round-trip CCLTEBPSK_12
[PASS] Viterbi round-trip CCLTEBPSK_23
[PASS] Viterbi round-trip CCLTEBPSK_34
[PASS] Viterbi corrects 8/93 hard errors (rate 1/3)
[PASS] interleave 32 bits over 16 carriers
[PASS] deinterleave inverts interleave
[PASS] scrambler example sequence
[PASS] scramble is an involution
[PASS] descramble (soft) recovers source
[PASS] ZC sequence constant amplitude
[PASS] ZC periodic ACF is a delta function

44 passed, 0 failed
```

B Fixed-Point Validation Suite Output

Verbatim output of `./venv/bin/python experiments/fixed_point.py`: primitive quality checks (FFT, Hilbert, NCO, CORDIC, integer Viterbi), the fixed/float TX×RX cross-matrix, the PER comparison table against the float receiver, and the LDPC / 16-QAM / calibration / HARQ / ROBUST / EXTREME decode checks referenced in Section 10.

```
[PASS] fixed FFT SQNR > 55 dB  (59.1 dB)
[PASS] Hilbert FIR SQNR > 55 dB  (60.7 dB)
[PASS] NCO derotation residual < -60 dBc  (-67.5 dBc)
[PASS] CORDIC atan2 error < 0.001 rad  (1.00001)
[PASS] CORDIC magnitude error < 0.1%  (19999)
[PASS] integer Viterbi corrects noisy 6-bit LLRs
[PASS] fixed TX waveform correlation > 0.999  (0.99998)
[PASS] clean fixed TX -> fixed RX
[PASS] clean float TX -> fixed RX
[PASS] clean fixed TX -> float RX

NORMAL PER, fixed RX vs float RX, 30 packets/point:
  -6 dB: fixed 26/30   float 30/30
  -7 dB: fixed 27/30   float 29/30
  -8 dB: fixed 26/30   float 23/30
[PASS] fixed RX within ~1 dB of float at -7 dB  (see table above)
[PASS] fixed RX decodes LDPC (ver=2) frames
[PASS] fixed RX decodes 16-QAM frames
[PASS] fixed TX LDPC (ver=2) -> fixed RX
[PASS] fixed TX LDPC -> float RX
[PASS] fixed TX 16-QAM -> fixed RX
[PASS] fixed TX 16-QAM -> float RX
[PASS] integer SNR estimate within 2 dB  (errs 0.6/0.8 dB at -7/0)
[PASS] calibrated mode (alpha fit + reliability ROM) decodes
[PASS] HARQ chase combining (complementary erasures)
[PASS] repacked LC word roundtrips fixed chain
[PASS] fixed TX emits ver=1 (conv) headers by default
[PASS] fixed ROBUST decode @ -11 dB  (cfo +29.4 Hz)
[PASS] fixed EXTREME decode @ -17 dB  (cfo +33.8 Hz)

24 passed, 0 failed
```

C System-Demonstration Transcript

Verbatim output of `./venv/bin/python experiments/demo_wav.py`: the two-station session over the fading SSB RF chain (Section 11). The first block is the simplex scheduler's air-time log as heard by the passive monitor (per-frame carrier snapshots include each station's LO drift and AFC trims); the second block is the *blind-decoded* transcript recovered from the monitor's WAV audio alone — mode, modulation and rate from the frame itself, measured CFO, and the unpacked link-control word (seq/ack/rung-request/SNR-report) for every decodable frame. A's and B's distinct carrier fingerprints (≈ -18 and ≈ -22 Hz at the monitor after netting) identify the transmitter of every frame. Encrypted-looking payloads are mid-message fragments of the bulk transfer; `b'\x00'` frames are ACK-only.

```
expected CFOs at monitor: A frames +28.4 Hz, B frames -71.0 Hz
expected station-to-station CFO A->B: +99.4 Hz (AFC netting active)
```

```
t=  91.1s  A->B phase complete, B starts its reply traffic
final trims: A -64.0 Hz, B +36.0 Hz; residual A->B offset +5.0 Hz
saved /home/ivan/projects/ofdm_transceiver_proto/results/
   system_demo_timeline.png
saved /home/ivan/projects/ofdm_transceiver_proto/results/system_demo.wav
     (113.6 s, 2663 KiB, 40 frames)
  t=   0.0s  A transmits  17.5 s (carrier  +42.6 Hz)
  t=  17.5s  B transmits  16.8 s (carrier  -57.1 Hz)
  t=  34.3s  A transmits   2.1 s (carrier   -4.4 Hz)
  t=  36.5s  B transmits   0.8 s (carrier   -5.5 Hz)
  t=  37.4s  A transmits   1.1 s (carrier  -32.3 Hz)
  t=  38.5s  B transmits   0.8 s (carrier   -5.6 Hz)
  t=  39.4s  A transmits   1.2 s (carrier  -20.2 Hz)
  t=  40.7s  B transmits   0.8 s (carrier  -21.6 Hz)
  t=  41.5s  A transmits   1.4 s (carrier  -12.2 Hz)
  t=  49.6s  A transmits   1.2 s (carrier  -11.9 Hz)
  t=  50.9s  B transmits   0.8 s (carrier  -21.8 Hz)
  t=  51.7s  A transmits   1.4 s (carrier  -19.8 Hz)
  t=  62.0s  A transmits   1.2 s (carrier  -19.5 Hz)
  t=  68.2s  A transmits   1.8 s (carrier  -19.4 Hz)
  t=  70.0s  B transmits   0.8 s (carrier  -22.2 Hz)
  t=  70.8s  A transmits   1.4 s (carrier  -19.3 Hz)
  t=  72.3s  B transmits   0.8 s (carrier  -22.2 Hz)
  t=  73.1s  A transmits   1.4 s (carrier  -19.2 Hz)
  t=  83.4s  A transmits   1.2 s (carrier  -18.9 Hz)
  t=  84.7s  B transmits   0.8 s (carrier  -22.5 Hz)
  t=  85.5s  A transmits   1.4 s (carrier  -18.8 Hz)
  t=  87.0s  B transmits   0.8 s (carrier  -22.5 Hz)
  t=  87.8s  A transmits   1.4 s (carrier  -18.8 Hz)
  t=  89.3s  B transmits   0.8 s (carrier  -22.6 Hz)
  t=  90.1s  A transmits   1.0 s (carrier  -18.7 Hz)
  t=  91.2s  B transmits   1.2 s (carrier  -22.6 Hz)
  t=  92.5s  A transmits   0.8 s (carrier  -18.6 Hz)
  t=  93.4s  B transmits   1.0 s (carrier  -22.7 Hz)
  t=  94.4s  A transmits   0.8 s (carrier  -18.6 Hz)
  t=  95.3s  B transmits   1.1 s (carrier  -22.7 Hz)
  t=  96.5s  A transmits   0.8 s (carrier  -18.5 Hz)
  t=  97.4s  B transmits   1.2 s (carrier  -22.7 Hz)
  t=  98.7s  A transmits   0.8 s (carrier  -18.4 Hz)
  t=  99.6s  B transmits   1.2 s (carrier  -22.8 Hz)
  t= 100.9s  A transmits   0.8 s (carrier  -18.4 Hz)
  t= 101.8s  B transmits   1.2 s (carrier  -22.8 Hz)
  t= 108.6s  B transmits   1.1 s (carrier  -23.0 Hz)
  t= 109.8s  A transmits   0.8 s (carrier  -18.1 Hz)
  t= 110.7s  B transmits   1.0 s (carrier  -23.0 Hz)
  t= 111.7s  A transmits   0.8 s (carrier  -18.0 Hz)

   t, s mode     rate           cfo seq ack req_rung -> speed selection
     snr rpt  payload
    5.8 EXTREME  BPSK 1/3    +28.4Hz   1   0 rung 0 (8 bit/s)
      -24.0dB  b'CQ'
   23.3 EXTREME  BPSK 1/3    -71.4Hz   0   1 rung 6 (235 bit/s)
       -2.0dB  b'\x00'
   34.7 NORMAL   BPSK 1/2    -18.6Hz   2   0 rung 8 (471 bit/s)
       +0.0dB  b'MSG DE R9FEU: QTH...'
   36.9 NORMAL   QPSK 1/2    -20.1Hz   0   2 rung 6 (235 bit/s)
       -2.0dB  b'\x00'
```

```
   37.8 NORMAL   QPSK 1/3    -46.3Hz   3   0 rung 8 (471 bit/s)
       +0.0dB  b' IVAN 73'
   38.9 NORMAL   QPSK 1/2    -19.8Hz   0   3 rung 8 (471 bit/s)
       +0.0dB  b'\x00'
   39.8 NORMAL   QPSK 2/3    -34.7Hz   0   0 rung 9 (529 bit/s)
       +0.0dB  b'!\x87\xeak\xde\xa6\xc8\xde \x04\x07\xf6D\xdalI\xbe...'
   50.0 NORMAL   QPSK 2/3    -26.4Hz   1   0 rung 9 (529 bit/s)
       +0.0dB  b'\xaaAy\x0c\xc7\x85*\x83\xa4\xbb\xaaP\xb7l\x1bO\xea...'
   51.3 NORMAL   QPSK 2/3    -35.6Hz   0   1 rung 8 (471 bit/s)
       +0.0dB  b'\x00'
   52.1 NORMAL   QPSK 1/2    -33.9Hz   2   0 rung 9 (529 bit/s)
       +0.0dB  b'B\xdc\x17b\x94\xb8\n\x9f\x11\xeb\xac\tX\x03*t\xce...'
   62.4 NORMAL   QPSK 2/3    -34.1Hz   2   0 rung 9 (529 bit/s)
       +0.0dB  b'B\xdc\x17b\x94\xb8\n\x9f\x11\xeb\xac\tX\x03*t\xce...'
   68.6 NORMAL   QPSK 1/3    -30.4Hz   2   0 rung 9 (529 bit/s)
       +0.0dB  b'B\xdc\x17b\x94\xb8\n\x9f\x11\xeb\xac\tX\x03*t\xce...'
   70.4 NORMAL   QPSK 2/3    -33.9Hz   0   2 rung 8 (471 bit/s)
       +0.0dB  b'\x00'
   71.2 NORMAL   QPSK 1/2    -35.6Hz   3   0 rung 9 (529 bit/s)
       +0.0dB  b'\xef\x88\xbf\xb8O%|\xf0\x84\xd8\xe6\n\xf2\xad\x10;\x1d
   ...'
   72.7 NORMAL   QPSK 2/3    -36.5Hz   0   3 rung 8 (471 bit/s)
       +0.0dB  b'\x00'
   73.5 NORMAL   QPSK 1/2    -33.5Hz   0   0 rung 9 (529 bit/s)
       +0.0dB  b'C\x9b\xa3\xd4.\x08\xe3J%\xb2\xb9\xfb4\xba9Y\xf9...'
   83.8 NORMAL   QPSK 2/3    -32.7Hz   0   0 rung 9 (529 bit/s)
       +0.0dB  b'C\x9b\xa3\xd4.\x08\xe3J%\xb2\xb9\xfb4\xba9Y\xf9...'
   85.1 NORMAL   QPSK 2/3    -37.5Hz   0   0 rung 8 (471 bit/s)
       +0.0dB  b'\x00'
   88.2 NORMAL   QPSK 1/2    -31.7Hz   2   0 rung 9 (529 bit/s)
       +2.0dB  b'?\xf7\x87\x83\x9dG\x16\xcc${\xd3\xe0W\xc8\x96\xf7\xbc
   ...'
   91.6 NORMAL   QPSK 2/3    -36.3Hz   1   3 rung 8 (471 bit/s)
       -2.0dB  b'R R TNX MSG DE UB...'
   92.9 NORMAL   QPSK 1/2    -31.2Hz   3   1 rung 9 (529 bit/s)
       +0.0dB  b'\x00'
   93.8 NORMAL   QPSK 3/4    -37.0Hz   2   3 rung 8 (471 bit/s)
       +0.0dB  b'9 QTH KP50 73'
   94.8 NORMAL   QPSK 1/2    -33.2Hz   3   2 rung 9 (529 bit/s)
       +0.0dB  b'\x00'
   95.7 NORMAL   QPSK 3/4    -35.6Hz   3   3 rung 8 (471 bit/s)
       -2.0dB  b'N\xcc@"\x10\xfa\xe9 gz\r\xcc\x8a\xac\xd0\x7f|...'
   97.8 NORMAL   QPSK 2/3    -36.5Hz   0   3 rung 8 (471 bit/s)
       +0.0dB  b'\xb6\xb0)X\x07\x8d\xf1I|\xdf{\xde%T7\xc8f...'
   99.1 NORMAL   QPSK 1/2    -32.5Hz   3   0 rung 9 (529 bit/s)
       +0.0dB  b'\x00'
  109.0 NORMAL   QPSK 3/4    -36.7Hz   2   3 rung 8 (471 bit/s)
       -2.0dB  b'$K\xdeVUv^;9O%\xdbt\xea\x82\xd5\x8a...'
  110.2 NORMAL   QPSK 1/2    -32.8Hz   3   2 rung 9 (529 bit/s)
       +0.0dB  b'\x00'
  111.1 NORMAL   QPSK 2/3    -36.9Hz   3   3 rung 8 (471 bit/s)
       -2.0dB  b'w\xa3zF\xffY\x10\xdb(\xe0Q#'
  112.1 NORMAL   QPSK 1/2    -30.7Hz   3   3 rung 9 (529 bit/s)
       +0.0dB  b'\x00'

decoded 30/40 frames from the noisy/fading WAV (fade dropouts are
   expected -- the stations' ARQ recovered them)
```

D Fixed-Point-Pipeline Demonstration Transcript

Verbatim output of `./venv/bin/python experiments/demo_wav.py --phy fixed --snr 5`: the +5 dB two-station session with both stations and the passive monitor running the *full fixed-point pipeline* (Section 10) — fixed transmitters, per-mode fixed receivers with blind mode auto-detection, the integer SNR estimator driving rate adaptation, and integer HARQ. The blind transcript shows the climb from the EXTREME bootstrap through the QPSK rungs into 16-QAM (rungs 9–10 on the air, requests up to rung 11), with the integer SNR reports in the link-control words and the AFC-netted carrier fingerprints in the CFO column.

```
expected CFOs at monitor: A frames +28.4 Hz, B frames -71.0 Hz
expected station-to-station CFO A->B: +99.4 Hz (AFC netting active)

t=  61.5s  A->B phase complete, B starts its reply traffic
final trims: A -56.0 Hz, B +40.0 Hz; residual A->B offset +7.6 Hz
saved /home/ivan/projects/ofdm_transceiver_proto/results/
   system_demo_fixed_qam16_timeline.png
saved /home/ivan/projects/ofdm_transceiver_proto/results/
   system_demo_fixed_qam16.wav  (85.3 s, 1999 KiB, 40 frames)
  t=   0.0s  A transmits  17.5 s (carrier  +42.6 Hz)
  t=  17.5s  B transmits  16.8 s (carrier  -57.1 Hz)
  t=  34.3s  A transmits   1.2 s (carrier   -4.4 Hz)
  t=  35.6s  B transmits   0.8 s (carrier   -5.5 Hz)
  t=  36.4s  A transmits   0.9 s (carrier  -32.3 Hz)
  t=  37.3s  B transmits   0.8 s (carrier   -5.5 Hz)
  t=  38.1s  A transmits   1.1 s (carrier  -20.3 Hz)
  t=  39.3s  B transmits   0.8 s (carrier  -17.6 Hz)
  t=  40.1s  A transmits   1.2 s (carrier  -12.2 Hz)
  t=  41.4s  B transmits   0.8 s (carrier  -17.6 Hz)
  t=  42.2s  A transmits   1.2 s (carrier  -12.1 Hz)
  t=  43.5s  B transmits   0.8 s (carrier  -17.7 Hz)
  t=  44.3s  A transmits   1.2 s (carrier  -12.1 Hz)
  t=  45.6s  B transmits   0.8 s (carrier  -17.7 Hz)
  t=  46.4s  A transmits   1.2 s (carrier  -12.0 Hz)
  t=  47.7s  B transmits   0.8 s (carrier  -17.8 Hz)
  t=  48.5s  A transmits   1.2 s (carrier  -11.9 Hz)
  t=  49.8s  B transmits   0.8 s (carrier  -17.8 Hz)
  t=  50.6s  A transmits   1.2 s (carrier  -11.9 Hz)
  t=  51.9s  B transmits   0.8 s (carrier  -17.8 Hz)
  t=  58.5s  A transmits   1.1 s (carrier  -11.6 Hz)
  t=  59.7s  B transmits   0.8 s (carrier  -18.0 Hz)
  t=  60.5s  A transmits   1.0 s (carrier  -11.6 Hz)
  t=  61.5s  B transmits   1.1 s (carrier  -18.0 Hz)
  t=  68.2s  B transmits   1.1 s (carrier  -18.2 Hz)
  t=  70.3s  A transmits   0.9 s (carrier  -11.3 Hz)
  t=  71.3s  B transmits   1.1 s (carrier  -18.2 Hz)
  t=  72.5s  A transmits   0.8 s (carrier  -11.2 Hz)
  t=  73.3s  B transmits   0.9 s (carrier  -18.3 Hz)
  t=  74.2s  A transmits   0.8 s (carrier  -11.2 Hz)
  t=  75.0s  B transmits   1.0 s (carrier  -18.3 Hz)
  t=  76.1s  A transmits   0.8 s (carrier  -11.1 Hz)
  t=  76.9s  B transmits   1.0 s (carrier  -18.3 Hz)
  t=  78.0s  A transmits   0.8 s (carrier  -11.1 Hz)
  t=  78.8s  B transmits   1.0 s (carrier  -18.4 Hz)
  t=  79.9s  A transmits   0.8 s (carrier  -11.0 Hz)
  t=  80.7s  B transmits   1.0 s (carrier  -18.4 Hz)
```

```
t=  81.8s  A transmits   0.8 s (carrier  -10.9 Hz)
t=  82.6s  B transmits   0.9 s (carrier  -18.5 Hz)
t=  83.5s  A transmits   0.8 s (carrier  -10.9 Hz)

 t, s mode     rate           cfo seq ack req_rung -> speed selection
   snr rpt  payload
 23.3 EXTREME  BPSK 1/3    -71.4Hz   0   1 rung 9 (529 bit/s)
     +2.0dB  b'\x00'
 34.7 NORMAL   QPSK 2/3    -18.6Hz   2   0 rung 10 (706 bit/s)
     +4.0dB  b'MSG DE R9FEU: QTH...'
 38.5 NORMAL   QPSK 3/4    -33.7Hz   0   0 rung 10 (706 bit/s)
     +4.0dB  b'!\x87\xeak\xde\xa6\xc8\xde \x04\x07\xf6D\xdalI\xbe...'
 39.7 NORMAL   QPSK 3/4    -32.7Hz   0   0 rung 9 (529 bit/s)
     +2.0dB  b'\x00'
 48.1 NORMAL   QPSK 2/3    -31.6Hz   0   0 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 48.9 NORMAL   QPSK 2/3    -26.7Hz   1   0 rung 9 (529 bit/s)
     +2.0dB  b'P\xce>m\xfa\xe9\xbf\x90\xb4[\x93\xf9\xf7|!\x97L...'
 50.2 NORMAL   QPSK 2/3    -31.6Hz   0   1 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 51.0 NORMAL   QPSK 2/3    -25.0Hz   2   0 rung 9 (529 bit/s)
     +2.0dB  b'?\xf7\x87\x83\x9dG\x16\xcc${\xd3\xe0W\xc8\x96\xf7\xbc
 ...'
 52.3 NORMAL   QPSK 2/3    -32.0Hz   0   2 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 58.9 NORMAL   QPSK 3/4    -26.9Hz   2   0 rung 9 (529 bit/s)
     +2.0dB  b'?\xf7\x87\x83\x9dG\x16\xcc${\xd3\xe0W\xc8\x96\xf7\xbc
 ...'
 60.1 NORMAL   QPSK 2/3    -33.5Hz   0   2 rung 9 (529 bit/s)
     +0.0dB  b'\x00'
 60.9 NORMAL   QPSK 2/3    -26.0Hz   3   0 rung 10 (706 bit/s)
     +4.0dB  b'!z\xfczI\x03\x1d\xfcG\x03\xe2'
 61.9 NORMAL   QPSK 3/4    -32.4Hz   1   3 rung 9 (529 bit/s)
     +2.0dB  b'R R TNX MSG DE UB...'
 68.6 NORMAL   QPSK 3/4    -31.5Hz   1   3 rung 9 (529 bit/s)
     +2.0dB  b'R R TNX MSG DE UB...'
 70.7 NORMAL   QPSK 3/4    -26.3Hz   3   0 rung 10 (706 bit/s)
     +4.0dB  b'!z\xfczI\x03\x1d\xfcG\x03\xe2'
 71.7 NORMAL   QPSK 3/4    -32.6Hz   1   3 rung 9 (529 bit/s)
     +2.0dB  b'R R TNX MSG DE UB...'
 72.9 NORMAL   QPSK 2/3    -25.3Hz   3   1 rung 10 (706 bit/s)
     +2.0dB  b'\x00'
 73.7 NORMAL   QAM16 1/2   -32.7Hz   2   3 rung 9 (529 bit/s)
     +2.0dB  b'9 QTH KP50 73'
 74.6 NORMAL   QPSK 2/3    -24.0Hz   3   2 rung 10 (706 bit/s)
     +2.0dB  b'\x00'
 76.5 NORMAL   QPSK 2/3    -24.9Hz   3   3 rung 11 (941 bit/s)
     +6.0dB  b'\x00'
 77.3 NORMAL   QAM16 1/2   -31.8Hz   0   3 rung 9 (529 bit/s)
     +2.0dB  b'\xb6\xb0)X\x07\x8d\xf1I|\xdf{\xde%T7\xc8f...'
 80.3 NORMAL   QPSK 2/3    -23.6Hz   3   1 rung 11 (941 bit/s)
     +6.0dB  b'\x00'
 81.1 NORMAL   QAM16 1/2   -32.0Hz   2   3 rung 10 (706 bit/s)
     +4.0dB  b'$K\xdeVUv^;9O%\xdbt\xea\x82\xd5\x8a...'
 82.2 NORMAL   QPSK 3/4    -25.0Hz   3   2 rung 11 (941 bit/s)
     +6.0dB  b'\x00'
 83.0 NORMAL   QAM16 1/2   -31.6Hz   3   3 rung 10 (706 bit/s)
     +4.0dB  b'w\xa3zF\xffY\x10\xdb(\xe0Q#'
```

```
   83.9 NORMAL    QPSK 3/4     -26.6Hz    3    3 rung 11 (941 bit/s)
       +6.0dB  b'\x00'

decoded 26/40 frames from the noisy/fading WAV (fade dropouts are
   expected -- the stations' ARQ recovered them)
```